

Tensor networks as surrogate models of training dynamics

Research note, October 9, 2026

Abstract

This note motivates tree tensor networks (TTNs) as surrogate models of the training dynamics of deep neural networks, and makes the underlying mathematics precise. We define tensor networks, TTNs, their edge matricizations, hierarchical ranks and hierarchical singular value decomposition (HSVD), and compare the relevant notions of tensor rank. A deep network with polynomial activations is exactly a weight-tied TTN evaluated on copies of the homogenized input. This suggests reading the HSVD of that TTN as a gauge-invariant description of the hierarchical features learned by the network, and its hierarchical ranks as a measure of learning capacity; we make both notions precise and discuss their limits. Taylor-expanding the activations of a smooth network at a training checkpoint gives a local TTN surrogate, whose fidelity to the original training dynamics is controlled by prediction and Jacobian errors. We state a global and a local surrogate hypothesis, the open questions they raise, and the experiments that would test them.

Contents

1	Introduction	2
2	Theoretical background	5
2.1	Tensors, multilinear maps and matricizations	5
2.2	Tensor networks and tree tensor networks	6
2.3	Edge cuts, bond dimensions and hierarchical ranks	7
2.4	The hierarchical singular value decomposition	8
2.5	Notions of rank	10
2.6	Deep polynomial networks as tree tensor networks	11
2.6.1	Monomial activations: tied TTNs on copies of the input	12
2.6.2	General polynomial activations: homogenization	13
2.6.3	Hierarchical features and learning capacity	15
2.7	Surrogate model of a deep neural network	18
3	Training dynamics of TTNs	22
4	How to compare TTNs and DNNs	22
4.1	The teacher–student setting	23
4.2	The data-weighted HSVD of the teacher	25
4.3	Atoms: identity and strength	30
4.4	Output observables: has the predictor acquired an atom?	31
4.5	The functional HSVD of a student	34
4.6	Teachers with repeated inputs	37

4.7	Representation observables: where is an atom represented?	39
4.8	Use and causal reliance	41
4.9	Observables for TTN students	44
4.10	Acquisition times, comparison of students, and estimation	45
4.11	Summary	47
5	Experiments	48
6	Experiments (slop)	50

1 Introduction

In this research note I motivate tree tensor networks (TTNs) as a surrogate model of the training dynamics of deep neural networks (DNNs). I also suggest experiments to check whether TTNs are a promising surrogate for the training dynamics of more realistic neural networks. Section 2 develops the mathematical background, and the experiments are collected in Section 6.

Why training dynamics? To be brief, my current position is that AI safety is bottlenecked by understanding. Interpretability as a field remains shallow, in the sense that current methods explain little about the behaviour of current models, or are unlikely to scale. One of the main reasons for this state of affairs is that we are still confused about fundamental questions on how neural networks work. One such question is *what kind of structure neural networks learn*. Extracting this structure from a network at the end of training seems extremely challenging. The bet behind studying training dynamics is that internal structure decomposes more naturally during training, as it is progressively learned.

Why tensor networks? We have a good understanding of deep linear networks (DLNs). For whitened inputs and a suitably aligned initialization, [Saxe et al. \(2014\)](#) solved the gradient-flow dynamics of DLNs exactly: the singular modes of the input–output map decouple and are learned one after the other, each after a plateau, stronger modes first (see also [Saxe et al., 2019](#)). From a small initialization, DLNs and diagonal linear networks follow saddle-to-saddle dynamics in which the rank (or the support) of the learned map increases step by step ([Jacot et al., 2021](#); [Pesme and Flammarion, 2023](#)), and similar dynamics explain a simplicity bias across architectures ([Zhang et al., 2026](#)). We are, however, still far from understanding more complex architectures such as transformers. Even the learning dynamics of deep networks with quadratic activations remain largely open, although there are interesting results for shallow ones (e.g. [Du and Lee, 2018](#); [Sarao Mannelli et al., 2020](#)).

TTNs provide an interesting model of more complex deep networks ([Furman et al., 2026](#)). A TTN is a tree of multilinear maps (Definition 2.7); it generalizes DLNs, which are chains, and Tucker decompositions, which are stars. Unlike a DLN, a TTN computes nonlinear functions of its inputs: it can implement any read-once Boolean formula, and it can be evaluated in time polynomial in the size of the tree and in the bond dimensions. Consequently, TTNs contain targets that are easy to evaluate but cannot be learned by gradient descent in polynomial time, such as parities, for the same reason as neural networks. Nevertheless their loss landscape is conditionally benign, and the difficulty can instead come from high-order saddle points, which are located at rank-deficient parameters ([Furman et al., 2026](#), Theorems 4.3 and 5.2, Proposition 5.3). Hierarchical tensor formats can moreover be exponentially more compact than shallow (CP) ones ([Cohen et al.,](#)

2016). At the same time, TTNs look more interpretable than generic networks because they come with a canonical, gauge-invariant hierarchical singular value decomposition (HSVD, Section 2.4). Empirically, TTNs trained from a small initialization undergo saddle-to-saddle dynamics, i.e. they learn features sequentially, consistently with the incremental hierarchical-rank bias of hierarchical tensor factorizations (Razin et al., 2022) and with the rank-deficient saddles of Furman et al. (2026). TTNs are also more expressive than DLNs, and they can display *soft hierarchical learning*: learning some features accelerates the learning of subsequent features. The following definition makes this precise.

Definition 1.1 (Hard and soft acceleration; after Diagonal TTN note, 2026, Definition 3.1). Let $q_B(\vartheta)$ be an observable measuring the acquisition of a feature B , with target value $q_B^* > 0$, and fix a threshold $\delta \in (0, 1)$. For an intervention $\mathcal{I}$ on training (for example, clamping the parameters that compute another feature A to a learned or to a blocked value, all else being equal), let

$$\tau_B(\delta \mid \mathcal{I}) = \inf\{t \geq 0 : q_B(\vartheta_{\mathcal{I}}(t)) \geq \delta q_B^*\} \in [0, \infty].$$

Learning A *accelerates* learning B if $\tau_B(\delta \mid A \text{ learned}) < \tau_B(\delta \mid A \text{ blocked})$. The acceleration is *hard* if $\tau_B(\delta \mid A \text{ blocked}) = \infty$, and *soft* if both times are finite.

In a DLN with decoupled modes, the only such effect is the symmetric facilitation between the factors of a single mode. In TTNs, acceleration between distinct features can be computed exactly in simple cases (Diagonal TTN note, 2026, Sections 5–6): readouts of nested sub-products (“staircases”) produce hard prerequisite gating, and residual paths produce soft acceleration. For instance, in a three-leaf channelwise residual TTN with residual strength ρ and composition strength γ , learning the lower core (teacher value α) divides the acquisition time of the upper core by $1 + \gamma^2 \alpha^2 / (2\rho^2)$ (Diagonal TTN note, 2026, Proposition 6.1). Expressivity alone is not the mechanism: with fixed orthogonal leaf features, a homogeneous diagonal TTN has exactly the amplitude dynamics of a diagonal DLN, and acceleration comes from parameters shared by several output features (Diagonal TTN note, 2026, Proposition 4.1 and Section 7). For a broader review of hierarchical feature learning and of the observables proposed to track it, see Literature review (2026).

Two hypotheses. While TTNs have properties that make them attractive as a model of deep networks, we would like to first establish how far their training dynamics is from that of more realistic architectures. If TTNs are a good model of the training dynamics of neural networks, then according to a range of observables the training dynamics of TTNs should track the training dynamics of more realistic architectures.

To state this precisely, fix a distribution ν of input–label pairs $(x, y) \in \mathbb{R}^{d_0} \times \mathbb{R}$, with input marginal ν_x , the square loss $\mathcal{L}(f) = \frac{1}{2} \mathbb{E}_\nu[(f(x) - y)^2]$, and an optimizer with its hyperparameters, for instance full-batch gradient descent $\vartheta_{n+1} = \vartheta_n - \eta_n \nabla_{\vartheta} \mathcal{L}(f_{\vartheta_n})$, or its gradient-flow limit in the time variable $t = \sum_n \eta_n$. Training a model class $\vartheta \mapsto f_{\vartheta}$ from an initialization produces a trajectory $t \mapsto f_{\vartheta(t)}$. By an *observable* we mean a quantity that can be evaluated along the trajectory of any architecture. The most robust observables are computed from the predictors $f_{\vartheta(t)}$ alone: the loss $\mathcal{L}(f_{\vartheta(t)})$; the function-space distance $\|f_{\vartheta(t)} - g_{\vartheta(t)}\|_{L^2(\nu_x)}$ between two models; and, in a teacher–student setting where the target $f^* = \sum_{m=1}^M \sqrt{\lambda_m} \chi_m$ is built from features χ_m that are orthonormal in $L^2(\nu_x)$, the feature coefficients, the acquisition times and the order of acquisition,

$$a_m(t) = \frac{\mathbb{E}_{\nu_x}[f_{\vartheta(t)}(x) \chi_m(x)]}{\sqrt{\lambda_m}}, \quad \tau_m(\delta) = \inf\{t \geq 0 : a_m(t) \geq \delta\} \quad (0 < \delta < 1), \quad (1.1)$$

the order being the permutation of $\{1, \dots, M\}$ that sorts $\tau_1(\delta), \dots, \tau_M(\delta)$. Other observables depend on the parameterization but still live in a space common to all architectures, such as the

empirical tangent kernel on a fixed set of inputs (Remark 1.4). Quantities attached to a particular representation, such as the HSVD of the TTN of Section 2.6, are diagnostics rather than observables: they can be compared across models only through a common representation (Remark 2.39). The hypothesis can be formulated at two levels. The first one is global.

Hypothesis 1.2 (Global surrogate). Fix the data, the loss function and the hyperparameters, a family of observables, a tolerance $\varepsilon > 0$ and a training horizon $t_{\max}$. There exist a TTN architecture and an initialization scheme such that, for typical initializations of both models, the observables of the TTN trajectory $g_{\vartheta(t)}$ and of the DNN trajectory $f_{\theta(t)}$ differ by at most ε for all $t \in [0, t_{\max}]$, in the same time variable and without post hoc time warping, and discrete observables, such as the order in which features are learned, coincide.

The second one is local. Even if the global version fails, we could still use TTNs as *local* surrogates of the training dynamics. Consider a DNN with smooth activation functions and Taylor expand its activations to some order around a training checkpoint. The resulting network is polynomial and, as shown in Section 2.6, it can be represented exactly as a TTN.

Hypothesis 1.3 (Local surrogate). Let $\theta_j = \theta(t_j)$ be a checkpoint of DNN training and $\hat{f}_j$ the TTN surrogate built at θ_j (Definition 2.44). Fix a tolerance $\varepsilon > 0$ and a horizon $H > 0$ that is not small compared with the time scale on which features are acquired. When the surrogate is started from θ_j , with the same optimizer state, data and hyperparameters as the DNN, its trajectory $\tilde{\theta}$ satisfies

$$\sup_{0 \leq s \leq H} \|f_{\theta(t_j+s)} - \hat{f}_j(\tilde{\theta}(t_j+s), \cdot)\|_{L^2(\nu_x)} \leq \varepsilon,$$

and the chosen observables agree within ε on $[t_j, t_j + H]$.

If ε exceeds the initial prediction error, some positive horizon always exists by continuity; the content of the hypothesis is that the horizon is long enough to cover non-trivial learning, such as the acquisition of a feature. A sequence of such local surrogates, rebuilt at successive checkpoints, could provide an interpretable “atlas” of training even when no single TTN is globally faithful (Surrogate note, 2026).

Note that even if these hypotheses fail, solving the training dynamics of TTNs could still be highly instructive in the same way that studying the training dynamics of DLNs is instructive although their training dynamics differs from more realistic architectures. For example the saddle to saddle dynamics regime is a regime that we can find in more realistic architecture and they teach us about the theoretical gap between simple toy models and more realistic architectures

Remark 1.4 (Comparing different parameterizations). A DNN and a TTN have different parameters, but their dynamics can be compared in function space. Let $x_1, \dots, x_n$ be the training inputs, $F_\theta = (f_\theta(x_i))_{i \leq n}$, $J_\theta = D_\theta F_\theta$, and $K_\theta = \frac{1}{n} J_\theta J_\theta^\top \in \mathbb{R}^{n \times n}$ the empirical tangent kernel (Jacot et al., 2018); define $\tilde{F}_\vartheta$ and $\tilde{K}_\vartheta$ in the same way for the TTN. Under full-batch gradient flow on the empirical square loss, $\dot{F}_\theta = -K_\theta(F_\theta - y)$, hence

$$\frac{d}{dt}(F_\theta - \tilde{F}_\vartheta) = -K_\theta(F_\theta - \tilde{F}_\vartheta) - (K_\theta - \tilde{K}_\vartheta)(\tilde{F}_\vartheta - y).$$

Function-space agreement is therefore plausible when the two models have close predictions and close residual-weighted tangent kernels (Surrogate note, 2026, Section 4.3). The two Jacobians need not have the same number of columns; their kernels are both $n \times n$.

Outline. Section 2 defines tensors, tensor networks and TTNs (Sections 2.1–2.2), their edge matricizations and hierarchical ranks (Section 2.3), the HSVD (Section 2.4) and the relevant notions of rank (Section 2.5). It then shows that deep polynomial networks are weight-tied TTNs and defines hierarchical features and learning capacity (Section 2.6), and constructs the TTN surrogate of a smooth DNN (Section 2.7). Section 4 constructs observables for comparing the training dynamics of TTN and DNN students of a TTN teacher. Section 6 lists the experiments suggested in the text.

2 Theoretical background

We define tree tensor networks and review some of their important mathematical properties. Throughout, vector spaces are finite-dimensional real Euclidean spaces $\mathbb{R}^d$ with their standard basis and inner product, and we use the resulting identification $\mathbb{R}^d \cong (\mathbb{R}^d)^*$ freely (Furman et al., 2026, Appendix A). General references are Kolda and Bader (2009), Hackbusch (2012) and Lim (2021). We write $[n] = \{1, \dots, n\}$.

2.1 Tensors, multilinear maps and matricizations

Definition 2.1 (Tensor). Let $d_1, \dots, d_n$ be positive integers. An *order- n tensor* is an element $T \in \mathbb{R}^{d_1} \otimes \dots \otimes \mathbb{R}^{d_n}$. Expanding $T = \sum T_{i_1 \dots i_n} e_{i_1} \otimes \dots \otimes e_{i_n}$ in the standard bases identifies T with the array $(T_{i_1 \dots i_n}) \in \mathbb{R}^{d_1 \times \dots \times d_n}$. The index i_k , and the space $\mathbb{R}^{d_k}$, is the k -th *mode*, or *leg*, of T . We write $\langle S, T \rangle = \sum_{i_1, \dots, i_n} S_{i_1 \dots i_n} T_{i_1 \dots i_n}$ and $\|T\|_F = \langle T, T \rangle^{1/2}$.

Definition 2.2 (Multilinear map). Let $V_1, \dots, V_p, W$ be vector spaces. A map $F : V_1 \times \dots \times V_p \rightarrow W$ is *p -linear* if it is linear in each argument when the others are fixed. By the universal property of the tensor product, p -linear maps $V_1 \times \dots \times V_p \rightarrow W$ are in one-to-one correspondence with linear maps $V_1 \otimes \dots \otimes V_p \rightarrow W$.

Proposition 2.3 (Tensors as multilinear maps; Furman et al., 2026, Proposition A.1). Let $T \in \bigotimes_{k=1}^n \mathbb{R}^{d_k}$, and let $I \sqcup O = [n]$ be a partition of the modes into inputs and outputs. The formula

$$F_T((v_k)_{k \in I})_{(i_o)_{o \in O}} = \sum_{(i_k)_{k \in I}} T_{i_1 \dots i_n} \prod_{k \in I} (v_k)_{i_k} \quad (2.1)$$

defines a multilinear map $F_T : \prod_{k \in I} \mathbb{R}^{d_k} \rightarrow \bigotimes_{o \in O} \mathbb{R}^{d_o}$, and $T \mapsto F_T$ is a linear isomorphism onto the space of such maps. Equivalently, T is a linear map from $\bigotimes_{k \in I} \mathbb{R}^{d_k}$ to $\bigotimes_{o \in O} \mathbb{R}^{d_o}$.

This is the precise sense in which “a tensor is a multilinear map”: it maps a tensor product of vector spaces linearly (equivalently, a Cartesian product of vector spaces multilinearly) to another tensor product of vector spaces. The split of the modes into inputs and outputs is a choice, not a property of T .

Example 2.4. An order-3 tensor $W \in \mathbb{R}^{d_1} \otimes \mathbb{R}^{d_2} \otimes \mathbb{R}^{d_3}$ with inputs $\{1, 2\}$ and output $\{3\}$ is the bilinear map $B_W(u, v)_k = \sum_{i,j} W_{ijk} u_i v_j$: a vertex with two input legs and one output leg. With input $\{1\}$ and outputs $\{2, 3\}$, the same W is a linear map $\mathbb{R}^{d_1} \rightarrow \mathbb{R}^{d_2} \otimes \mathbb{R}^{d_3}$. For $d_1 = d_2 = d_3 = d$, the diagonal tensor $\Delta_{ijk} = \mathbf{1}[i = j = k]$ gives the Hadamard (coordinatewise) product $\Delta(u, v) = u \odot v$, which is linear in u and in v but not jointly linear.

Definition 2.5 (Matricization). Let T be an order- n tensor, $\emptyset \neq A \subsetneq [n]$ with complement A^c , and $m_A = \prod_{k \in A} d_k$. Bipartition the indices of T into the groups A and A^c and flatten each group, using the lexicographic order on multi-indices. The result is the *A -matricization* $T^{(A)} \in \mathbb{R}^{m_A \times m_{A^c}}$, with

entries $(T^{(A)})_{(i_k)_{k \in A}, (i_k)_{k \in A^c}} = T_{i_1 \dots i_n}$. As a linear map, $T^{(A)}$ is F_T with inputs A^c and outputs A . Its rank $\text{rank}_A(T) := \text{rank } T^{(A)}$ does not depend on the orderings, and $\text{rank}_A(T) = \text{rank}_{A^c}(T)$. For $A = \{k\}$ one obtains the mode- k matricization $T_{(k)}$ (Kolda and Bader, 2009).

2.2 Tensor networks and tree tensor networks

Definition 2.6 (Tensor network). A *tensor network* consists of

- (i) a finite graph with vertex set V and edge set $E = E_{\text{int}} \sqcup E_{\text{ext}}$, where each *internal edge* or *bond* $e \in E_{\text{int}}$ joins two distinct vertices, and each *external edge* or *open leg* $a \in E_{\text{ext}}$ is attached to a single vertex;
- (ii) a dimension for each edge: a *bond dimension* D_e for $e \in E_{\text{int}}$, and a leg dimension d_a for $a \in E_{\text{ext}}$;
- (iii) for each vertex v , a *core* C_v : a tensor with one mode for each edge incident to v , of the corresponding dimension.

The tensor *represented* by the network is $T \in \bigotimes_{a \in E_{\text{ext}}} \mathbb{R}^{d_a}$, obtained by contracting all bonds:

$$T_{(i_a)_{a \in E_{\text{ext}}}} = \sum_{(i_e)_{e \in E_{\text{int}}}} \prod_{v \in V} (C_v)_{(i_e)_{e \ni v}}, \quad (2.2)$$

where each bond index i_e ranges over $[D_e]$, and $(i_e)_{e \ni v}$ collects the indices of all edges, bonds and legs, incident to v . We write $\vartheta = (C_v)_{v \in V}$ for the parameters and $T(\vartheta)$ for the represented tensor.

Informally: vertices carry tensors and edges are indices. Each core is linear in each of its legs; an open leg is a free index of the represented tensor; and joining two legs by a bond means summing over the shared index, which composes the corresponding multilinear maps (Furman et al., 2026, Proposition A.4). For instance, the product AB of two matrices is the network with two vertices and one bond, and a vertex with two input legs and one output leg is a bilinear map (Example 2.4). The order in which the sums in (2.2) are performed is irrelevant.

Definition 2.7 (Tree tensor network). A *tree tensor network* (TTN) is a tensor network whose graph (V, E_{int}) is a tree. We always assume that every leaf of this tree (a vertex of degree one in (V, E_{int})) carries at least one open leg.

Choosing a root $\rho \in V$ and orienting every bond towards ρ turns a TTN into a feedforward computation: by Proposition 2.3, each non-root core C_v is a multilinear map from its child bonds and its open legs to its parent bond, and T is computed by composing these maps from the leaves to the root (Furman et al., 2026, Definition 3.1 and Appendix A.3). The root and the orientation are a presentation choice, since (2.2) does not depend on them. Figure 1 shows three examples: a chain is a DLN $T = W_L \cdots W_1$, and a star is a Tucker decomposition. A binary tree with one leg per leaf is the hierarchical Tucker format (Hackbusch and Kühn, 2009; Grasedyck, 2010), and a path with one leg per vertex is a tensor train, or matrix product state (Oseledets, 2011). TTNs originate in quantum many-body physics (Shi et al., 2006).

A TTN defines a model of functions once its legs receive inputs. If the input x splits into blocks $(x^{(a)})_{a \in E_{\text{ext}}}$ and each leg receives a fixed embedding $\xi_a(x^{(a)}) \in \mathbb{R}^{d_a}$ of its block (for example $\xi(z) = (1, z)$, or a one-hot encoding), the TTN computes

$$g_{\vartheta}(x) = \left\langle T(\vartheta), \bigotimes_{a \in E_{\text{ext}}} \xi_a(x^{(a)}) \right\rangle, \quad (2.3)$$

which is multilinear in the embedded blocks and nonlinear in x (Stoudenmire and Schwab, 2016). We will meet two situations: *physical* TTNs, whose legs receive distinct blocks of the input, and *computational* TTNs, whose legs all receive copies of the same input (Section 2.6).

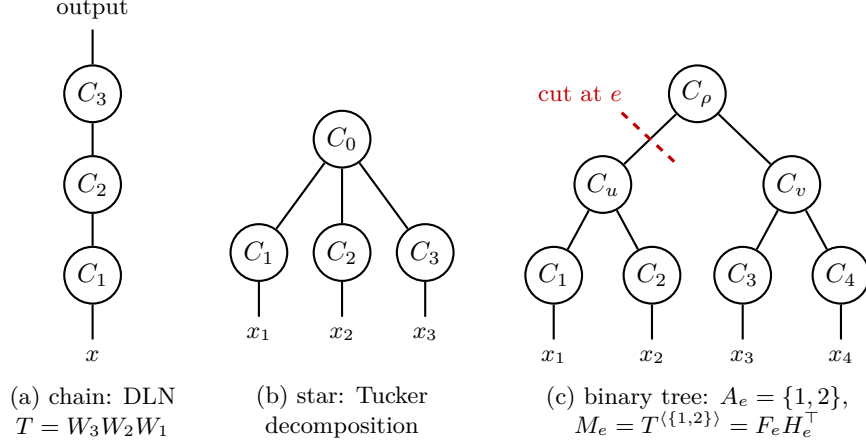


Figure 1: Examples of tree tensor networks. Circles are cores, lines between circles are bonds, and dangling lines are open legs. (a) A chain is a deep linear network. (b) A star is a Tucker decomposition. (c) In a tree, cutting the bond e splits the open legs into $A_e = \{1, 2\}$ and its complement; the corresponding matricization factors through the bond, $M_e = F_e H_e^\top$ (Lemma 2.8), so its rank is at most the bond dimension D_e .

2.3 Edge cuts, bond dimensions and hierarchical ranks

The key structural property of a tree is that removing any edge disconnects it. Each bond therefore defines a bipartition of the open legs, hence a matricization of the represented tensor.

Lemma 2.8 (Edge cuts). Let T be the tensor represented by a TTN and let $e \in E_{\text{int}}$. Removing e splits the tree into two subtrees with vertex sets V_e and V_e^c . Let $A_e \subseteq E_{\text{ext}}$ be the set of open legs attached to V_e , and $A_e^c = E_{\text{ext}} \setminus A_e$.

- (i) A_e and A_e^c are nonempty, so e defines the matricization $M_e := T^{\langle A_e \rangle} \in \mathbb{R}^{m_e \times m_e^c}$, where $m_e = \prod_{a \in A_e} d_a$ and $m_e^c = \prod_{a \in A_e^c} d_a$.
- (ii) $M_e = F_e H_e^\top$, where $F_e \in \mathbb{R}^{m_e \times D_e}$ (resp. $H_e \in \mathbb{R}^{m_e^c \times D_e}$) is the contraction of the cores in V_e (resp. V_e^c) with the index of the bond e left open.
- (iii) $\text{rank } M_e \leq D_e$.

Proof. (i) The subtree on V_e is either a single vertex, which then has degree one in the tree, or it has at least two leaves, at most one of which is the endpoint of e . In both cases V_e contains a leaf of the original tree, which carries an open leg. The same argument applies to V_e^c . (ii) Since e is the only bond between V_e and V_e^c , the sum (2.2) factorizes as a sum over $i_e \in [D_e]$ of the product of the contraction over V_e and the contraction over V_e^c . (iii) follows from (ii). $\square$

Definition 2.9 (Bond dimension and hierarchical rank). The dimension D_e of the index along which the two subtree matrices F_e and H_e are contracted is the *bond dimension* of e . The family

$$(M_e, r_e)_{e \in E_{\text{int}}}, \quad r_e := \text{rank } M_e,$$

collects the edge matricizations of T and their ranks, and the tuple $r(T) := (r_e)_{e \in E_{\text{int}}}$ is the *hierarchical (Tucker) rank* of T with respect to the tree. By Lemma 2.8, $r_e \leq D_e$: the rank across a cut is upper bounded by the bond dimension.

The matrices M_e and ranks r_e depend only on the represented tensor T and on which legs lie on each side of each bond, not on the cores. Cutting an open leg $a \in E_{\text{ext}}$ instead of a bond gives the mode matricization $T^{\langle\{a\}\rangle}$.

The bond dimensions describe exactly what a TTN can represent.

Proposition 2.10 (Realizability; [Grasedyck and Hackbusch, 2011](#); [Furman et al., 2026](#), Proposition B.1). Fix a tree with open legs. A tensor T is represented by a TTN on this tree with bond dimensions $(D_e)_{e \in E_{\text{int}}}$ if and only if $r_e(T) \leq D_e$ for every $e \in E_{\text{int}}$. In that case one can take $D_e = r_e(T)$.

Sketch. Necessity is Lemma 2.8(iii). For sufficiency, the column spaces of the matrices M_e are nested along the tree (Proposition 2.14 below). Choosing a basis of each column space and expressing it in the bases of the children defines cores with $D_e = r_e$ that represent T ([Furman et al., 2026](#), Proposition B.1); padding the cores with zeros gives any larger bond dimensions. $\square$

Hence the model class of a TTN architecture is the set $\{T : r_e(T) \leq D_e \forall e \in E_{\text{int}}\}$. Since the set of matrices of rank at most r is closed, this set is closed, and best approximations within it exist; this is not the case for the CP rank (Section 2.5).

Remark 2.11 (Why trees: canonical matricizations). In a TTN, every bond e determines exactly one bipartition $(A_e \mid A_e^c)$ of the open legs, and its bond dimension bounds the rank across that bipartition. The edge matricizations are therefore canonically attached to the network. This is not the case for tensor networks with cycles. There, removing a single edge does not disconnect the graph; a bipartition of the open legs is separated only by a set C of edges, the rank across it is bounded by $\prod_{e \in C} D_e$, and a given edge belongs to the separating sets of many bipartitions. The correspondence between edges and matricizations is lost, and with it several properties used below: no characterization of the model class by ranks of matricizations such as Proposition 2.10 is available in general, sets of tensors representable with bounded bond dimensions need not be closed ([Landsberg et al., 2012](#)), and canonical (minimum-norm) parameterizations are no longer unique up to gauge ([Furman et al., 2026](#), Remark F.7).

The parameterization of a TTN is redundant: an invertible change of basis on a bond can be absorbed by the two adjacent cores.

Proposition 2.12 (Gauge freedom). Let $e = \{u, v\} \in E_{\text{int}}$ and $X \in \text{GL}(D_e)$. For a core C incident to e , let $C \times_e X$ denote the core obtained by multiplying the mode of C corresponding to e by X , that is $(C \times_e X)_{\dots j \dots} = \sum_i C_{\dots i \dots} X_{ij}$. Replacing C_u by $C_u \times_e X$ and C_v by $C_v \times_e X^{-\top}$ leaves T unchanged. Consequently the *gauge group* $\mathcal{G} = \prod_{e \in E_{\text{int}}} \text{GL}(D_e)$ acts on the parameters and preserves T . Every quantity defined from T alone, in particular M_e , r_e , and the singular values and singular subspaces of M_e , is gauge invariant, whereas individual core entries are not. If $D_e = r_e(T)$ for every e (a minimal representation), then any two parameterizations of T with these bond dimensions differ by a unique gauge transformation.

Proof. If V_e contains u , the substitution maps F_e to $F_e X$ and H_e to $H_e X^{-\top}$, so $M_e = F_e H_e^{\top}$ is unchanged; since M_e is a rearrangement of T , so is T . For the uniqueness statement for minimal representations see [Hackbusch \(2012\)](#) and [Uschmajew and Vandereycken \(2013\)](#). $\square$

2.4 The hierarchical singular value decomposition

Since every bond defines a canonical matricization, we can take its singular value decomposition.

Definition 2.13 (SVD at a bond; hierarchical SVD). For $e \in E_{\text{int}}$, let

$$M_e = U_e S_e V_e^\top$$

be a compact SVD: $U_e \in \mathbb{R}^{m_e \times r_e}$ and $V_e \in \mathbb{R}^{m_e^c \times r_e}$ have orthonormal columns, and $S_e = \text{diag}(\sigma_{e,1}, \dots, \sigma_{e,r_e})$ with $\sigma_{e,1} \geq \dots \geq \sigma_{e,r_e} > 0$. Reshaping the j -th columns of U_e and V_e gives tensors $u_{e,j}$ over the legs in A_e and $v_{e,j}$ over the legs in A_e^c , and, up to the order of the legs,

$$T = \sum_{j=1}^{r_e} \sigma_{e,j} u_{e,j} \otimes v_{e,j}. \quad (2.4)$$

The $\sigma_{e,j}$ are the *hierarchical singular values* of T at e , and $u_{e,j}$ (resp. $v_{e,j}$) its left (resp. right) singular vectors, which describe the subtree on one side of e (resp. its environment). The *hierarchical SVD* (HSVD) of T is the family $(U_e, S_e, V_e)_{e \in E_{\text{int}}}$ of the SVDs across all bonds of the TTN (Grasedyck, 2010).

The HSVD is more than a list of unrelated SVDs: the singular vectors at different bonds are nested and fit together into a TTN.

Proposition 2.14 (Nestedness; the HSVD is a TTN). Root the tree at a vertex ρ and, for each bond e , let A_e be the set of legs below e . Let $u \neq \rho$ be a vertex with parent bond e , child bonds $e_1, \dots, e_k$, and open legs of total dimension n_u ($n_u = 1$ if u has no open leg). Then, after reordering the legs,

$$\text{col}(M_e) \subseteq \mathbb{R}^{n_u} \otimes \text{col}(M_{e_1}) \otimes \dots \otimes \text{col}(M_{e_k}). \quad (2.5)$$

Consequently $B_u := (I_{n_u} \otimes U_{e_1} \otimes \dots \otimes U_{e_k})^\top U_e$ satisfies $U_e = (I_{n_u} \otimes U_{e_1} \otimes \dots \otimes U_{e_k}) B_u$. Set $B_\rho := (I_{n_\rho} \otimes U_{e_1} \otimes \dots \otimes U_{e_k})^\top \text{vec}(T)$ at the root, where $e_1, \dots, e_k$ now denote the child bonds of ρ . The TTN with cores $(B_v)_{v \in V}$ and bond dimensions $(r_e)_{e \in E_{\text{int}}}$ represents T , and in it the contraction of the subtree below each bond e equals U_e ; in particular $F_e = U_e$ has orthonormal columns.

Proof. Each column of M_e is a partial contraction of T against a fixed vector on the legs outside the subtree of e . Matricizing such a column with respect to the legs below a child bond e_j gives columns that are partial contractions of T against vectors on the legs outside the subtree of e_j , hence columns of M_{e_j} . A tensor whose matricizations along every child subtree have their columns in $\text{col}(M_{e_j})$ lies in the right-hand side of (2.5) (Furman et al., 2026, proof of Proposition B.1). The identity for U_e follows because $I \otimes U_{e_1} U_{e_1}^\top \otimes \dots \otimes U_{e_k} U_{e_k}^\top$ is the orthogonal projector onto that right-hand side. The representation statement follows by induction from the leaves, the root step using (2.5) for T itself. $\square$

Remark 2.15 (Canonical and gauge invariant). The HSVD depends only on T and on the tree, so it is gauge invariant (Proposition 2.12). It is unique up to the usual ambiguities of the SVD: the signs of singular vectors, and rotations within the eigenspaces of $M_e M_e^\top$ associated with repeated singular values. The singular values, and the singular subspaces of distinct singular values, are therefore canonical, and near a degeneracy one should compare whole singular subspaces rather than individual vectors. The orthonormal TTN of Proposition 2.14 is one canonical gauge. Another is the balanced, or minimum-norm, gauge of Furman et al. (2026, Appendix F), which is unique up to the orthogonal gauge group $\prod_e \text{O}(D_e)$ and preserved by gradient flow; both carry the same gauge-invariant data.

Truncating the HSVD gives a quasi-optimal compression.

Proposition 2.16 (Truncation; Grasedyck, 2010). Let $k_e \leq r_e$ for $e \in E_{\text{int}}$, and let P_e be the orthogonal projector onto the span of the k_e leading left singular vectors of M_e , acting on the legs in A_e . Enumerate the bonds $e_1, \dots, e_m$ so that every bond comes before the bonds below it. Then $T_k := P_{e_m} \cdots P_{e_1} T$ satisfies $r_e(T_k) \leq k_e$ for every e , and

$$\|T - T_k\|_F^2 \leq \sum_{e \in E_{\text{int}}} \sum_{j > k_e} \sigma_{e,j}^2 \leq |E_{\text{int}}| \cdot \min \{ \|T - T'\|_F^2 : r_e(T') \leq k_e \ \forall e \in E_{\text{int}} \}.$$

Proof. A projection at a bond below e , or in a subtree disjoint from the subtree below e , acts on one side of the cut at e only, so it does not increase r_e ; projections at bonds above e are applied before P_e . For orthogonal projectors $\pi_1, \dots, \pi_m$, the decomposition $A - \pi_m B = (I - \pi_m)A + \pi_m(A - B)$ into orthogonal terms gives, by induction, $\|A - \pi_m \cdots \pi_1 A\|^2 \leq \sum_i \|(I - \pi_i)A\|^2$, and $\|(I - P_e)T\|_F^2 = \sum_{j > k_e} \sigma_{e,j}^2$ by the Eckart–Young theorem (Eckart and Young, 1936). Finally, for every T' with $r_e(T') \leq k_e$, Eckart–Young gives $\sum_{j > k_e} \sigma_{e,j}^2 \leq \|M_e - T'^{(A_e)}\|_F^2 = \|T - T'\|_F^2$. The minimum exists because the constraint set is closed. $\square$

If two bonds define the same bipartition, as the two child bonds of a binary root do, projecting at one of them suffices; this gives the factor $2d - 3$ of Grasedyck (2010) for binary trees with d leaves.

Remark 2.17 (Stability). By Weyl’s inequality, $|\sigma_{e,j}(T) - \sigma_{e,j}(T')| \leq \|M_e(T) - M_e(T')\|_{\text{op}} \leq \|T - T'\|_F$: hierarchical singular values are 1-Lipschitz functions of the tensor. Singular subspaces are stable only relative to the gap that separates their singular values from the rest of the spectrum (Davis and Kahan, 1970; Wedin, 1972). These facts matter whenever the HSVDs of two models, or of one model at two times, are compared.

2.5 Notions of rank

Tensors admit several notions of rank, which coincide for matrices but not for higher-order tensors (Kolda and Bader, 2009; Lim, 2021).

Definition 2.18 (CP rank). The *CP rank* of T , $\text{rank}_{\text{CP}}(T)$, is the smallest number R of rank-one tensors into which T can be decomposed: $T = \sum_{s=1}^R a_s^{(1)} \otimes \cdots \otimes a_s^{(n)}$ (Hitchcock, 1927).

The CP rank is challenging to compute in practice: deciding whether $\text{rank}_{\text{CP}}(T) \leq R$ is NP-hard (Håstad, 1990; Hillar and Lim, 2013). Moreover, for $n \geq 3$ and $R \geq 2$ the set of tensors of CP rank at most R is in general not closed, so a best approximation of low CP rank may fail to exist (de Silva and Lim, 2008).

Definition 2.19 (Mode rank and Tucker rank). The *mode- k rank* of T is $r_k(T) = \text{rank } T_{(k)}$, the rank of the matricization obtained by separating index k from the others. The *Tucker* (or multilinear) *rank* is the tuple $(r_1(T), \dots, r_n(T))$ of all mode ranks. It is the smallest core size in a Tucker decomposition $T = \sum_{j_1, \dots, j_n} \Omega_{j_1 \dots j_n} u_{j_1}^{(1)} \otimes \cdots \otimes u_{j_n}^{(n)}$ (Tucker, 1966; De Lathauwer et al., 2000), i.e. a TTN on a star (Figure 1b).

For a TTN, the *hierarchical Tucker rank* is the tuple $(r_e)_{e \in E_{\text{int}}}$ of Definition 2.9 (Hackbusch and Kühn, 2009; Grasedyck, 2010). Mode ranks and hierarchical ranks are ranks of matrices: unlike the CP rank, they can be computed by SVDs, up to a numerical tolerance.

Proposition 2.20 (Comparison of ranks).

- (i) For every bipartition A , $\text{rank}_A(T) \leq \text{rank}_{\text{CP}}(T)$. In particular $r_k(T) \leq \text{rank}_{\text{CP}}(T)$, and $r_e(T) \leq \text{rank}_{\text{CP}}(T)$ for every tree.
- (ii) In a rooted TTN, if a vertex u has parent bond e , child bonds $e_1, \dots, e_k$ and open legs of total dimension n_u , then $r_e \leq n_u \prod_{j=1}^k r_{e_j}$.
- (iii) A tensor of CP rank R is represented on every tree, with every assignment of legs to vertices, by a TTN whose bond dimensions all equal R and whose cores are diagonal in the bond indices.

Proof. (i) The A -matricization of a rank-one tensor has rank one, and the matrix rank is sub-additive. (ii) Take dimensions in (2.5). (iii) Given $T = \sum_{s=1}^R \bigotimes_{a \in E_{\text{ext}}} w_s^{(a)}$, set $(C_v)_{(i_a)_a, (j_e)_e} = \sum_{s=1}^R \prod_{e \ni v} \mathbf{1}[j_e = s] \prod_a (w_s^{(a)})_{i_a}$, where a ranges over the open legs at v and e over the bonds at v . Contracting forces all bond indices to agree, because the tree is connected (Diagonal TTN note, 2026, Section 2.4). $\square$

The converse of (i) fails dramatically: TTNs with polynomially many parameters represent tensors whose CP rank is exponential in the number of leaves, for almost every choice of cores (Cohen et al., 2016). This is one precise sense in which hierarchical (deep) factorizations are more expressive than shallow ones.

Remark 2.21 (Ranks and critical points). Furman et al. (2026) use a parameter-level notion: ϑ has *full Tucker rank* if, for every bond e and each of its endpoints v , the matricization of C_v that separates the mode of e has rank D_e . In general $r_e(T(\vartheta))$ is at most the ranks of these two matricizations, and equality holds at balanced (minimum-norm) parameters (Furman et al., 2026, Proposition D.2). For a TTN with free cores trained on the Frobenius loss $\frac{1}{2} \|T(\vartheta) - T^*\|_F^2$ (their setting, in which inputs are drawn uniformly among the entries of T , i.e. one-hot inputs) towards a realizable target T^* , every critical point that is not a global minimum is rank deficient (Furman et al., 2026, Theorem 5.2); at such a point the network behaves as a TTN with smaller bond dimensions. For the parity of n bits ($n \geq 4$ a power of two), realized on a balanced binary tree with bond dimension 2, the rank-one point whose cores all have entries $\frac{1}{2}$, which is a best rank-one approximation, is a saddle of order at least $n/2$ (Furman et al., 2026, Proposition 5.3). Hierarchical ranks are therefore natural dynamical variables, and saddle-to-saddle dynamics should appear as successive increases of the rank profile. Whether these results extend to other losses, data distributions or constrained cores is open.

2.6 Deep polynomial networks as tree tensor networks

We now show that deep networks with polynomial activations are TTNs with tied weights, evaluated on copies of the input.

Definition 2.22 (Deep polynomial network). A feedforward network with L hidden layers of widths $d_1, \dots, d_L$, acting on inputs $x \in \mathbb{R}^{d_0}$, is

$$h_0 = x, \quad z_\ell = W_\ell h_{\ell-1} + b_\ell, \quad h_\ell = \sigma_\ell(z_\ell) \quad (1 \leq \ell \leq L), \quad f_\theta(x) = c^\top h_L + b_{\text{out}}, \quad (2.6)$$

with $W_\ell \in \mathbb{R}^{d_\ell \times d_{\ell-1}}$, $b_\ell \in \mathbb{R}^{d_\ell}$, $c \in \mathbb{R}^{d_L}$ and $b_{\text{out}} \in \mathbb{R}$ collected in θ , and activations acting coordinatewise, $\sigma_\ell(z)_i = \sigma_{\ell,i}(z_i)$. It is a *deep polynomial network* (DPN) if every $\sigma_{\ell,i}$ is a polynomial of degree at most K . It is a *deep monomial network* (DMN) if moreover $\sigma_{\ell,i}(z) = z^K$ and all biases vanish ($b_\ell = 0$, $b_{\text{out}} = 0$). A DMN with $K = 2$ is a *deep quadratic network* (DQN).

Allowing the polynomial to depend on the neuron covers the Taylor surrogates of Section 2.7. Since degrees multiply under composition, a DPN is a polynomial in x of degree at most K^L .

2.6.1 Monomial activations: tied TTNs on copies of the input

Proposition 2.23 (A monomial layer is one TTN vertex). Let $\sigma(z) = z^K$ and $W \in \mathbb{R}^{d' \times d}$. Then $\sigma(Wh) = A(h, \dots, h)$, where A is the symmetric K -linear map

$$A_{i j_1 \dots j_K} = \prod_{s=1}^K W_{i j_s}, \quad \text{i.e.} \quad A(u_1, \dots, u_K) = (Wu_1) \odot \dots \odot (Wu_K).$$

In particular, for $K = 2$,

$$\sigma(Wx)_i = \left(\sum_j W_{ij} x_j \right)^2 = \sum_{j,k} W_{ij} x_j W_{ik} x_k = A(x, x)_i. \quad (2.7)$$

The proof is the expansion of the product. Graphically, (2.7) is a vertex with one output leg i and two input legs; equivalently, it is the Hadamard vertex Δ of Example 2.4 whose two input legs carry the same (tied) weight matrix W , applied to two copies of x . Composing layers gives a tree.

Proposition 2.24 (DMNs are tied TTNs). Let f_θ be a DMN with exponent $K \geq 2$ and L layers, and let $N = K^L$. Let $\mathcal{T}_{K,L}$ be the complete K -ary rooted tree of depth L in which every non-root vertex at height ℓ carries the core A_ℓ of Proposition 2.23 built from W_ℓ , the root (height L) carries the K -linear form $c^\top A_L$, and the vertices at height 1 carry the N open legs. Let $T(\theta) \in (\mathbb{R}^{d_0})^{\otimes N}$ be the represented tensor. Then

$$f_\theta(x) = \langle T(\theta), x^{\otimes N} \rangle \quad \text{for all } x \in \mathbb{R}^{d_0},$$

and the bonds between heights ℓ and $\ell + 1$ have dimension d_ℓ .

Proof. Let $\Phi_\ell : (\mathbb{R}^{d_0})^{K^\ell} \rightarrow \mathbb{R}^{d_\ell}$ be the contraction of a subtree of height ℓ : $\Phi_0 = \text{id}$ and $\Phi_\ell(y_1, \dots, y_{K^\ell}) = A_\ell(\Phi_{\ell-1}(y^{(1)}), \dots, \Phi_{\ell-1}(y^{(K)}))$, where $y^{(s)}$ is the s -th block of $K^{\ell-1}$ consecutive arguments. Since $A_\ell(h, \dots, h) = \sigma(W_\ell h)$, induction gives $\Phi_\ell(x, \dots, x) = h_\ell(x)$, hence $f_\theta(x) = c^\top \Phi_L(x, \dots, x) = \langle T(\theta), x^{\otimes N} \rangle$. $\square$

Remark 2.25 (Copies, not copying). The map $x \mapsto x^{\otimes K}$ is not linear, so no tensor implements “copying” a vector. A DMN is therefore not a tensor network applied to x : it is the restriction of the N -linear map $T(\theta)$ to the diagonal $(x, \dots, x)$. The open legs are formal copies of the same input, and a cut of $\mathcal{T}_{K,L}$ separates copies, not input variables. The same identification, for networks with bilinear layers, is described as a TTN with a copying operation by [Pearce et al. \(2024\)](#) and [Dooms et al. \(2025\)](#) (see [Furman et al., 2026](#), Section 2).

Remark 2.26 (Hidden-unit symmetries are gauge transformations). Permuting the hidden units of layer ℓ by a permutation matrix P ($W_\ell \mapsto PW_\ell$, $W_{\ell+1} \mapsto W_{\ell+1}P^\top$) does not change f_θ , and neither does rescaling them, for monomial activations, by an invertible diagonal matrix Λ ($W_\ell \mapsto \Lambda W_\ell$, $W_{\ell+1} \mapsto W_{\ell+1}\Lambda^{-K}$, with c in place of W_{L+1}). On the tied TTN these symmetries act as gauge transformations, by P and by Λ^K respectively, on every bond of height ℓ (Proposition 2.12). Hence $T(\theta)$ and its HSVD are invariant under them, while the weights are not. This is the advantage of the TTN over the DMN parameterization: the HSVD gives a more canonical, gauge-independent description of internal structure. The idea is then to use the TTN representing a DMN to interpret its internal structure, reading the HSVD as the features that the network learns. Section 2.6.3 makes this precise, and Remark 2.39 discusses its limits.

2.6.2 General polynomial activations: homogenization

Biases and lower-order terms break homogeneity. They are absorbed by adding a constant coordinate.

Definition 2.27 (Homogenization). Let $p(z) = \sum_{k=0}^K a_k z^k$ be a polynomial of degree at most K . Its degree- K homogenization is

$$p_h(z_0, z) := \sum_{k=0}^K a_k z_0^{K-k} z^k,$$

which equals $z_0^K p(z/z_0)$ for $z_0 \neq 0$. It is homogeneous of degree K , and $p_h(1, z) = p(z)$. The construction generalizes to multivariate polynomial maps: if $P : \mathbb{R}^n \rightarrow \mathbb{R}^m$ has degree at most K , $P(z) = \sum_{|\alpha| \leq K} c_\alpha z^\alpha$ with $c_\alpha \in \mathbb{R}^m$, then $P_h(z_0, z) = \sum_{|\alpha| \leq K} c_\alpha z_0^{K-|\alpha|} z^\alpha$ is a homogeneous polynomial map on $\mathbb{R}^{1+n}$ with $P_h(1, z) = P(z)$.

A homogeneous polynomial can in turn be identified with a symmetric multilinear map.

Lemma 2.28 (Polarization). Let $Q : V \rightarrow W$ be a homogeneous polynomial map of degree K between finite-dimensional real vector spaces. There is a unique symmetric K -linear map $S_Q : V^K \rightarrow W$ such that $S_Q(y, \dots, y) = Q(y)$ for all y , namely

$$S_Q(y_1, \dots, y_K) = \frac{1}{K!} \sum_{\emptyset \neq J \subseteq [K]} (-1)^{K-|J|} Q\left(\sum_{j \in J} y_j\right).$$

Conversely, every symmetric K -linear map S defines the homogeneous polynomial $y \mapsto S(y, \dots, y)$.

This is the classical polarization identity. Combining the two steps, a polynomial of degree at most K is a homogeneous polynomial of degree K in one more variable, which is a symmetric K -linear map on the augmented space. For a polynomial activation, this map is a vertex with one output leg and K input legs, which is why a DPN has a tree structure.

We write $H_\ell := \mathbb{R} \oplus \mathbb{R}^{d_\ell} \cong \mathbb{R}^{1+d_\ell}$, with coordinates indexed from 0, and $\bar{h} := (1, h) \in H_\ell$ for $h \in \mathbb{R}^{d_\ell}$.

Proposition 2.29 (A polynomial layer is one TTN vertex; [Local TN note, 2026](#), Section 3). Let $P_\ell(h) = \sigma_\ell(W_\ell h + b_\ell)$ with polynomial activations $\sigma_{\ell,i}(z) = \sum_{s=0}^K \alpha_{\ell,i,s} (z - m_{\ell,i})^s$, for arbitrary expansion points $m_{\ell,i}$. Then $P_\ell(h) = \sum_{r=0}^K A_\ell^{(r)} [h^{\otimes r}]$, where $A_\ell^{(r)} : (\mathbb{R}^{d_\ell-1})^r \rightarrow \mathbb{R}^{d_\ell}$ is the symmetric r -linear map with components

$$A_{\ell,i}^{(r)} = \sum_{s=r}^K \binom{s}{r} \alpha_{\ell,i,s} q_{\ell,i}^{s-r} w_{\ell,i}^{\otimes r}, \quad q_{\ell,i} = b_{\ell,i} - m_{\ell,i}, \quad (2.8)$$

and $w_{\ell,i}^\top$ is the i -th row of W_ℓ . Define $G_\ell : H_{\ell-1}^K \rightarrow H_\ell$ by, for $u_s = (a_s, v_s) \in \mathbb{R} \oplus \mathbb{R}^{d_{\ell-1}}$,

$$G_\ell(u_1, \dots, u_K)_0 = \prod_{s=1}^K a_s, \quad G_\ell(u_1, \dots, u_K)_{1:d_\ell} = \sum_{r=0}^K \binom{K}{r}^{-1} \sum_{\substack{S \subseteq [K] \\ |S|=r}} A_\ell^{(r)} [v_s : s \in S] \prod_{s \notin S} a_s. \quad (2.9)$$

Then G_ℓ is symmetric and K -linear, it is the polarization of the homogenized layer map, and

$$G_\ell(\bar{h}, \dots, \bar{h}) = \overline{P_\ell(h)} = (1, P_\ell(h)) \quad \text{for all } h \in \mathbb{R}^{d_{\ell-1}}. \quad (2.10)$$

In coordinates, an entry of G_ℓ with r nonzero input indices equals the corresponding entry of $A_\ell^{(r)}$ divided by $\binom{K}{r}$, and the only nonzero entry of the constant output slice is $(G_\ell)_{0;0\dots 0} = 1$.

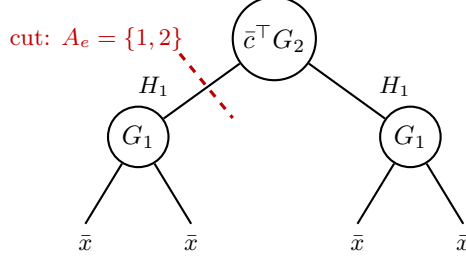


Figure 2: The order-two surrogate of a network with two hidden layers, as a weight-tied binary TTN (Theorem 2.30 and Example 2.32, $K = L = 2$). Both lower vertices carry the same core G_1 ; the $N = 4$ open legs all receive the augmented input $\bar{x} = (1, x) \in H_0$; the bonds carry augmented hidden states $\bar{h}_1 \in H_1 = \mathbb{R} \oplus \mathbb{R}^{d_1}$; and the root carries $\bar{c}^\top G_2$, i.e. its output is contracted with $\bar{c} = (b_{\text{out}}, c)$. The cut shown defines M_e with $A_e = \{1, 2\}$; the other bond of height one gives the complementary bipartition and the same singular values (Proposition 2.33).

Proof. Equation (2.8) follows from the binomial expansion of $(w_{\ell,i}^\top h + q_{\ell,i})^s$. Each term of (2.9) is linear in each u_s , and permuting the u_s permutes the subsets S , so G_ℓ is symmetric and K -linear. With $u_s = \bar{h}$ for all s , the inner sum consists of $\binom{K}{r}$ equal terms $A_\ell^{(r)}[h^{\otimes r}]$, which gives (2.10). With $u_s = (a, v)$ for all s , G_ℓ returns $(a^K, \sum_r a^{K-r} A_\ell^{(r)}[v^{\otimes r}])$, the homogenization of $h \mapsto (1, P_\ell(h))$; by Lemma 2.28, G_ℓ is its polarization. $\square$

The factors $\binom{K}{r}^{-1}$ prevent lower-order terms from being counted several times. Equation (2.8) also shows that the cores are constrained: all orders r reuse the same weight row $w_{\ell,i}$, so G_ℓ is not a free tensor.

Theorem 2.30 (Exact tied-TTN representation of a DPN; [Local TN note, 2026](#), Proposition 1; [Surrogate note, 2026](#), Proposition 1). Let f_θ be a DPN with L layers and activations of degree at most $K \geq 2$, and let $N = K^L$. Consider the complete K -ary tree $\mathcal{T}_{K,L}$ in which every non-root vertex at height ℓ carries the core G_ℓ of Proposition 2.29, the root carries the K -linear form $\bar{c}^\top G_L$, i.e. its output is contracted with $\bar{c} = (b_{\text{out}}, c) \in H_L$, and the N open legs, attached to the vertices of height 1, all receive the augmented input $\bar{x} = (1, x) \in H_0$. Let $T(\theta) \in H_0^{\otimes N}$ be the represented tensor. Then

$$f_\theta(x) = \langle T(\theta), \bar{x}^{\otimes N} \rangle \quad \text{for all } x \in \mathbb{R}^{d_0} \text{ and all } \theta, \quad (2.11)$$

and the bonds between heights ℓ and $\ell + 1$ have dimension $d_\ell + 1$. The map $\theta \mapsto T(\theta)$ is polynomial.

Proof. As in the proof of Proposition 2.24, let $\Phi_\ell : H_0^{K^\ell} \rightarrow H_\ell$ be the contraction of a subtree of height ℓ : $\Phi_0 = \text{id}$ and $\Phi_\ell(y_1, \dots, y_{K^\ell}) = G_\ell(\Phi_{\ell-1}(y^{(1)}), \dots, \Phi_{\ell-1}(y^{(K)}))$. By (2.10) and induction, $\Phi_\ell(\bar{x}, \dots, \bar{x}) = \bar{h}_\ell(x)$. Hence $\langle T(\theta), \bar{x}^{\otimes N} \rangle = \bar{c}^\top \Phi_L(\bar{x}, \dots, \bar{x}) = b_{\text{out}} + c^\top h_L(x) = f_\theta(x)$. The identity is algebraic, so it holds for every θ . $\square$

So a DPN is a weight-tied TTN applied to copies of the input $(1, x)$, the extra coordinate 1 being induced by the homogenization; the bond dimension $d_\ell + 1$ counts the hidden units of layer ℓ plus the constant. Figure 2 shows the case $K = L = 2$. Since (2.11) holds identically in θ , it can be differentiated: training the DPN in its own parameters is the same as training the tied TTN through the map $\theta \mapsto ((G_\ell(\theta))_\ell, \bar{c})$ (Remark 2.52).

Remark 2.31 (What the representation is, and is not).

- (i) It is a *constrained* TTN: the cores are tied across each level, their entries satisfy the algebraic relations (2.8), and the constant slice is fixed.
- (ii) It is evaluated on the *diagonal*: the leaves are formal copies of the whole input (Remark 2.25).
- (iii) The $N = K^L$ leaves are never materialized in computations: the recurrence (2.6) evaluates the same function at a cost linear in the depth, by sharing repeated subcomputations, i.e. as a circuit (a directed acyclic graph) rather than a tree.
- (iv) For $K > 2$, every K -ary vertex can be refined exactly into binary vertices, with new bond dimensions given by matricization ranks of G_ℓ .
- (v) The multilinear extension is not unique. Since $\langle T, \bar{x}^{\otimes N} \rangle = \langle \text{Sym } T, \bar{x}^{\otimes N} \rangle$, where Sym averages over permutations of the N legs, f_θ determines only the symmetric part $\text{Sym } T(\theta)$. It determines it uniquely: a homogeneous polynomial on H_0 is determined by its values on the hyperplane $\{y_0 = 1\}$, and a symmetric tensor by its homogeneous polynomial (Lemma 2.28). The tensor $T(\theta)$ is the particular extension induced by the architecture.

Example 2.32 (Two layers, quadratic activations; [Local TN note, 2026](#), Section 4). For $K = 2$, write each layer polynomial as $P_\ell(h) = a_\ell + A_\ell h + B_\ell(h, h)$ with $a_\ell \in \mathbb{R}^{d_\ell}$, A_ℓ linear and B_ℓ symmetric bilinear (these are $A_\ell^{(0)}, A_\ell^{(1)}, A_\ell^{(2)}$ in Proposition 2.29). For the order-two Taylor polynomial of σ_ℓ at m_ℓ (Section 2.7), with $\gamma_\ell^{(r)} = \sigma_\ell^{(r)}(m_\ell)$ taken coordinatewise and $q_\ell = b_\ell - m_\ell$,

$$\begin{aligned} a_\ell &= \gamma_\ell^{(0)} + \gamma_\ell^{(1)} \odot q_\ell + \frac{1}{2} \gamma_\ell^{(2)} \odot q_\ell \odot q_\ell, & A_\ell u &= \gamma_\ell^{(1)} \odot W_\ell u + \gamma_\ell^{(2)} \odot q_\ell \odot W_\ell u, \\ B_\ell(u, v) &= \frac{1}{2} \gamma_\ell^{(2)} \odot W_\ell u \odot W_\ell v. \end{aligned}$$

The core (2.9) is $G_\ell((\alpha, u), (\beta, v)) = (\alpha\beta, a_\ell \alpha\beta + \frac{1}{2} A_\ell(\alpha v + \beta u) + B_\ell(u, v))$. For an output coordinate $i \geq 1$, its matrix in the two input indices is

$$(G_\ell)_{i;\dots} = \begin{pmatrix} (a_\ell)_i & \frac{1}{2}(A_\ell)_{i,:} \\ \frac{1}{2}(A_\ell)_{i,:}^\top & (B_\ell)_{i,::} \end{pmatrix}.$$

The network is $f_\theta(x) = \bar{c}^\top G_2(G_1(\bar{x}, \bar{x}), G_1(\bar{x}, \bar{x}))$, a polynomial of degree four (not two) in x , with coefficient tensor $T_{j_1 j_2 j_3 j_4} = \sum_{r,a,b} \bar{c}_r (G_2)_{r;ab} (G_1)_{a;j_1 j_2} (G_1)_{b;j_3 j_4}$. As a scalar check ($d_0 = d_1 = d_2 = 1$), for the SiLU activation $\sigma(z) = z/(1 + e^{-z})$ with unit weights, zero biases, centres 0 and unit readout, $p(z) = z/2 + z^2/4$ and $f(x) = p(p(x)) = x/4 + 3x^2/16 + x^3/16 + x^4/64$.

2.6.3 Hierarchical features and learning capacity

The representation allows us to define a notion of hierarchical features of a DPN, from the HSVD of its TTN, and a measure of learning capacity, from its hierarchical ranks. For the tied tree, the HSVD reduces to one SVD per layer.

Proposition 2.33 (One SVD per layer). In the setting of Theorem 2.30, $T(\theta)$ is invariant under the permutations of its N legs induced by automorphisms of $\mathcal{T}_{K,L}$. Consequently, for two bonds e, e' at the same height, $M_{e'}$ is obtained from M_e by permuting rows and columns: the two bonds have the same hierarchical singular values, and their singular vectors correspond under the relabelling of the leaves.

Proof. The cores G_ℓ and $\bar{c}^\top G_L$ are symmetric in their K inputs, so the contraction is invariant under exchanging the children of any vertex. These exchanges generate the automorphism group of $\mathcal{T}_{K,L}$, which acts transitively on the vertices of each height. $\square$

We may therefore index the HSVD of a DPN by height: for $\ell = 1, \dots, L-1$, it consists of singular values $\sigma_{\ell,1} \geq \dots \geq \sigma_{\ell,r_\ell} > 0$ and singular vectors $u_{\ell,j} \in H_0^{\otimes K^\ell}$, $v_{\ell,j} \in H_0^{\otimes (N-K^\ell)}$. (A network with one hidden layer has no bond; its cuts are mode matricizations.)

Definition 2.34 (Hierarchical features). The *hierarchical features* of the DPN f_θ at height ℓ are the pairs $(\sigma_{\ell,j}, \varphi_{\ell,j})$, $j \leq r_\ell$, where

$$\varphi_{\ell,j}(x) := \langle u_{\ell,j}, \bar{x}^{\otimes K^\ell} \rangle$$

is a polynomial of degree at most K^ℓ . The complementary functions are $\psi_{\ell,j}(x) := \langle v_{\ell,j}, \bar{x}^{\otimes (N-K^\ell)} \rangle$.

Proposition 2.35 (What hierarchical features are). For every $\ell \in \{1, \dots, L-1\}$:

- (i) $f_\theta(x) = \sum_{j=1}^{r_\ell} \sigma_{\ell,j} \varphi_{\ell,j}(x) \psi_{\ell,j}(x)$ for all x ;
- (ii) there is a matrix $Q_\ell \in \mathbb{R}^{(d_\ell+1) \times r_\ell}$ such that $\varphi_{\ell,j}(x) = \sum_i (Q_\ell)_{ij} \bar{h}_{\ell,i}(x)$: the hierarchical features at height ℓ are linear readouts of the augmented activations $\bar{h}_\ell = (1, h_\ell)$ of layer ℓ ;
- (iii) $r_\ell \leq d_\ell + 1$.

Proof. (i) Contract (2.4) with $\bar{x}^{\otimes N}$. (ii) By Lemma 2.8, $M_e = F_e H_e^\top$, so $\text{col}(U_e) = \text{col}(M_e) \subseteq \text{col}(F_e)$, i.e. $U_e = F_e Q_\ell$ for some matrix Q_ℓ . The i -th column of F_e is the i -th coordinate of the subtree contraction Φ_ℓ , and $\Phi_\ell(\bar{x}, \dots, \bar{x}) = \bar{h}_\ell(x)$ (proof of Theorem 2.30). (iii) is Lemma 2.8(iii). $\square$

The HSVD at height ℓ thus selects, among the directions of the activation space of layer ℓ , those that the rest of the network uses, ranks them by $\sigma_{\ell,j}$, and is invariant under the hidden-unit symmetries of Remark 2.26. Two cautions apply. The orthonormality of the $u_{\ell,j}$ refers to the Frobenius inner product of coefficient tensors, not to $L^2(\nu_x)$. And $\varphi_{\ell,j}$ and $\psi_{\ell,j}$ are functions of the same input, so (i) is not a decomposition of f_θ into functions of independent variables (Remark 2.39).

Definition 2.36 (Learning capacity). The *learning capacity* of a DPN at θ is the hierarchical rank profile of its TTN, $r(\theta) = (r_e(T(\theta)))_{e \in E_{\text{int}}}$, that is $(r_\ell(\theta))_{\ell=1}^{L-1}$. It is bounded by the bond dimensions, $r_\ell \leq D_\ell = d_\ell + 1$, which form the maximal capacity of the architecture.

By Proposition 2.10, a free TTN with bond dimensions D represents exactly the tensors with $r \leq D$, so $r(\theta)$ measures how much of the available capacity the model uses. Remark 2.21 further suggests that learning proceeds by increasing ranks, although its results are proved for free TTNs on the Frobenius loss, not for the tied TTN of a DPN, whose loss depends on $T(\theta)$ only through $\text{Sym } T(\theta)$ (Remark 2.31(v)). A tuple of ranks raises a natural question.

Question 2.37 (A global notion of learning capacity). What would be a reasonable global notion of learning capacity? The minimum, the sum, or the maximum of the ranks? Or some other complexity measure using the structure of the TTN?

Remark 2.38 (Some candidates).

- (a) $\min_e r_e$ and $\max_e r_e$ measure the narrowest interface and the cost of contraction, respectively, but ignore where the ranks sit in the tree.
- (b) $\sum_e \log r_e$ is the logarithm of the product of the ranks; $\log r_e$ bounds the entropy of the normalized squared singular values $\sigma_{e,j}^2 / \sum_i \sigma_{e,i}^2$ at e (the entanglement entropy of quantum physics).
- (c) For a feasible profile r , the dimension of the set of tensors whose rank profile is exactly r ,

$$\dim(r) = \sum_{v \in V} n_v \prod_{e \ni v} r_e - \sum_{e \in E_{\text{int}}} r_e^2,$$

where n_v is the product of the dimensions of the open legs at v , counts the parameters of a minimal TTN modulo the gauge group. It weights each rank by its position in the tree, and reduces to $r(m+n-r)$ for $m \times n$ matrices (Holtz et al., 2012; Uschmajew and Vandereycken, 2013).

- (d) Exact ranks are unstable, since a generic perturbation makes them maximal. Continuous surrogates, such as the participation ratio $(\sum_j \sigma_{e,j}^2)^2 / \sum_j \sigma_{e,j}^4$ (Diagonal TTN note, 2026, Section 9.4) or the entropy-based effective rank (Roy and Vetterli, 2007), can be aggregated as in (a)–(c).

Which of these, if any, predicts learning behaviour is an empirical question.

Remark 2.39 (Which tensor carries the features?). Two caveats, emphasized by Local TN note (2026, Section 8) and Surrogate note (2026, Sections 3.4 and 7.4), qualify the identification of the HSVD of $T(\theta)$ with the features learned by the network.

First, the HSVD of $T(\theta)$ is invariant under gauge transformations, hence under hidden-unit symmetries, but it is not an invariant of the function f_θ . By Remark 2.31(v), f_θ determines only $\text{Sym } T(\theta)$, and different networks computing the same function can have different hierarchical ranks (Example 2.40). The HSVD is an invariant of the computation, not of the input–output map.

Second, cuts of the unrolled tree separate copies of the input, not input variables. When the input has independent blocks $x = (x_S, x_{S^c})$, with $\nu_x = \mu_S \otimes \mu_{S^c}$, an architecture-independent alternative is the functional SVD across a physical cut: every $g \in L^2(\nu_x)$ has a Schmidt decomposition

$$g(x_S, x_{S^c}) = \sum_{q \geq 1} s_q(g) \phi_q^S(x_S) \phi_q^{S^c}(x_{S^c}),$$

with $(\phi_q^S)_q$ orthonormal in $L^2(\mu_S)$ and $(\phi_q^{S^c})_q$ orthonormal in $L^2(\mu_{S^c})$. Its singular values and subspaces depend only on g and ν_x , so they can be compared between a teacher, a DNN, its polynomial surrogate and any TTN. Organizing such cuts along a tree of input blocks gives a physical-block TTN and HSVD, whose ranks are not bounded by the widths d_ℓ . In practice, they can be estimated from the crossed-grid matrix $[g(u_a, v_b) / \sqrt{n_L n_R}]_{a \leq n_L, b \leq n_R}$ built from independent samples $u_a \sim \mu_S$ and $v_b \sim \mu_{S^c}$ (Local TN note, 2026, Section 8.4). The computational HSVD of Definition 2.34 and the physical one answer different questions: the first describes how the network computes, the second what it computes.

Example 2.40 (Same function, different hierarchical ranks). Take $d_0 = 2$ and two DQNs with two layers (Definition 2.22, $K = L = 2$) computing $f(x) = x_1^2 x_2^2$. Network P has $W_1 = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$, so that $h_1 = ((x_1 + x_2)^2, (x_1 - x_2)^2)$, then $W_2 = \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}$ and $c = 1$, so that $z_2 = x_1 x_2$ and $f = z_2^2$. Network Q has $W_1 = I_2$, so that $h_1 = (x_1^2, x_2^2)$, then $W_2 = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$ and $c = \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}$. With the bilinear forms $Z(u, v) = \frac{1}{2}(u_1 v_2 + u_2 v_1)$, $Z_a(u, v) = u_1 v_1$ and $Z_b(u, v) = u_2 v_2$, Proposition 2.24 gives

$$\begin{aligned} T_P(y_1, y_2, y_3, y_4) &= Z(y_1, y_2) Z(y_3, y_4), \\ T_Q(y_1, y_2, y_3, y_4) &= \frac{1}{2} [Z_a(y_1, y_2) Z_b(y_3, y_4) + Z_b(y_1, y_2) Z_a(y_3, y_4)]. \end{aligned}$$

Both restrict to $x_1^2 x_2^2$ on the diagonal, but the hierarchical rank of the bond at height one is $r_1 = 1$ for P and $r_1 = 2$ for Q. By contrast, for independent inputs x_1 and x_2 , the functional rank of f across the physical cut $x_1 \mid x_2$ is 1, whichever network computes it.

Question 2.41 (Interpretability). If we use the TTN as a surrogate model and its HSVD to interpret it, do the hierarchical singular values and vectors correspond to some meaningful structure? For example, if we use a toy model that we can interpret, do the features of the TTN correspond to the known features? Modular arithmetic would be an interesting example to investigate.

The following computation shows what to expect in the modular arithmetic case, in which the mechanism learned by small networks is known.

Example 2.42 (Modular addition). Let p be an odd prime, encode inputs $a, b \in \mathbb{Z}_p$ as one-hot vectors, and let a model output logits $o(a, b) \in \mathbb{R}^p$, as in the grokking setting of Power et al. (2022). Since every function of (a, b) is bilinear in the one-hot encodings, the model is exactly a tensor $L \in \mathbb{R}^{p \times p \times p}$, $L_{abc} = o(a, b)_c$: a TTN on the physical tree with leaves a, b and output c . Its HSVD across the cut that separates a is the SVD of $L^{\langle\{a\}\rangle} \in \mathbb{R}^{p \times p^2}$. This is a function-level (physical) HSVD: it depends only on the logits, not on the architecture that computes them (Remark 2.39).

- (i) The target $T_{abc}^* = \mathbf{1}[c \equiv a + b]$ has a maximally degenerate spectrum. The rows of $T^{\star\langle\{a\}\rangle}$ are orthogonal and each contains p ones, so $T^{\star\langle\{a\}\rangle} (T^{\star\langle\{a\}\rangle})^\top = p I_p$ and all p singular values equal $\sqrt{p}$. The SVD alone therefore singles out no basis, in particular not the Fourier basis. The same holds for the two other cuts.
- (ii) Nanda et al. (2023) found that one-layer transformers trained on modular addition implement a Fourier algorithm whose logits are approximately $L_{abc} = \sum_{k \in \mathcal{F}} \lambda_k \cos(2\pi k(a + b - c)/p)$, for a small set $\mathcal{F} \subseteq \{1, \dots, (p-1)/2\}$ of key frequencies. If the $\lambda_k > 0$ are distinct, then $L^{\langle\{a\}\rangle}$ has rank $2|\mathcal{F}|$ and singular values $\lambda_k p^{3/2}/2$, each with multiplicity two, and the corresponding left singular subspaces are $\text{span}\{(\cos(2\pi k a/p))_a, (\sin(2\pi k a/p))_a\}$. Indeed, with $\omega_k = 2\pi k/p$,

$$\cos(\omega_k(a + b - c)) = \cos(\omega_k a) \cos(\omega_k(b - c)) - \sin(\omega_k a) \sin(\omega_k(b - c)),$$

and these vectors are mutually orthogonal across k and between $\cos$ and $\sin$, with squared norms $p/2$ as functions of a and $p^2/2$ as functions of (b, c) .

For this idealized mechanism, the physical HSVD thus recovers exactly the known features, as two-dimensional frequency subspaces ranked by their strengths λ_k ; the rotation within each subspace reflects the translation symmetry of $\mathbb{Z}_p$. Question 2.41 asks more: whether the HSVD of the computational TTN of a trained network (Section 2.6.3), which describes how the network computes, also recovers these subspaces. A natural test case is an MLP with quadratic activations acting on the concatenated one-hot inputs, for which an analytic Fourier solution is known in the case of one hidden layer (Gromov, 2023). With one hidden layer, the computational cuts are the mode matricizations of the root core; with more layers, Definition 2.34 applies.

2.7 Surrogate model of a deep neural network

Consider a general DNN (2.6) with smooth activation functions, more precisely of class C^{k+1} on intervals that contain all the pre-activations of the network and of its surrogate. The idea is to Taylor expand the DNN into a DPN around a training checkpoint, and to represent this DPN as a TTN.

Definition 2.43 (Layerwise Taylor surrogate; Local TN note, 2026, Section 2.2). Let $\theta_j = \theta(t_j)$ be a checkpoint, $\mathcal{C}$ a finite calibration set of inputs, and $k \geq 1$. For every layer ℓ and neuron i , let the *centre*

$$m_{\ell,j,i} := \frac{1}{|\mathcal{C}|} \sum_{x \in \mathcal{C}} z_{\ell,i}(\theta_j, x)$$

be the average pre-activation of the neuron over the calibration set at the checkpoint, and let

$$p_{\ell,j,i}^{(k)}(z) := \sum_{r=0}^k \frac{\sigma_{\ell,i}^{(r)}(m_{\ell,j,i})}{r!} (z - m_{\ell,j,i})^r$$

be the order- k Taylor polynomial of $\sigma_{\ell,i}$ at that centre. The *order- k surrogate at checkpoint j* is the DPN obtained by replacing every activation by its Taylor polynomial, while using the surrogate’s own hidden states:

$$\hat{h}_0 = x, \quad \hat{z}_\ell = W_\ell \hat{h}_{\ell-1} + b_\ell, \quad \hat{h}_\ell = p_{\ell,j}^{(k)}(\hat{z}_\ell), \quad \hat{f}_j(\theta, x) = c^\top \hat{h}_L + b_{\text{out}}.$$

The centres and the Taylor coefficients are frozen, and θ remains trainable.

Definition 2.44 (TTN surrogate). For $k \geq 2$, the *TTN surrogate* of the DNN at checkpoint j is the tied TTN of Theorem 2.30, with $K = k$, that represents the surrogate: $\hat{f}_j(\theta, x) = \langle T_j(\theta), \bar{x}^{\otimes k^L} \rangle$. We say that this TTN is a surrogate model of the DNN at the training point θ_j .

Remark 2.45 (On the definition).

- (i) The surrogate must use its own hidden states: substituting the true hidden states of the DNN at every layer would define an error diagnostic, not a trainable model.
- (ii) Its input degree is k^L , not k . Layerwise Taylorization is not the Taylor expansion of the predictor in θ (Bai et al., 2020), since composition keeps cross-layer terms of all orders. For $k = 1$, every activation is replaced by its tangent line and the surrogate is a deep affine network, i.e. a DLN with biases, not the linearized (NTK) model; its TTN degenerates into a chain. The family of surrogates therefore starts from the model whose dynamics is understood.
- (iii) The centre lives in pre-activation space, since σ is expanded in its argument. Expanding around the mean pre-activation over a calibration set places the expansion at a typical value over the input distribution; the mean minimizes the mean squared distance to the centre, which controls the order-one remainder, but it is a heuristic and not an optimal choice in general.
- (iv) Choosing instead the per-example centre $m_{\ell,j}(x) = z_\ell(\theta_j, x)$ makes the surrogate agree with the DNN and its θ -derivatives up to order k at θ_j , so that $\hat{f}_j(\theta_j + \Delta, \cdot) - f_{\theta_j + \Delta} = O(\|\Delta\|^{k+1})$ under suitable bounds. Its coefficients, however, depend on x , so it is not one polynomial in x and has no single TTN. It is a useful control, which separates the error due to the spread of pre-activations across inputs from the error due to training (Local TN note, 2026, Section 2.3).
- (v) Layerwise Taylorization of trained networks also underlies NN2Poly (Moralá et al., 2025); for higher-order dynamical approximations in parameter space, see Bai et al. (2020) and Huang and Yau (2020).
- (vi) In automatic differentiation, the centres and the derivatives $\sigma^{(r)}(m)$ must be detached from the computational graph, while the cores G_ℓ , recomputed from the current θ , must remain differentiable.

We would like to know how good an approximation of the DNN such a TTN is. Since training follows gradients, the relevant errors are on derivatives as well as values.

Lemma 2.46 (Taylor remainders). If σ is C^{k+1} on the segment between m and z , and M_{k+1} bounds $|\sigma^{(k+1)}|$ there, then

$$|\sigma(z) - p^{(k)}(z)| \leq \frac{M_{k+1}}{(k+1)!} |z - m|^{k+1}, \quad |\sigma'(z) - (p^{(k)})'(z)| \leq \frac{M_{k+1}}{k!} |z - m|^k.$$

Proof. Use the Lagrange form of the remainder; for the second inequality, note that $(p^{(k)})'$ is the order- $(k-1)$ Taylor polynomial of σ' at m . $\square$

Proposition 2.47 (Loss-gradient defect; [Local TN note, 2026](#), Proposition 2). Let $\mathcal{L}(\theta) = \frac{1}{2}\mathbb{E}_\nu[(f_\theta(x) - y)^2]$ and $\hat{\mathcal{L}}_j(\theta) = \frac{1}{2}\mathbb{E}_\nu[(\hat{f}_j(\theta, x) - y)^2]$, and define at a common parameter θ

$$\begin{aligned}\varepsilon_{0,j} &= \|f_\theta - \hat{f}_j(\theta, \cdot)\|_{L^2(\nu_x)}, & \varepsilon_{1,j} &= \|\nabla_\theta f_\theta - \nabla_\theta \hat{f}_j(\theta, \cdot)\|_{L^2(\nu_x)}, \\ R_j &= \|f_\theta - y\|_{L^2(\nu)}, & B_j &= \|\nabla_\theta \hat{f}_j(\theta, \cdot)\|_{L^2(\nu_x)},\end{aligned}$$

where $\|v\|_{L^2} = \mathbb{E}[\|v\|^2]^{1/2}$ for vector-valued v . Whenever differentiation under the expectation is valid,

$$\|\nabla \mathcal{L}(\theta) - \nabla \hat{\mathcal{L}}_j(\theta)\| \leq R_j \varepsilon_{1,j} + B_j \varepsilon_{0,j} =: \beta_j(\theta).$$

Proof. $\nabla \mathcal{L} - \nabla \hat{\mathcal{L}}_j = \mathbb{E}[(f - y)(\nabla f - \nabla \hat{f}_j) + (f - \hat{f}_j)\nabla \hat{f}_j]$; apply the triangle and Cauchy–Schwarz inequalities. $\square$

Two functions can thus be close while their parameter derivatives, and therefore their training updates, are different: prediction agreement alone is not sufficient.

Proposition 2.48 (Trajectory comparison; [Local TN note, 2026](#), Section 6). Let θ and $\tilde{\theta}$ solve $\dot{\theta} = -\nabla \mathcal{L}(\theta)$ and $\dot{\tilde{\theta}} = -\nabla \hat{\mathcal{L}}_j(\tilde{\theta})$ on $[t_j, t_j + h]$, and assume that both stay in a set $\mathcal{U}$ on which $\nabla \mathcal{L}$ is κ_j -Lipschitz and $\|\nabla \mathcal{L} - \nabla \hat{\mathcal{L}}_j\| \leq \beta_j$. Then $E(s) := \|\theta(t_j + s) - \tilde{\theta}(t_j + s)\|$ satisfies

$$E(s) \leq e^{\kappa_j s} E(0) + \beta_j \frac{e^{\kappa_j s} - 1}{\kappa_j}, \quad 0 \leq s \leq h,$$

the fraction being s if $\kappa_j = 0$. For gradient descent with steps η_n , $E_{n+1} \leq (1 + \eta_n \kappa_j) E_n + \eta_n \beta_j$.

Proof. The upper Dini derivative satisfies $D^+ E \leq \|\nabla \mathcal{L}(\theta) - \nabla \mathcal{L}(\tilde{\theta})\| + \|\nabla \mathcal{L}(\tilde{\theta}) - \nabla \hat{\mathcal{L}}_j(\tilde{\theta})\| \leq \kappa_j E + \beta_j$; integrate (Grönwall). The discrete case is identical. $\square$

These statements make the quality of the approximation precise: the local surrogate is faithful over windows on which the loss-gradient defect β_j is small compared with the gradient itself. They are conditional, and their constants can grow with depth, width and parameter norms; [Local TN note \(2026, Appendix A\)](#) propagates the remainders of Lemma 2.46 through the layers into $\varepsilon_{0,j}$ and $\varepsilon_{1,j}$.

Remark 2.49 (Fixed centres have an error floor). With input-independent centres, the radius $|z - m|$ in Lemma 2.46 includes the spread $S_{\ell,j} = \sup_x \|z_\ell(\theta_j, x) - m_{\ell,j}\|_\infty$ of the pre-activations around the centres, which does not shrink with the time window ([Local TN note, 2026](#), Section 6.3). The gradient defect $\|\nabla \mathcal{L}(\theta_j) - \nabla \hat{\mathcal{L}}_j(\theta_j)\|$ at the checkpoint is then generally nonzero, and since both trajectories start at θ_j , $E(s) = s \|\nabla \mathcal{L}(\theta_j) - \nabla \hat{\mathcal{L}}_j(\theta_j)\| + o(s)$: the error grows linearly from the start of the window, and Proposition 2.48 gives a matching upper bound. If the fixed-centre surrogate fails while the per-example surrogate of Remark 2.45(iv) succeeds, the problem is the spread of activations across inputs, not the tensor representation. Chebyshev or distribution-weighted polynomial fits over the observed range of pre-activations are then natural, non-Taylor alternatives ([Surrogate note, 2026](#), Section 8.2).

We would also like to know whether the HSVD of the TTN can be used to extract the structure that the DPN has learned, and to predict its training dynamics. The following identity connects the two.

Proposition 2.50 (Dynamics of hierarchical singular values). Let $N = k^L$. Along the gradient flow $\dot{\theta} = -\nabla \hat{\mathcal{L}}_j(\theta)$ of the surrogate, the tensor $T(t) := T_j(\theta(t))$ evolves as

$$\dot{T} = -\mathcal{K}_\theta[R_\theta], \quad \mathcal{K}_\theta := DT_j(\theta)DT_j(\theta)^*, \quad R_\theta := \mathbb{E}_\nu[(\hat{f}_j(\theta, x) - y)\bar{x}^{\otimes N}],$$

where $\mathcal{K}_\theta$ is a positive semidefinite operator on $H_0^{\otimes N}$ and R_θ is the residual-weighted moment tensor. If $\sigma_{e,i}$ is a simple nonzero singular value of $M_e(t)$, then it is differentiable and

$$\dot{\sigma}_{e,i} = u_{e,i}^\top \dot{M}_e v_{e,i} = -\langle Y_{e,i}, \mathcal{K}_\theta[R_\theta] \rangle,$$

where $Y_{e,i}$ is the tensor whose A_e -matricization is $u_{e,i} v_{e,i}^\top$.

Proof. We have $\hat{\mathcal{L}}_j = \mathcal{J} \circ T_j$ with $\mathcal{J}(T) = \frac{1}{2} \mathbb{E}_\nu[(\langle T, \bar{x}^{\otimes N} \rangle - y)^2]$, whose gradient is R . By the chain rule, $\dot{\theta} = -DT_j(\theta)^*[R_\theta]$ and $\dot{T} = DT_j(\theta)[\dot{\theta}]$. For a simple singular value with unit singular vectors, $\sigma = u^\top M v$ and $\dot{\sigma} = u^\top \dot{M} v + \sigma(u^\top \dot{u} + v^\top \dot{v}) = u^\top \dot{M} v$, because $\|u\| = \|v\| = 1$. $\square$

The identity is exact but not closed: $\mathcal{K}_\theta$, R_θ and the singular vectors all evolve. Predicting the training dynamics from the HSVD amounts to finding regimes in which a few singular values and subspaces approximately close under these equations (Surrogate note, 2026, Section 12), as the singular modes of a DLN do in the analysis of Saxe et al. (2014). When surrogates built at different checkpoints are compared, their HSVDs should be evaluated at the same parameter value, since otherwise a change of Taylor centres would be mistaken for learning (Local TN note, 2026, Section 8.5; Surrogate note, 2026, Section 9.1); Remark 2.17 bounds how much the spectrum can move.

A more direct approach: polynomial activations. Another, more direct approach is to replace the activation functions of the original network by polynomial activation functions. We then train the corresponding TTN, define some useful observables, such as the HSVD, and check whether their training dynamics is similar to that of the original DNN.

Definition 2.51 (Polynomial-activation proxy). Given the architecture (2.6) with activation σ , choose polynomials q_ℓ of degree K , for example the Taylor polynomial of σ at a fixed point, or a least-squares fit of σ on an interval $[-a, a]$ that contains most pre-activations. The *polynomial proxy* f_θ^q is the DPN obtained by replacing σ_ℓ with q_ℓ , initialized and trained exactly like the DNN. By Theorem 2.30, $f_\theta^q(x) = \langle T^q(\theta), \bar{x}^{\otimes K^L} \rangle$, so the hierarchical singular values $\sigma_{\ell,j}(t)$ and ranks $r_\ell(t)$ of Definitions 2.34 and 2.36 can be tracked along its training. They are diagnostics of the TTN representation; they can be compared with the DNN through the HSVDs of its local TTN surrogates (Definition 2.44), or through physical HSVDs (Remark 2.39).

Remark 2.52 (Algebraic equivalence is not dynamical equivalence). “Training the corresponding TTN” can mean two different things. (a) Training the DPN in its parameters θ is identical to training the tied TTN through the map $\Gamma : \theta \mapsto ((G_\ell(\theta))_\ell, \bar{c})$, because (2.11) holds identically in θ . (b) Training the cores g as free variables is a different dynamical system: if $g = \Gamma(\theta)$ and $\mathcal{J}(g)$ denotes the loss as a function of the cores, (a) induces $\dot{g} = -D\Gamma(\theta)D\Gamma(\theta)^\top \nabla_g \mathcal{J}(g)$, a preconditioned and possibly degenerate flow, whereas (b) is $\dot{g} = -\nabla_g \mathcal{J}(g)$. Moreover, free training breaks the ties and constraints of Remark 2.31(i). Surrogate note (2026, Section 4.1) proposes a ladder of relaxations that isolates each simplification: the tied TTN trained through θ ; free tied cores with the induced metric; a block-diagonal approximation of that metric; Euclidean core gradients; untied cores; compressed ranks; and finally a different tree. For computations, one should use the recurrence (2.6) rather than the exponentially large tree.

Remark 2.53 (Choosing the degree). A problem is to choose the degree of the nodes of the TTN. Several considerations bear on it.

- (i) Approximation: on a bounded range of pre-activations, least-squares or Chebyshev approximations of a smooth activation improve rapidly with K , whereas Taylor polynomials are accurate only near their centre.
- (ii) Stability: polynomials are unbounded and the degree K^L of the network grows exponentially with depth, so pre-activations that leave the fitted range are amplified. The fraction of pre-activations outside that range is a direct diagnostic (Surrogate note, 2026, Section 8.8).
- (iii) Structure: $K = 2$ is the smallest nonlinear choice and gives binary trees. Networks with bilinear layers $(Wx) \odot (Vx)$, which are untied versions of quadratic activations, are competitive in practice and can be interpreted through their tensors (Pearce et al., 2024).
- (iv) Cost: each core has order $K + 1$, while the bond dimensions $d_\ell + 1$ do not depend on K .

A pragmatic rule is to choose the smallest K for which the defects of Proposition 2.47 are below a tolerance at initialization and at a few checkpoints.

3 Training dynamics of TTNs

4 How to compare TTNs and DNNs

We want to test experimentally whether the training dynamics of TTNs is similar to that of DNNs with standard activation functions, such as ReLU, GELU or SiLU. Hypothesis 1.2 phrases this question in terms of observables, i.e. quantities that can be evaluated along the trajectory of any architecture. This section constructs such observables in the setting in which they are easiest to interpret: a teacher–student experiment in which the teacher is a TTN. The teacher’s tree then provides a canonical hierarchy of interfaces, its bonds, and the singular value decomposition across each bond, taken in the geometry of the data, provides a gauge-invariant list of candidate features, which we call *atoms*. Comparing a TTN with a DNN then amounts to asking, for each atom and each student, when the student acquires the atom, where it computes it, and whether it relies on it.

These questions call for different kinds of evidence, which should not be conflated (Figure 3):

- (i) *output*: the predictor of the student contains the atom, with the teacher’s strength;
- (ii) *representation*: the atom, or the two factors of which it is the product, can be read off a hidden layer of the student;
- (iii) *use*: the computation downstream of that layer is sensitive to this internal code, and causally relies on it.

The answers also depend on how the input is distributed over the leaves of the teacher. We distinguish three regimes:

- (a) *product law*: the leaves receive distinct blocks of the input, and the blocks are independent;
- (b) *dependent blocks*: the leaves receive distinct blocks of the input, with an arbitrary joint law;
- (c) *repeated inputs*: all the leaves receive copies of the same input, as in the tied TTN of a deep polynomial network (Section 2.6).

Independence of the blocks is not needed to define any of the observables below. It is a source of orthogonality and of exact computability: it makes functions of disjoint blocks uncorrelated, it makes the population loss of a TTN depend on the data only through the second moments of the leaf embeddings, and it turns the decomposition of the teacher across each bond into a Schmidt decomposition in the geometry of the loss. We therefore define every object for an arbitrary joint law, and state separately what independence adds. Regime (c) is not a limit of the other two: its leaves are copies of one variable rather than distinct variables, and its decompositions across bonds

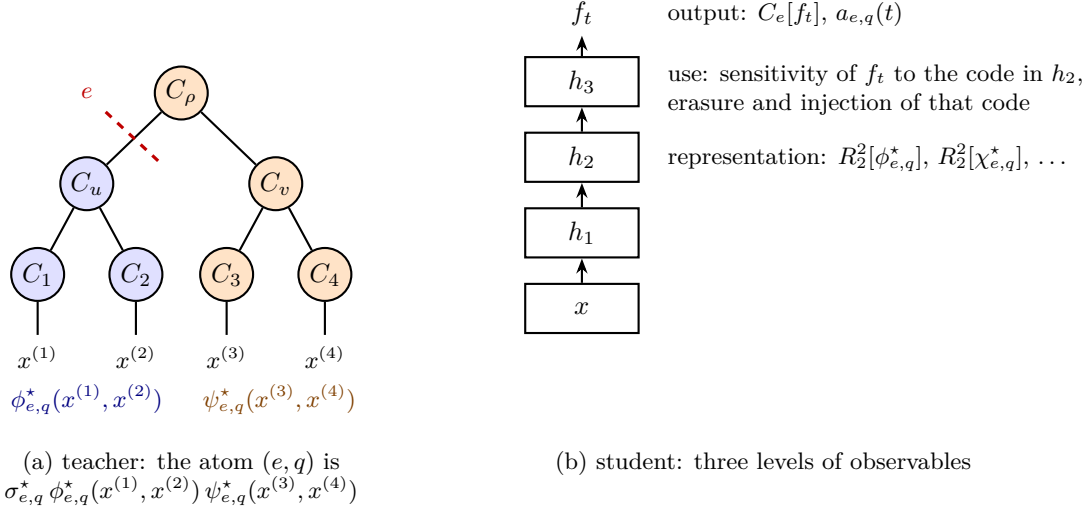


Figure 3: Comparing a student with a TTN teacher. (a) Cutting the bond e splits the teacher into the subtree below e (blue), which computes the left factors $\phi_{e,q}^*$ of the inputs $x^{(1)}, x^{(2)}$, and its environment (orange), which computes the right factors $\psi_{e,q}^*$ of $x^{(3)}, x^{(4)}$. An atom is the product of two such factors, weighted by a singular value of the data-weighted HSVD (Sections 4.2 and 4.3). (b) For a student, one asks whether its output contains the atom (Section 4.4), whether the factors and the atom can be read off a hidden layer, here h_2 (Section 4.7), and whether the layers above use and rely on that code (Section 4.8).

describe a computation rather than a function.

Section 4.1 fixes the setting. Section 4.2 defines the data-weighted HSVD of a TTN, whose singular functions are the hierarchical features of the teacher, and of any TTN student, in the geometry of the data; Section 4.3 defines the atoms. Section 4.4 defines output observables in the coordinates of the teacher, and Section 4.5 a teacher-free decomposition of any student, its functional HSVD, which coincides with the data-weighted HSVD for TTNs. Section 4.6 treats teachers with repeated inputs. Sections 4.7 and 4.8 define the representation and use observables, Section 4.9 explains which observables can be computed exactly for TTN students, and Section 4.10 turns the observables into acquisition times, comparisons between students and an estimation protocol. Section 4.11 summarizes them in Tables 1 and 2.

4.1 The teacher–student setting

Teacher. Fix a binary tree whose N leaves carry one open leg each, indexed by $a \in [N]$; a balanced tree, with N a power of two, is the simplest choice. We root the tree at an internal vertex ρ and, as in Proposition 2.14, let A_e be the set of legs below the bond e , so that A_e and A_e^c are the two sides of the cut of Lemma 2.8. A TTN on this tree (Definition 2.7) represents a tensor T^* . Leg a receives the block $x^{(a)} \in \mathcal{X}_a$ of the input through a fixed embedding $\xi_a : \mathcal{X}_a \rightarrow \mathbb{R}^{d_a}$, for example $\xi_a(z) = z$, $\xi_a(z) = (1, z)$ or a one-hot encoding, and, as in (2.3), the teacher is

$$f^*(x) = \left\langle T^*, \bigotimes_{a=1}^N \xi_a(x^{(a)}) \right\rangle. \quad (4.1)$$

The labels are $y = f^*(x)$, possibly plus independent zero-mean noise. This is a *physical* TTN in the sense of Section 2.2: distinct legs receive distinct blocks of the input. For example, the balanced

bilinear teacher of [Surrogate note \(2026, Section 6.2\)](#), which applies linear maps at the leaves, planted bilinear maps at the internal vertices and a linear readout at the root, is of this form with $\xi_a(z) = z$. Formula (4.1) defines f^* at every point of $\prod_a \mathcal{X}_a$, not only on the support of the data; this will matter when the blocks are dependent. Teachers with repeated inputs are introduced in Section 4.6.

Data. The blocks $X = (X^{(1)}, \dots, X^{(N)})$ of a random input have an arbitrary joint law ν_x . For $A \subseteq [N]$, we write $x_A = (x^{(a)})_{a \in A}$, ν_A for the law of X_A , $\mu_a := \nu_{\{a\}}$ for the law of the block a , $\Xi_A(x_A) = \bigotimes_{a \in A} \xi_a(x^{(a)})$, and

$$\Sigma_A := \mathbb{E}[\Xi_A(X_A) \Xi_A(X_A)^\top], \quad \Sigma_a := \Sigma_{\{a\}}, \quad (4.2)$$

for the Gram matrix of the embedding of the legs in A . We assume that $\mathbb{E} \|\Xi_A(X_A)\|^2 < \infty$ for every A . Norms and inner products of functions are those of $L^2(\nu_x)$, or of $L^2(\nu_A)$ for functions of x_A . The *product law* is the special case

$$\nu_x = \mu_1 \otimes \dots \otimes \mu_N, \quad (4.3)$$

in which $\Sigma_A = \bigotimes_{a \in A} \Sigma_a$ for every A . In neither case do the blocks need to be Gaussian, centred, whitened or identically distributed.

Two further notions will be used. At each bond e , the *decoupled law* $\nu_e^\otimes := \nu_{A_e} \otimes \nu_{A_e^c}$ gives the two sides of the cut their correct marginal laws but draws them independently. It coincides with ν_x if and only if X_{A_e} and $X_{A_e^c}$ are independent, and a sample from it, which we call a *recombined input*, is obtained by joining the A_e -part of one data point to the A_e^c -part of another, independent one. The law ν_x is *nondegenerate* if $\Sigma_{[N]}$ is nonsingular, i.e. if no function $\langle D, \Xi_{[N]} \rangle$ with $D \neq 0$ vanishes almost surely. This holds under the product law when every Σ_a is nonsingular; for polynomial embeddings with linearly independent coordinates and a law with a density that is positive on a nonempty open set, since a nonzero polynomial cannot vanish almost everywhere on an open set; and for one-hot embeddings of discrete blocks when every combination of values has positive probability. It fails when the support of ν_x is degenerate, for instance when a block is a deterministic function of other blocks. Synthetic experiments realize the product law by construction. Dependence can then be introduced in a controlled way, for instance with correlated Gaussian blocks, which give a nondegenerate law, or with a discrete latent variable, which makes ν_x a finite mixture of product laws and keeps exact computations available (Remark 4.7).

Students and time. A DNN f_θ of the form (2.6), with a standard activation, and a TTN g_ϑ of the form (2.3), normally on the teacher's tree, are trained on the same examples with the same loss. We write f_t for the predictor of a student at time t , whichever its architecture: $f_t = f_{\theta(t)}$ or $f_t = g_{\vartheta(t)}$. Time is the integrated learning rate $t = \sum_n \eta_n$ of Hypothesis 1.2. Students that share their parameters, such as a DNN and its polynomial proxy or its TTN surrogate (Section 2.7), are compared under identical updates from the same optimizer state. A DNN and a TTN, by contrast, have different parameter metrics, so the same raw learning rate can move them at very different speeds in function space. The optimizer family, the tuning budget and the time convention must then be fixed before any feature curve is examined. A single global calibration of the time scale, fixed in advance, for instance by matching the initial speeds $\|\partial_t f_t\|$ at $t = 0$, is compatible with Hypothesis 1.2: for gradient descent, it amounts to a constant multiplier in the parameterization of the TTN, i.e. to a choice of architecture. Feature-specific or post hoc time warps are not, since they would make the hypothesis unfalsifiable. Both the number of updates and the integrated

learning rate should be reported. Three further precautions apply (Surrogate note, 2026, Sections 6.6 and 6.7). Every student should be able to fit the teacher under a generous training budget, since an underfitting student says nothing about dynamics. There is no single notion of equal capacity for a DNN and a TTN, so widths, ranks and initialization scales should be bracketed rather than matched by one number. Finally, a TTN on a wrong tree is a useful structural control.

4.2 The data-weighted HSVD of the teacher

Fix a bond e of the teacher. It splits the legs into the legs A_e below e and the others, A_e^c , and contracting (4.1) gives, up to the ordering of the legs,

$$f^\star(x) = \Xi_{A_e}(x_{A_e})^\top M_e^\star \Xi_{A_e^c}(x_{A_e^c}), \quad M_e^\star := T^{\star\langle A_e \rangle}, \quad (4.4)$$

at every point x , whatever the law of the data. The HSVD of Definition 2.13 decomposes $M_e^\star$ in the Euclidean geometry of coefficients. This is the right geometry for comparing tensors, but not for comparing functions: learning is driven by the population loss, hence by the geometry of $L^2(\nu_x)$. The two geometries agree when the embeddings are orthonormal under the data, $\Sigma_A = I$ for every A , for instance for independent whitened blocks and $\xi_a(z) = z$. Otherwise they differ, and the Euclidean singular values can be misleading about which features matter (Example 4.4). We therefore express each side of the cut in coordinates that are orthonormal under its own law, and take the SVD there. No preprocessing is applied to the inputs seen by the students: the change of coordinates only concerns the description of the teacher. Since the same construction will be applied to TTN students, we state it for a general tensor.

Definition 4.1 (Data-weighted HSVD). Let T be a tensor over the legs of the tree, for instance the tensor represented by a TTN on it, and let $g_T := \langle T, \Xi_{[N]} \rangle$ be the function it defines. Write $(\cdot)^{1/2}$ for the positive semidefinite square root and $(\cdot)^+$ for the Moore–Penrose pseudoinverse, and let $\tilde{\Xi}_A := (\Sigma_A^{1/2})^+ \Xi_A$ be the *whitened embedding* of the legs in A . For each bond e , let

$$\tilde{M}_e(T) := \Sigma_{A_e}^{1/2} T^{\langle A_e \rangle} \Sigma_{A_e^c}^{1/2} = U_e S_e V_e^\top, \quad S_e = \text{diag}(\sigma_{e,1}, \dots, \sigma_{e,\tilde{r}_e}), \quad \sigma_{e,1} \geq \dots \geq \sigma_{e,\tilde{r}_e} > 0,$$

be a compact SVD, with columns $u_{e,q}$ of U_e and $v_{e,q}$ of V_e . The *data-weighted singular functions* of T at e are

$$\phi_{e,q}(x_{A_e}) := u_{e,q}^\top \tilde{\Xi}_{A_e}(x_{A_e}), \quad \psi_{e,q}(x_{A_e^c}) := v_{e,q}^\top \tilde{\Xi}_{A_e^c}(x_{A_e^c}), \quad q \leq \tilde{r}_e. \quad (4.5)$$

The family $(\sigma_{e,q}, \phi_{e,q}, \psi_{e,q})_{e \in E_{\text{int}}, q \leq \tilde{r}_e}$ is the *data-weighted HSVD* of T , and the left singular functions $\phi_{e,q}$ are its *data-weighted hierarchical features*. For the teacher, $T = T^\star$, we write $\tilde{M}_e^\star$, $U_e^\star$, $S_e^\star$, $V_e^\star$, $\sigma_{e,q}^\star$, $\phi_{e,q}^\star$, $\psi_{e,q}^\star$ and $r_e^\star$.

In the rest of this section, singular values and singular vectors always refer to data-weighted SVDs. The data enter only through the Gram matrices Σ_{A_e} and $\Sigma_{A_e^c}$ of the embeddings of the two sides of each cut. These involve the law of each side, but never the dependence between the two sides. Under the product law they are Kronecker products of the leaf Gram matrices Σ_a , which are computed analytically or estimated once, from a large sample independent of all other data; for $\xi_a(z) = (1, z)$, Σ_a is the matrix of the first and second moments of $x^{(a)}$. Under dependence, Σ_{A_e} also contains mixed moments of the embeddings of different leaves on the same side; Proposition 4.6 below shows how to compute the decomposition without forming these exponentially large matrices. The data-weighted rank $\tilde{r}_e = \text{rank } \tilde{M}_e(T)$ is at most $\text{rank } T^{\langle A_e \rangle}$, with equality when Σ_{A_e} and $\Sigma_{A_e^c}$ are nonsingular: directions of coefficients that vanish on the data do not count.

Proposition 4.2 (Properties of the data-weighted HSVD). For every joint law ν_x , every tensor T and every bond e :

- (i) $(\phi_{e,q})_q$ is orthonormal in $L^2(\nu_{A_e})$, and $(\psi_{e,q})_q$ is orthonormal in $L^2(\nu_{A_e^c})$;
- (ii) the expansion

$$g_T(x) = \sum_{q=1}^{\tilde{r}_e} \sigma_{e,q} \phi_{e,q}(x_{A_e}) \psi_{e,q}(x_{A_e^c}) \quad (4.6)$$

holds for ν_x -almost every x and for $\nu_e^\otimes$ -almost every x ;

- (iii) the products $\phi_{e,i} \psi_{e,j}$ are orthonormal in $L^2(\nu_e^\otimes)$, and (4.6) is the Schmidt decomposition of g_T in $L^2(\nu_e^\otimes)$ across the cut $(A_e \mid A_e^c)$. Its singular values, and the spans of its singular functions that belong to equal singular values, depend only on g_T as an element of $L^2(\nu_e^\otimes)$. In particular, they are gauge invariant, and they do not change if the embeddings and the tensor are changed in a way that leaves g_T unchanged on recombined inputs;
- (iv) if X_{A_e} and $X_{A_e^c}$ are independent, in particular under the product law, then $\nu_e^\otimes = \nu_x$: the products $\phi_{e,i} \psi_{e,j}$ are orthonormal in $L^2(\nu_x)$, (4.6) is the Schmidt decomposition of g_T across the physical cut $(A_e \mid A_e^c)$ of Remark 2.39, and $\tilde{r}_e$ is the rank of g_T across this cut.

Proof. Let $\Pi_A := \Sigma_A^{1/2}(\Sigma_A^{1/2})^+$ be the orthogonal projector onto the range of Σ_A . If $\Sigma_A w = 0$, then $\mathbb{E}[(w^\top \Xi_A)^2] = w^\top \Sigma_A w = 0$; hence $\Xi_A(X_A)$ lies in the range of Σ_A almost surely, i.e. $\Xi_A = \Pi_A \Xi_A = \Sigma_A^{1/2} \tilde{\Xi}_A$ almost surely. (i) $\mathbb{E}[\tilde{\Xi}_A \tilde{\Xi}_A^\top] = (\Sigma_A^{1/2})^+ \Sigma_A (\Sigma_A^{1/2})^+ = \Pi_A$, and the columns of U_e lie in $\text{col}(\tilde{M}_e)$, which is contained in the range of $\Sigma_{A_e}^{1/2}$, so $\mathbb{E}[\phi_{e,q} \phi_{e,q'}] = u_{e,q}^\top \Pi_{A_e} u_{e,q'} = \delta_{qq'}$; the same argument applies to the $\psi_{e,q}$. (ii) The events $\{\Xi_{A_e} = \Sigma_{A_e}^{1/2} \tilde{\Xi}_{A_e}\}$ and $\{\Xi_{A_e^c} = \Sigma_{A_e^c}^{1/2} \tilde{\Xi}_{A_e^c}\}$ have probability one under ν_x and under $\nu_e^\otimes$, since these two laws have the same marginals. On their intersection, $g_T = \Xi_{A_e}^\top T^{(A_e)} \Xi_{A_e^c} = \tilde{\Xi}_{A_e}^\top \tilde{M}_e \tilde{\Xi}_{A_e^c}$, and inserting the SVD gives (4.6). (iii) Under $\nu_e^\otimes$, $\mathbb{E}[\phi_{e,i} \psi_{e,j} \phi_{e,k} \psi_{e,l}] = \mathbb{E}[\phi_{e,i} \phi_{e,k}] \mathbb{E}[\psi_{e,j} \psi_{e,l}] = \delta_{ik} \delta_{jl}$ by (i). The function g_T is the kernel of the Hilbert–Schmidt operator $K : L^2(\nu_{A_e^c}) \rightarrow L^2(\nu_{A_e})$, $(K\psi)(x_{A_e}) = \mathbb{E}[g_T(x_{A_e}, X'_{A_e^c}) \psi(X'_{A_e^c})]$ with $X'_{A_e^c} \sim \nu_{A_e^c}$, and by (i) and (ii), $KK^* = \sum_q \sigma_{e,q}^2 \phi_{e,q} \langle \phi_{e,q}, \cdot \rangle$. The $\sigma_{e,q}^2$ are therefore the nonzero eigenvalues of KK^* , with eigenspaces spanned by the $\phi_{e,q}$ of equal $\sigma_{e,q}$, and KK^* depends only on g_T as an element of $L^2(\nu_e^\otimes)$; the same argument with K^*K applies to the $\psi_{e,q}$. Gauge invariance holds because $\tilde{M}_e(T)$ depends only on T (Proposition 2.12). (iv) If X_{A_e} and $X_{A_e^c}$ are independent, their joint law is the product of their marginal laws; apply (iii). $\square$

Proposition 4.2 says what the data-weighted HSVD is in every regime: at each bond, it is the Schmidt decomposition of the function in the *decoupled geometry* $L^2(\nu_e^\otimes)$, in which the subtree below e and its environment receive independent inputs with the correct marginal laws. Under the product law, this is the geometry of the loss, and what the HSVD of Section 2.4 says about the Frobenius norm holds for the population loss: the squared singular values are the energies carried by the modes, and truncations are optimal (Proposition 4.12). Under dependence, the squared singular values are energies on recombined inputs, which can differ from the energies on the data; Sections 4.3 and 4.4 account for the difference. The decoupled geometry is nevertheless the natural one for a physical TTN: its formula (4.1) is defined on recombined inputs, and the subtree below e computes the same functions whatever its environment receives.

Under the product law, the data-weighted HSVD is moreover the HSVD of a single TTN.

Proposition 4.3 (Product law: one data-weighted TTN). Under the product law (4.3), for every tensor T :

- (i) $\tilde{\Xi}_A = \bigotimes_{a \in A} \tilde{\xi}_a$, with $\tilde{\xi}_a := (\Sigma_a^{1/2})^+ \xi_a$, and $\tilde{M}_e(T)$ is the A_e -matricization of the *data-weighted tensor* $\tilde{T}$ obtained from T by multiplying each leg a by $\Sigma_a^{1/2}$;

- (ii) if T is represented by a TTN, then $\tilde{T}$ is represented by the same TTN with the open leg of each leaf core multiplied by $\Sigma_a^{1/2}$, with the same tree and bond dimensions. Hence the data-weighted HSVD of T is the HSVD of $\tilde{T}$: Propositions 2.14 and 2.16 and Remark 2.17 apply to it;
- (iii) $g_T(x) = \langle \tilde{T}, \bigotimes_a \tilde{\xi}_a(x^{(a)}) \rangle$ for ν_x -almost every x .

Proof. (i) Under (4.3), $\Sigma_A = \bigotimes_{a \in A} \Sigma_a$, and the square root and the pseudoinverse of a Kronecker product of positive semidefinite matrices are the Kronecker products of the square roots and of the pseudoinverses. Matricizing $\tilde{T}$ across $(A_e \mid A_e^c)$ gives $\Sigma_{A_e}^{1/2} T^{(A_e)} \Sigma_{A_e^c}^{1/2}$. (ii) Multiplying an open leg of a core by a matrix multiplies the corresponding mode of the contraction (2.2) by that matrix. (iii) $\langle \tilde{T}, \bigotimes_a \tilde{\xi}_a \rangle = \langle T, \bigotimes_a \Pi_a \xi_a \rangle$ with $\Pi_a := \Sigma_a^{1/2} (\Sigma_a^{1/2})^+$, and $\Pi_a \xi_a(X^{(a)}) = \xi_a(X^{(a)})$ almost surely (proof of Proposition 4.2). $\square$

Example 4.4 (Why the data weighting matters). Take $N = 2$ blocks $x^{(1)}, x^{(2)} \in \mathbb{R}^2$, $\xi_a(z) = z$, and the teacher $f^*(x) = \langle x^{(1)}, x^{(2)} \rangle = x_1^{(1)} x_1^{(2)} + x_2^{(1)} x_2^{(2)}$, a TTN with two leaves and one bond. Its coefficient matrix is $M^* = I_2$, so the Euclidean HSVD has a degenerate pair $\sigma_1 = \sigma_2 = 1$ and suggests two equally important atoms, $x_1^{(1)} x_1^{(2)}$ and $x_2^{(1)} x_2^{(2)}$. Now let $x^{(1)}$ and $x^{(2)}$ be independent, with $x^{(2)} \sim \mathcal{N}(0, I_2)$ and $x^{(1)} \sim \mathcal{N}(0, \Sigma_1)$, where $\Sigma_1 = \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}$ and $0 < \rho < 1$. Then $\tilde{M}^* = \Sigma_1^{1/2}$ has the singular values $\sqrt{1 + \rho}$ and $\sqrt{1 - \rho}$, and

$$f^* = \sqrt{1 + \rho} \phi_+^* \psi_+^* + \sqrt{1 - \rho} \phi_-^* \psi_-^*, \quad \phi_\pm^* = \frac{x_1^{(1)} \pm x_2^{(1)}}{\sqrt{2(1 \pm \rho)}}, \quad \psi_\pm^* = \frac{x_1^{(2)} \pm x_2^{(2)}}{\sqrt{2}}.$$

Under the data, there is one strong atom, which carries the fraction $(1 + \rho)/2$ of $\|f^*\|^2$, and one weak atom. For ρ close to 1, the best predictor of rank one across the cut already has the small population loss $(1 - \rho)/2$ (Proposition 4.12). Which atoms the loss favours, and plausibly which atoms a loss-driven learner acquires first, is decided by the data-weighted singular values, not by the Euclidean ones. The blocks are independent here: the weighting matters even under the product law, as soon as the coordinates of an embedding are correlated or not normalized.

Remark 4.5 (Which moments of the data enter). For every joint law and all tensors T and T' ,

$$\langle g_T, g_{T'} \rangle_{L^2(\nu_x)} = \text{vec}(T)^\top \Sigma_{[N]} \text{vec}(T'), \quad (4.7)$$

so the population loss of a TTN student depends on the data only through the Gram matrix $\Sigma_{[N]}$ of the full product embedding. The data-weighted SVD at e uses less: it replaces $\Sigma_{[N]}$ by $\Sigma_{A_e} \otimes \Sigma_{A_e^c}$ (up to the ordering of the legs), which is the Gram matrix of $\Xi_{[N]}$ under the decoupled law $\nu_e^{\otimes}$. Under the product law, $\Sigma_{[N]} = \bigotimes_a \Sigma_a$ and the two coincide: the data enter only through the second moments of the leaf embeddings. This is not a Gaussian assumption; it holds because a physical TTN is linear in each leaf embedding separately. Moreover, Σ_a is a second moment of the *embedding*, not of the raw input: for a scalar block and $\xi_a(z) = (1, z, z^2, z^3)$, it contains the moments of z up to order six. Under dependence, Σ_{A_e} contains mixed moments of the embeddings of different leaves on the same side of the cut, and $\Sigma_{[N]}$ contains mixed moments across the cut; the difference between $\Sigma_{[N]}$ and $\Sigma_{A_e} \otimes \Sigma_{A_e^c}$ is what the joint Gram matrices of Section 4.4 correct for. When the same input feeds several legs, $\Sigma_{[N]}$ contains the moments of the input up to twice the number of copies (Section 4.6; [Local TN note, 2026](#), Section 8.1).

The data-weighted SVD at a bond can be computed from the two messages that the network sends through it.

Proposition 4.6 (Data-weighted SVD from the bond messages). Let e be a bond, T a tensor, and $T^{(A_e)} = F_e H_e^\top$ a factorization through $\mathbb{R}^{D_e}$, for instance the one given by the cores of a TTN (Lemma 2.8). Let

$$h_e(x_{A_e}) := F_e^\top \Xi_{A_e}(x_{A_e}), \quad n_e(x_{A_e^c}) := H_e^\top \Xi_{A_e^c}(x_{A_e^c})$$

be the messages that the two sides of the cut send through the bond, so that $g_T(x) = h_e(x_{A_e})^\top n_e(x_{A_e^c})$, and let $\Sigma_e^h := \mathbb{E}[h_e h_e^\top]$ and $\Sigma_e^n := \mathbb{E}[n_e n_e^\top]$. Then the data-weighted singular values of T at e are the nonzero singular values of the $D_e \times D_e$ matrix $(\Sigma_e^h)^{1/2}(\Sigma_e^n)^{1/2}$. If $(\Sigma_e^h)^{1/2}(\Sigma_e^n)^{1/2} = WSY^\top$ is a compact SVD, the singular vectors can be chosen so that, almost surely,

$$\Phi_e = W^\top ((\Sigma_e^h)^{1/2})^+ h_e, \quad \Psi_e = Y^\top ((\Sigma_e^n)^{1/2})^+ n_e,$$

where $\Phi_e := (\phi_{e,1}, \dots, \phi_{e,\tilde{r}_e})^\top$ and $\Psi_e := (\psi_{e,1}, \dots, \psi_{e,\tilde{r}_e})^\top$. In particular, the data-weighted hierarchical features at e are linear readouts of the message h_e .

Proof. Let $\tilde{F}_e := \Sigma_{A_e}^{1/2} F_e$ and $\tilde{H}_e := \Sigma_{A_e^c}^{1/2} H_e$, so that $\tilde{M}_e(T) = \tilde{F}_e \tilde{H}_e^\top$, $\Sigma_e^h = F_e^\top \Sigma_{A_e} F_e = \tilde{F}_e^\top \tilde{F}_e$ and $\Sigma_e^n = \tilde{H}_e^\top \tilde{H}_e$. Write $\Sigma^h := \Sigma_e^h$ and $\Sigma^n := \Sigma_e^n$, and let $U := \tilde{F}_e ((\Sigma^h)^{1/2})^+ W$ and $V := \tilde{H}_e ((\Sigma^n)^{1/2})^+ Y$. Since $WS = (\Sigma^h)^{1/2}(\Sigma^n)^{1/2}Y$, the columns of W lie in the range of Σ^h , so $U^\top U = W^\top ((\Sigma^h)^{1/2})^+ \Sigma^h ((\Sigma^h)^{1/2})^+ W = W^\top W = I$; likewise $V^\top V = I$. Moreover $USV^\top = \tilde{F}_e \Pi^h \Pi^n \tilde{H}_e^\top$, where Π^h and Π^n are the orthogonal projectors onto the ranges of Σ^h and Σ^n . Since $\ker \Sigma^h = \ker \tilde{F}_e$, $\tilde{F}_e \Pi^h = \tilde{F}_e$, and likewise $\Pi^n \tilde{H}_e^\top = \tilde{H}_e^\top$. Hence $\tilde{M}_e(T) = USV^\top$ is a compact SVD. Finally, $h_e = F_e^\top \Sigma_{A_e}^{1/2} \tilde{\Xi}_{A_e} = \tilde{F}_e^\top \tilde{\Xi}_{A_e}$ almost surely (proof of Proposition 4.2), so $U^\top \tilde{\Xi}_{A_e} = W^\top ((\Sigma^h)^{1/2})^+ h_e$; the same computation applies to Ψ_e . $\square$

Remark 4.7 (Computing the data-weighted HSVD). The matrices $\tilde{M}_e$ are exponentially large and are never formed. By Proposition 4.6, the data-weighted SVD at e only requires the two $D_e \times D_e$ second-moment matrices of the bond messages, and the values of the singular functions at a sample are obtained by contracting the sample through the two halves of the network. Under the product law, these matrices are computed exactly by propagating second moments through the tree. For a leaf bond e with leaf core $C_a \in \mathbb{R}^{d_a \times D_e}$, $\Sigma_e^h = C_a^\top \Sigma_a C_a$; at an internal vertex u with core C_u , child bonds e_1, e_2 and parent bond e , $(h_e)_k = \sum_{i,j} (C_u)_{ijk} (h_{e_1})_i (h_{e_2})_j$, so $(\Sigma_e^h)_{kk'} = \sum_{i,i',j,j'} (C_u)_{ijk} (C_u)_{i'j'k'} (\Sigma_{e_1}^h)_{ii'} (\Sigma_{e_2}^h)_{jj'}$, because $X_{A_{e_1}}$ and $X_{A_{e_2}}$ are independent. The environment moments Σ_e^n are propagated in the same way, from the root downwards. The cost is polynomial in the bond and leg dimensions and linear in N . Equivalently, one multiplies the leaf cores by $\Sigma_a^{1/2}$ (Proposition 4.3(ii)), orthonormalizes the network towards e by QR decompositions, and takes the SVD of the resulting small centre matrix (Grasedyck, 2010), (Diagonal TTN note, 2026, Section 9.2). Under dependence, the recursion fails, since in general $\mathbb{E}[h_{e_1} h_{e_1}^\top \otimes h_{e_2} h_{e_2}^\top] \neq \Sigma_{e_1}^h \otimes \Sigma_{e_2}^h$, and Σ_e^h and Σ_e^n are estimated by Monte Carlo, on a large sample independent of all other data; the estimates are unbiased, and only two $D_e \times D_e$ matrices are needed per bond. If ν_x is a finite mixture of product laws, $\nu_x = \sum_k \pi_k \otimes_a \mu_a^{(k)}$, the recursion is run for each component and the results are averaged with the weights π_k , which keeps the computation exact.

The data-weighted features of a TTN are hierarchical: the features at a bond are built from the features at the bonds below it.

Proposition 4.8 (Hierarchical composition of the features). Let $u \neq \rho$ be an internal vertex with parent bond e and child bonds e_1 and e_2 , so that $A_e = A_{e_1} \sqcup A_{e_2}$, and let T be a tensor.

(i) Under the product law,

$$\phi_{e,q}(x_{A_e}) = \sum_{i \leq \tilde{r}_{e_1}} \sum_{j \leq \tilde{r}_{e_2}} (B_u)_{(i,j),q} \phi_{e_1,i}(x_{A_{e_1}}) \phi_{e_2,j}(x_{A_{e_2}}) \quad (4.8)$$

with $B_u := (U_{e_1} \otimes U_{e_2})^\top U_e$, which has orthonormal columns, and the products $\phi_{e_1,i} \phi_{e_2,j}$ are orthonormal in $L^2(\nu_{A_e})$.

(ii) For an arbitrary joint law, if $\Sigma_{A_{e'}}$ and $\Sigma_{A_{e'}^c}$ are nonsingular for $e' \in \{e, e_1, e_2\}$, in particular if ν_x is nondegenerate, then (4.8) holds with

$$B_u := (U_{e_1}^\top \Sigma_{A_{e_1}}^{1/2} \otimes U_{e_2}^\top \Sigma_{A_{e_2}}^{1/2}) \Sigma_{A_e}^{-1/2} U_e.$$

The products $\phi_{e_1,i} \phi_{e_2,j}$ are in general not orthonormal, and $B_u^\top \Gamma_u B_u = I$, where Γ_u is their Gram matrix in $L^2(\nu_{A_e})$.

Proof. Write $\Sigma_{e'} := \Sigma_{A_{e'}}$ and $M_{e'} := T^{(A_{e'})}$, and order the legs so that $\Xi_{A_e} = \Xi_{A_{e_1}} \otimes \Xi_{A_{e_2}}$. (i) By Proposition 4.3, the $U_{e'}$ are the left singular vectors of the matricizations of $\tilde{T}$, so Proposition 2.14, applied to $\tilde{T}$, gives $U_e = (U_{e_1} \otimes U_{e_2}) B_u$. Since $\tilde{\Xi}_{A_e} = \tilde{\Xi}_{A_{e_1}} \otimes \tilde{\Xi}_{A_{e_2}}$, $\Phi_e = U_e^\top \tilde{\Xi}_{A_e} = B_u^\top (\Phi_{e_1} \otimes \Phi_{e_2})$, which is (4.8). The columns of B_u are orthonormal because those of U_e are orthonormal and lie in the range of the isometry $U_{e_1} \otimes U_{e_2}$, and the products are orthonormal by the independence of $X_{A_{e_1}}$ and $X_{A_{e_2}}$ and Proposition 4.2(i). (ii) For $i = 1, 2$, since $\Sigma_{A_{e_i}^c}$ is nonsingular, $\text{col}(U_{e_i}) = \text{col}(\tilde{M}_{e_i}) = \Sigma_{e_i}^{1/2} \text{col}(M_{e_i})$, i.e. $\text{col}(M_{e_i}) = \text{col}(\Sigma_{e_i}^{-1/2} U_{e_i})$. Since Σ_e is nonsingular, $\phi_{e,q} = (\Sigma_e^{-1/2} u_{e,q})^\top \Xi_{A_e}$, and $\Sigma_e^{-1/2} u_{e,q} \in \text{col}(M_e \Sigma_{A_e}^{1/2}) \subseteq \text{col}(M_e) \subseteq \text{col}(M_{e_1}) \otimes \text{col}(M_{e_2})$ by (2.5). Hence $\Sigma_e^{-1/2} u_{e,q} = (\Sigma_{e_1}^{-1/2} U_{e_1} \otimes \Sigma_{e_2}^{-1/2} U_{e_2}) b_q$ for a unique vector b_q , which is the q -th column of B_u because $U_{e_i}^\top \Sigma_{e_i}^{1/2} \Sigma_{e_i}^{-1/2} U_{e_i} = I$. Since $\Phi_{e_i} = U_{e_i}^\top \Sigma_{e_i}^{-1/2} \Xi_{A_{e_i}}$, this gives $\phi_{e,q} = b_q^\top (\Phi_{e_1} \otimes \Phi_{e_2})$, which is (4.8), and $B_u^\top \Gamma_u B_u = I$ expresses the orthonormality of the $\phi_{e,q}$. Finally, if ν_x is nondegenerate and $c^\top \Xi_A = 0$ almost surely for some A , then $(c \otimes c')^\top \Xi_{[N]} = (c^\top \Xi_A)(c'^\top \Xi_{A^c}) = 0$ almost surely for every c' , hence $c = 0$: all the matrices Σ_A are nonsingular. $\square$

Remark 4.9 (Features of the teacher are built from features of the teacher). Proposition 4.8 is the precise sense in which the data-weighted features are hierarchical. Each left singular function at a bond is a *bilinear* function of the left singular functions at the two bonds below it, and at a *leaf bond*, i.e. a bond e that joins a leaf a to its parent, $A_e = \{a\}$ and $\phi_{e,q} = u_{e,q}^\top \tilde{\xi}_a$ is linear in the whitened embedding. This motivates looking for the features of the teacher in successive layers of a DNN (Hypothesis 4.27). Under the product law the composition is orthonormal, and the data-weighted SVDs at all bonds form the HSVD of a single tensor. Under dependence, each bond has its own whitening, so the data-weighted SVDs at different bonds are not the HSVD of a single tensor; the composition (4.8) survives under the assumption of Proposition 4.8(ii), but without orthonormality. Without that assumption it can fail. Take three scalar blocks on the tree $((1, 2), 3)$, with $\xi_a(z) = (1, z)$, $X^{(1)}$ and $X^{(2)}$ independent and non-constant, $X^{(3)} = X^{(2)}$, and the teacher $f^* = \ell_a(x^{(1)}) x^{(2)} + \ell_b(x^{(1)}) x^{(3)}$, where $\ell_a = a^\top \xi_1$ and $\ell_b = b^\top \xi_1$ for linearly independent $a, b \in \mathbb{R}^2$. The environment of the leaf bond of leaf 1 always receives $x^{(2)} = x^{(3)}$, so the only left singular function there is proportional to $\ell_a + \ell_b$; at the leaf bond of leaf 2, the left singular functions span 1 and $x^{(2)}$. At the bond e above leaves 1 and 2, whose environment receives $x^{(3)}$ independently of $(x^{(1)}, x^{(2)})$ on recombined inputs, they span $\ell_a(x^{(1)}) x^{(2)}$ and $\ell_b(x^{(1)})$, and no nonzero combination of these two functions is a combination of the products $(\ell_a + \ell_b)(x^{(1)})$ and $(\ell_a + \ell_b)(x^{(1)}) x^{(2)}$. Finally, the two child bonds of the root define the same bipartition, with left and right exchanged, and therefore have the same singular values and atoms.

Remark 4.10 (When are the data-weighted features properties of the data?). The data-weighted SVD is gauge invariant, but it is computed from the tensor $T^\star$, whereas the data only determine $f^\star$ as an element of $L^2(\nu_x)$. Let T and T' define the same function ν_x -almost surely. Since $\mathbb{E}_{\nu_e^\otimes}[g_{T-T'}^2] = \|\tilde{M}_e(T) - \tilde{M}_e(T')\|_F^2$, they have the same data-weighted SVD at e if and only if $g_{T-T'}$ also vanishes $\nu_e^\otimes$ -almost surely. Hence the data-weighted SVD at e is determined by the values of the function on the data, for every tensor with the given embeddings, if and only if

$$\text{every function } g_D = \langle D, \Xi_{[N]} \rangle \text{ that vanishes } \nu_x\text{-almost surely also vanishes } \nu_e^\otimes\text{-almost surely.} \quad (4.9)$$

This holds under the product law, whenever $\nu_e^\otimes$ is absolutely continuous with respect to ν_x , and whenever ν_x is nondegenerate, since then no nonzero g_D vanishes almost surely. If ν_x and $\nu_e^\otimes$ are mutually absolutely continuous, as for jointly Gaussian blocks with a nonsingular covariance, almost sure identities are the same under both laws, and $r_e^\star$ is also the rank of $f^\star$ across the cut on the data, i.e. the smallest r such that $f^\star = \sum_{k \leq r} a_k(x_{A_e}) b_k(x_{A_e^c})$ ν_x -almost surely, with $a_k \in L^2(\nu_{A_e})$ and $b_k \in L^2(\nu_{A_e^c})$. Condition (4.9) fails when the support of ν_x is degenerate: in the example of Remark 4.9, the teachers $f^\star$ and $(\ell_a + \ell_b)(x^{(1)}) x^{(2)}$ coincide on the data but have data-weighted ranks 2 and 1 at the bond e , and for teachers with repeated inputs the condition fails at every bond (Section 4.6). The data-weighted features are then properties of the teacher's computation, planted by the choice of $T^\star$, rather than of the function that it computes on the data.

4.3 Atoms: identity and strength

Definition 4.11 (Atoms). For a bond e and $i, j \leq r_e^\star$, let $\chi_{e,ij}^\star(x) := \phi_{e,i}^\star(x_{A_e}) \psi_{e,j}^\star(x_{A_e^c})$, and $\chi_{e,q}^\star := \chi_{e,qq}^\star$. The *atom* (e, q) of the teacher is the function $\sigma_{e,q}^\star \chi_{e,q}^\star$. We call $\chi_{e,q}^\star$ the *normalized atom*, $\phi_{e,q}^\star$ its *left factor* and $\psi_{e,q}^\star$ its *right factor*. For a set B of modes at e , the *block atom* is $m_{e,B}^\star := \sum_{q \in B} \sigma_{e,q}^\star \chi_{e,q}^\star$.

An atom is one mode of the data-weighted SVD at one bond: one ‘‘atom of computation’’ of the teacher. Its left factor is a function computed by the subtree below e , its right factor is a function computed by the rest of the tree, and the atom is their product, weighted by the strength $\sigma_{e,q}^\star$ with which the two sides are coupled.

Proposition 4.12 (Atoms). Let e be a bond.

- (i) For every joint law, $f^\star = \sum_{q \leq r_e^\star} \sigma_{e,q}^\star \chi_{e,q}^\star$ almost surely, and in $L^2(\nu_e^\otimes)$: the $\chi_{e,ij}^\star$, $i, j \leq r_e^\star$, are orthonormal; $\|\sigma_{e,q}^\star \chi_{e,q}^\star\| = \sigma_{e,q}^\star$ and $\|f^\star\|^2 = \sum_q \sigma_{e,q}^{*2}$; and, for $r < r_e^\star$, the truncation $f_{e,r}^\star := \sum_{q \leq r} \sigma_{e,q}^\star \chi_{e,q}^\star$ minimizes $\|f^\star - g\|$ over the functions g of rank at most r across $(A_e \mid A_e^c)$, i.e. of the form $g = \sum_{k \leq r} a_k(x_{A_e}) b_k(x_{A_e^c})$, with $\|f^\star - f_{e,r}^\star\|^2 = \sum_{q > r} \sigma_{e,q}^{*2}$.
- (ii) If X_{A_e} and $X_{A_e^c}$ are independent, in particular under the product law, all the statements of (i) hold in $L^2(\nu_x)$.

Proof. (i) The expansion is (4.6), and orthonormality is Proposition 4.2(iii). The truncation property is the Eckart–Young theorem (Eckart and Young, 1936), which holds verbatim for Hilbert–Schmidt operators, applied to the operator K of the proof of Proposition 4.2: its approximations of rank at most r are exactly the kernels of rank at most r across the cut. (ii) In this case $\nu_e^\otimes = \nu_x$. $\square$

Dependent blocks. Under dependence, the energies of Proposition 4.12(i) are energies on recombined inputs. On the data, the inner products $\langle \chi_{e,ij}^\star, \chi_{e,kl}^\star \rangle = \mathbb{E}[\phi_{e,i}^\star \phi_{e,k}^\star \psi_{e,j}^\star \psi_{e,l}^\star]$ involve mixed moments across the cut, so the atoms are correlated, $\|f^\star\|^2 \neq \sum_q \sigma_{e,q}^{*2}$ in general, and $\sigma_{e,q}^{*2}$ is not the part of $\mathbb{E}[f^\star(X)^2]$ carried by the atom. The truncation need not be optimal either. For instance, if

X_{A_e} is a deterministic function of X_{A_e} , every function of x coincides on the data with a function of x_{A_e} alone, hence has rank one across the cut; the best approximation of rank one in $L^2(\nu_x)$ is then f^* itself, whereas the truncation $f_{e,1}^*$ generally has a positive error. Section 4.4 defines output observables that take these correlations into account.

Identity and strength. The normalized factors $\phi_{e,q}^*$ and $\psi_{e,q}^*$ specify *which* computation the atom performs on each side of the interface; the singular value specifies its *strength*. Under the product law, $\sigma_{e,q}^{*2}$ is the part of the teacher’s second moment $\mathbb{E}[f^*(X)^2] = \|f^*\|^2$ carried by the atom, and replacing the truncation $f_{e,q-1}^*$ by $f_{e,q}^*$, i.e. adding the atom to the stronger atoms at e , lowers the population loss $\mathcal{L}(f) = \frac{1}{2}\mathbb{E}[(f(x) - y)^2]$ by $\sigma_{e,q}^{*2}/2$. Under dependence, these statements hold on recombined inputs, while on the data the decrease of the loss due to an atom also depends on its correlations with the other atoms. In all cases, a student can have acquired the identity of an atom, for instance by computing its factors internally, without having amplified it in its output; conversely, its output can contain a weak version of the right atom. Direction acquisition and amplification must therefore be tracked separately (Diagonal TTN note, 2026, Sections 3.2 and 9.4).

Remark 4.13 (Degenerate atoms). If $\sigma_{e,q}^*$ is a simple singular value, then $(\phi_{e,q}^*, \psi_{e,q}^*)$ is unique up to a simultaneous change of sign, which leaves $\chi_{e,q}^*$ unchanged: the atom is canonical. If instead a set B of $k \geq 2$ singular values equals a common value σ_B^* , then $(U_{e,B}^* R, V_{e,B}^* R)$ is another valid choice of the corresponding singular vectors for any $R \in O(k)$ (Remark 2.15). The canonical objects are then the left and right subspaces $\text{span}\{\phi_{e,q}^* : q \in B\}$ and $\text{span}\{\psi_{e,q}^* : q \in B\}$, and the block atom $m_{e,B}^* = \sigma_B^* \sum_{q \in B} \chi_{e,q}^*$, which is invariant because $\sum_q (UR)_{:,q} \otimes (VR)_{:,q} = \sum_q u_q \otimes v_q$ when $RR^\top = I$. Individual atoms inside a degenerate block have no separate meaning, and Example 2.42 shows that such blocks occur in natural tasks. When singular values are distinct but their gaps are comparable to the estimation error, the cluster should also be treated as a block, since singular functions are stable only relative to the gaps (Remark 2.17). The block atom is then still stable, because it corresponds to the part of $\hat{M}_e^*$ whose singular values lie in a window separated from the rest of the spectrum. The observables below are defined for a partition of the modes at e into such blocks, and reduce to single atoms for simple singular values.

Remark 4.14 (Atoms are attached to bonds). For each bond e , Proposition 4.12 decomposes the same function f^* into features with strengths. Under the product law, the features are orthonormal, exactly as in (1.1), with $\chi_m = \chi_{e,q}^*$ and $\lambda_m = \sigma_{e,q}^{*2}$; under dependence, they are orthonormal only in the decoupled geometry, and the coefficients of (1.1) must be replaced by coordinates of projections (Section 4.4). Different bonds give different decompositions. The teacher therefore supplies not one flat dictionary of features but one decomposition per bond, and these decompositions are linked by the composition (4.8). Accordingly, observables are indexed by a bond and a mode, or a block.

4.4 Output observables: has the predictor acquired an atom?

The most direct question is whether the predictor f_t of a student contains the atom $\sigma_{e,q}^* \chi_{e,q}^*$. The distance $\|f_t - \sigma_{e,q}^* \chi_{e,q}^*\|$ is not the right observable: a student that has learned this atom *and* the other ones is far from it. The right operation is a projection, which measures the component of f_t along the atom. Since a student can also couple the teacher’s factors in ways that the teacher does not, pairing $\phi_{e,i}^*$ with $\psi_{e,j}^*$ for $i \neq j$, we measure all pairings at once. Under dependence, the products $\chi_{e,ij}^*$ are not orthonormal in $L^2(\nu_x)$, so inner products with them are not coordinates. All output observables are therefore defined through orthogonal projections in $L^2(\nu_x)$, the geometry of the loss; they reduce to inner products when the two sides of the cut are independent.

Definition 4.15 (Coupling matrix and acquisition coefficients). Let $\Phi_e^* := (\phi_{e,1}^*, \dots, \phi_{e,r_e^*}^*)^\top$, $\Psi_e^* := (\psi_{e,1}^*, \dots, \psi_{e,r_e^*}^*)^\top$, and let $\chi_e^* := \text{vec}(\Phi_e^* \Psi_e^{*\top})$ be the vector of the products $\chi_{e,ij}^*$. The *joint Gram matrix* at e is $\Gamma_e := \mathbb{E}[\chi_e^*(X) \chi_e^*(X)^\top]$, with entries $\langle \chi_{e,ij}^*, \chi_{e,kl}^* \rangle$, and we let $\mathcal{D}_e := \text{span}\{\chi_{e,ij}^* : i, j \leq r_e^*\}$. The *coupling matrix* of $f \in L^2(\nu_x)$ at e is the matrix $C_e[f] \in \mathbb{R}^{r_e^* \times r_e^*}$ defined by

$$\text{vec } C_e[f] := \Gamma_e^+ \mathbb{E}[f(X) \chi_e^*(X)]. \quad (4.10)$$

Fix a partition of the modes $\{1, \dots, r_e^*\}$ into blocks B (Remark 4.13), let $\mathcal{A}_e := \text{span}\{m_{e,B}^*\}$ and $\Lambda_e := (\langle m_{e,B}^*, m_{e,B'}^* \rangle)_{B,B'}$, and let

$$a_e[f] := \Lambda_e^+(\langle f, m_{e,B}^* \rangle)_B,$$

with components $a_{e,B}[f]$. Along a trajectory f_t , the *output acquisition coefficient* of the block B is $a_{e,B}(t) := a_{e,B}[f_t]$, and that of the atom (e, q) is $a_{e,q}(t) := a_{e,\{q\}}(t)$ when $\{q\}$ is a block.

When X_{A_e} and $X_{A_e^c}$ are independent, Proposition 4.16(iv) below shows that $\Gamma_e = I$, $C_e[f]_{ij} = \langle f, \chi_{e,ij}^* \rangle$, and

$$a_{e,q}(t) = \frac{C_e[f_t]_{qq}}{\sigma_{e,q}^*} = \frac{\langle f_t, \chi_{e,q}^* \rangle}{\sigma_{e,q}^*}, \quad a_{e,B}(t) = \frac{\langle S_{e,BB}^*, C_e[f_t]_{BB} \rangle_F}{\|S_{e,BB}^*\|_F^2},$$

where $C_{BB'}$ denotes the submatrix of a matrix C with rows in B and columns in B' ; for a block of equal singular values, $a_{e,B}(t) = \text{tr } C_e[f_t]_{BB} / \text{tr } S_{e,BB}^*$. By Remark 4.14, $a_{e,q}$ is then the coefficient (1.1) of the decomposition of f^* at e . In general, $a_e[f]$ collects the coefficients of the best approximation of f by a combination of the teacher's block atoms at e : $a_{e,B}(t) = 0$ means that the block is absent from the predictor, and $a_{e,B}(t) = 1$ that it is present with the teacher's strength. The coupling matrix contains more information than these coefficients.

Proposition 4.16 (Exact error decomposition at a bond). Let Π_e , $P_{\mathcal{W}_e}$ and $P_{\mathcal{A}_e}$ be the orthogonal projectors of $L^2(\nu_x)$ onto $\mathcal{D}_e$, onto $\mathcal{W}_e := \text{span}\{\chi_{e,ij}^* : i, j \in B \text{ for some block } B\}$ and onto $\mathcal{A}_e$, so that $\mathcal{A}_e \subseteq \mathcal{W}_e \subseteq \mathcal{D}_e$. For every joint law and every $f \in L^2(\nu_x)$:

- (i) $\Pi_e f = \Phi_e^{*\top} C_e[f] \Psi_e^*$, $\|\Pi_e f\|^2 = \text{vec}(C_e[f])^\top \Gamma_e \text{vec}(C_e[f])$, and $P_{\mathcal{A}_e} f = \sum_B a_{e,B}[f] m_{e,B}^*$;
- (ii) if Γ_e is nonsingular, $C_e[f^*] = S_e^*$, and if Λ_e is nonsingular, $a_e[f^*] = \mathbf{1}$. If Γ_e (resp. Λ_e) is singular, a nonzero combination of the products $\chi_{e,ij}^*$ (resp. of the block atoms) vanishes almost surely, and the coordinates of $C_e[f]$ (resp. of $a_e[f]$) along such combinations are not determined by f ;
- (iii) the excess error splits into four nonnegative terms,

$$\begin{aligned} \|f - f^*\|^2 &= \underbrace{\|P_{\mathcal{A}_e}(f - f^*)\|^2}_{\text{amplitudes}} + \underbrace{\|(P_{\mathcal{W}_e} - P_{\mathcal{A}_e})f\|^2}_{\text{within blocks}} \\ &\quad + \underbrace{\|(\Pi_e - P_{\mathcal{W}_e})f\|^2}_{\text{across blocks}} + \underbrace{\|(I - \Pi_e)f\|^2}_{\text{outside}}, \end{aligned} \quad (4.11)$$

and, if Λ_e is nonsingular, the first term is $(a_e[f] - \mathbf{1})^\top \Lambda_e (a_e[f] - \mathbf{1})$;

- (iv) if X_{A_e} and $X_{A_e^c}$ are independent, in particular under the product law, then $\Gamma_e = I$, $C_e[f]_{ij} = \langle f, \chi_{e,ij}^* \rangle$ and $\Lambda_e = \text{diag}(\|S_{e,BB}^*\|_F^2)_B$, and, writing $C = C_e[f]$, $S = S_e^*$ and $a_B = a_{e,B}[f] = \langle S_{BB}, C_{BB} \rangle_F / \|S_{BB}\|_F^2$, the four terms of (4.11) are

$$\sum_B \|S_{BB}\|_F^2 (a_B - 1)^2, \quad \sum_B (\|C_{BB}\|_F^2 - a_B^2 \|S_{BB}\|_F^2), \quad \sum_{B \neq B'} \|C_{BB'}\|_F^2, \quad \|f\|^2 - \|C\|_F^2;$$

the first two add up to $\sum_B \|C_{BB} - S_{BB}\|_F^2$, and when all blocks are single modes they are $\sum_q \sigma_{e,q}^{*2} (a_{e,q} - 1)^2$ and 0;

- (v) if the blocks are unions of sets of equal singular values, then $\mathcal{A}_e$, $\mathcal{W}_e$, $\mathcal{D}_e$, the block atoms, the coefficients $a_{e,B}[f]$, the four terms of (4.11) and the singular values of $C_e[f]$ do not depend on the choices of singular functions allowed by Remark 4.13.

Proof. (i) If $\Gamma_e c = 0$, then $\mathbb{E}[(c^\top \chi_e^*)^2] = 0$, so $c^\top \chi_e^* = 0$ almost surely and $c^\top \mathbb{E}[f \chi_e^*] = 0$: the vector $\mathbb{E}[f \chi_e^*]$ lies in the range of Γ_e . The normal equations $\Gamma_e \beta = \mathbb{E}[f \chi_e^*]$ of the least-squares problem $\min_\beta \|f - \beta^\top \chi_e^*\|$ are therefore solved by $\beta = \text{vec } C_e[f]$, so $\Pi_e f = \text{vec}(C_e[f])^\top \chi_e^* = \Phi_e^{\star\top} C_e[f] \Psi_e^*$ and $\|\Pi_e f\|^2 = \beta^\top \Gamma_e \beta$. The same argument applies to the block atoms and Λ_e . (ii) By (4.6), $f^* = \text{vec}(S_e^*)^\top \chi_e^* = \sum_B m_{e,B}^*$ almost surely, so $\mathbb{E}[f^* \chi_e^*] = \Gamma_e \text{vec } S_e^*$ and $(\langle f^*, m_{e,B}^* \rangle)_B = \Lambda_e \mathbf{1}$, which gives the first statement. If $\Gamma_e c = 0$ with $c \neq 0$, then $c^\top \chi_e^* = 0$ almost surely, so the coefficient vectors β and $\beta + c$ define the same function; the same holds for Λ_e . (iii) Since the subspaces are nested, $P_{\mathcal{A}_e}$, $P_{\mathcal{W}_e} - P_{\mathcal{A}_e}$, $\Pi_e - P_{\mathcal{W}_e}$ and $I - \Pi_e$ are the orthogonal projectors onto mutually orthogonal subspaces whose sum is $L^2(\nu_x)$. Applying them to $f - f^*$ gives an orthogonal decomposition, and the last three annihilate $f^* \in \mathcal{A}_e$. The expression of the first term follows from (i) and (ii). (iv) By Proposition 4.12(ii), the $\chi_{e,ij}^*$ are orthonormal, so $\Gamma_e = I$ and $C_e[f]_{ij} = \langle f, \chi_{e,ij}^* \rangle$. The block atoms $m_{e,B}^* = \sum_{q \in B} \sigma_{e,q}^* \chi_{e,qq}^*$ are orthogonal, with $\|m_{e,B}^*\|^2 = \|S_{BB}\|_F^2$ and $\langle f, m_{e,B}^* \rangle = \langle S_{BB}, C_{BB} \rangle_F$, which gives Λ_e , a_B , $\|P_{\mathcal{A}_e}(f - f^*)\|^2 = \sum_B (a_B - 1)^2 \|S_{BB}\|_F^2$ and $\|P_{\mathcal{A}_e} f\|^2 = \sum_B a_B^2 \|S_{BB}\|_F^2$. Since $\mathcal{W}_e$ and $\mathcal{D}_e$ have orthonormal bases of products, $\|P_{\mathcal{W}_e} f\|^2 = \sum_B \|C_{BB}\|_F^2$ and $\|\Pi_e f\|^2 = \|C\|_F^2$, and the last three terms follow by taking differences. Adding the first two and using $a_B \|S_{BB}\|_F^2 = \langle S_{BB}, C_{BB} \rangle_F$ gives $\sum_B \|C_{BB} - S_{BB}\|_F^2$; for single modes, $a_q = C_{qq}/\sigma_{e,q}^*$ and the second term vanishes. (v) An allowed change of singular functions replaces Φ_e^* and Ψ_e^* by $R^\top \Phi_e^*$ and $R^\top \Psi_e^*$, where R is block diagonal and orthogonal: a sign for each simple singular value and an orthogonal matrix for each set of equal singular values. It leaves the spans of $\{\phi_{e,q}^* : q \in B\}$ and $\{\psi_{e,q}^* : q \in B\}$ unchanged, hence $\mathcal{W}_e$ and $\mathcal{D}_e$, and it leaves each block atom $m_{e,B}^* = \Phi_{e,B}^{\star\top} S_{e,BB}^* \Psi_{e,B}^*$ unchanged, because $R_B^\top S_{e,BB}^* R_B = S_{e,BB}^*$; hence $\mathcal{A}_e$, Λ_e , $a_e[f]$ and the four terms are unchanged. Finally, $C_e[f] \mapsto R^\top C_e[f] R$, which preserves singular values. $\square$

Interpretation. For noiseless labels, $\|f - f^*\|^2 = 2\mathcal{L}(f)$; with independent zero-mean label noise, it is twice the excess loss. For every bond e , (4.11) therefore splits the excess loss of a student into four nonnegative parts:

- (a) an *amplitude* error: the student contains the teacher's block atoms with the wrong amplitudes, typically because an atom has not yet been amplified ($a_{e,B} < 1$) or has overshoot;
- (b) an error *within blocks*: inside the factor spaces of a block, the student departs from the teacher's block atom, for instance by pairing the left and right subspaces of a degenerate block differently from the teacher (a coupling block $C_{BB} = \sigma_B^* R$ with an orthogonal $R \neq I$ has the right strengths and subspaces but the wrong pairing), or by giving the modes of a cluster the wrong relative strengths; this term vanishes when all blocks are single modes;
- (c) an error *across blocks*: the student couples a left factor of the teacher with a right factor that belongs to a different block;
- (d) an *outside* error: the energy of f outside the teacher's factor spaces at e , i.e. the student uses functions of $x_{A_e^c}$ or of $x_{A_e^c}$ that the teacher does not use across this cut.

Each bond gives a different split of the same number. Matching the coefficients $a_{e,q}$ while carrying a large outside error is not a success (Surrogate note, 2026, Section 7.1), so the four terms should be reported together, for instance divided by $\|f^*\|^2$, so that they add up to the relative error $\|f - f^*\|^2 / \|f^*\|^2$. Under dependence, the decomposition is still exact, but the subspaces spanned by different pairings are no longer orthogonal: the nesting $\mathcal{A}_e \subseteq \mathcal{W}_e \subseteq \mathcal{D}_e$ attributes the variance that a mispaired product shares with a teacher atom to the atom. The terms must then be computed

with the Gram matrices Γ_e and Λ_e : inserting the raw inner products $\mathbb{E}[f\chi_{e,ij}^*]$ into the formulas of (iv) gives terms that no longer add up to $\|f - f^*\|^2$, and that can be negative. If only the leading $r < r_e^*$ modes are used, the decomposition holds with the projectors onto the corresponding smaller subspaces, applied to $f - f^*$; the terms then also contain the omitted part of the teacher.

Identifiability. The matrix Λ_e is small, of the size of the number of blocks, and it is nonsingular as soon as the block atoms are linearly independent in $L^2(\nu_x)$; the joint Gram matrix Γ_e has size r_e^{*2} . Both are nonsingular when ν_x is nondegenerate, because the products $\chi_{e,ij}^*$, and hence the block atoms, are functions g_D with linearly independent coefficient tensors D . Both are computed once from the teacher, analytically or on a large independent sample, and their condition numbers should be reported. When one of them is singular or badly conditioned, the corresponding coordinates are not identifiable (Proposition 4.16(ii)), but the projections are: blocks should then be merged, and the subspace terms of (4.11), which are always identifiable, reported. Under dependence, the coefficients of non-orthogonal features also depend on the family of features against which they are computed: $a_{e,q}$ controls for the other teacher atoms, whereas the diagonal entry $C_e[f]_{qq}/\sigma_{e,q}^*$ of the coupling matrix controls in addition for the mispaired products. The two coincide when the two sides of the cut are independent.

Remark 4.17 (Removing an atom from the output is not informative). One may try to assess whether an atom is useful to the student by removing the student's own component along it and measuring the resulting change of the loss. When the two sides of the cut are independent, this component is $C_e[f]_{qq} \chi_{e,q}^*$, and for $f^{(-q)} := f - C_e[f]_{qq} \chi_{e,q}^*$, expanding the square, with or without independent label noise, and using $\langle f - f^*, \chi_{e,q}^* \rangle = C_e[f]_{qq} - \sigma_{e,q}^*$ gives

$$\mathcal{L}(f^{(-q)}) - \mathcal{L}(f) = \frac{\sigma_{e,q}^{*2}}{2} \left(1 - (1 - a_{e,q})^2\right), \quad a_{e,q} = \frac{C_e[f]_{qq}}{\sigma_{e,q}^*}.$$

This is a function of $a_{e,q}$ alone: it is positive for $0 < a_{e,q} < 2$, maximal at $a_{e,q} = 1$, and negative for $a_{e,q} < 0$ or $a_{e,q} > 2$. It carries no information beyond $a_{e,q}$. Under dependence, it also involves the correlations between the components of the output, but still not how the student computes the atom. Causal questions have to be asked inside the network (Section 4.8).

4.5 The functional HSVD of a student

The coupling matrix describes a student in the coordinates of the teacher. A complementary, teacher-free description uses the student's own decomposition across the same cuts, in the geometry of Proposition 4.2.

Definition 4.18 (Functional HSVD). Let e be a bond of the teacher's tree and $f \in L^2(\nu_e^\otimes)$. The operator $K_e[f] : L^2(\nu_{A_e^c}) \rightarrow L^2(\nu_{A_e})$, $(K_e[f]\psi)(x_{A_e}) := \mathbb{E}[f(x_{A_e}, X'_{A_e^c}) \psi(X'_{A_e^c})]$ with $X'_{A_e^c} \sim \nu_{A_e^c}$, is Hilbert–Schmidt, with Hilbert–Schmidt norm $\|f\|_{L^2(\nu_e^\otimes)}$, and its SVD gives the Schmidt decomposition

$$f(x) = \sum_{q \geq 1} s_{e,q}(f) \phi_{e,q}^f(x_{A_e}) \psi_{e,q}^f(x_{A_e^c}) \quad \text{in } L^2(\nu_e^\otimes), \quad (4.12)$$

with $s_{e,1}(f) \geq s_{e,2}(f) \geq \dots \geq 0$ and orthonormal families $(\phi_{e,q}^f)_q$ in $L^2(\nu_{A_e})$ and $(\psi_{e,q}^f)_q$ in $L^2(\nu_{A_e^c})$. The family of these decompositions over the bonds of the teacher's tree is the *functional HSVD* of f (Surrogate note, 2026, Section 7.4), (Diagonal TTN note, 2026, Section 9.2).

The functional HSVD only uses the values of f , so it is defined in the same way for a DNN, for a TTN on any tree and for a polynomial surrogate, and its ranks are not bounded by any width. By Proposition 4.2(iii), the functional HSVD of f^* , and that of any TTN student on the teacher's tree with the same leaf embeddings, is its data-weighted HSVD. For TTNs, the data-weighted hierarchical features of Definition 4.1 and the functional HSVD are therefore the same object, computed in two ways: from the cores and the moments of the data, or from the values of the function. This is what makes the comparison between a TTN and a DNN meaningful at the level of decompositions. The computational HSVD of Definition 2.34, which describes how a polynomial network computes, is a different object (Section 4.6).

Proposition 4.19 (What the functional HSVD measures). Let e be a bond and $f, g \in L^2(\nu_e^\otimes)$.

- (i) $|s_{e,q}(f) - s_{e,q}(g)| \leq \|f - g\|_{L^2(\nu_e^\otimes)}$ for every q .
- (ii) If X_{A_e} and $X_{A_e^c}$ are independent, in particular under the product law, then (4.12) is the Schmidt decomposition of f in $L^2(\nu_x)$; it depends only on f as an element of $L^2(\nu_x)$, and $\sum_q s_{e,q}(f)^2 = \|f\|^2$.
- (iii) In general, the functional HSVD depends on the values of f on the support of $\nu_e^\otimes$, which can be larger than the support of ν_x . If ν_x has a density w with respect to $\nu_e^\otimes$ and $w \geq c > 0$ $\nu_e^\otimes$ -almost everywhere, then $\|h\|_{L^2(\nu_e^\otimes)} \leq c^{-1/2} \|h\|_{L^2(\nu_x)}$ for every h , so that $|s_{e,q}(f) - s_{e,q}(g)| \leq c^{-1/2} \|f - g\|_{L^2(\nu_x)}$: functions that are close on the data have close functional HSVDs.

Proof. (i) The cut operator is linear in f , so $K_e[f] - K_e[g] = K_e[f - g]$, whose operator norm is at most its Hilbert–Schmidt norm $\|f - g\|_{L^2(\nu_e^\otimes)}$; apply Weyl's inequality for the singular values of compact operators. (ii) In this case $\nu_e^\otimes = \nu_x$. (iii) $\|h\|_{L^2(\nu_x)}^2 = \int h^2 w d\nu_e^\otimes \geq c \|h\|_{L^2(\nu_e^\otimes)}^2$; combine with (i). $\square$

Dependent blocks. Proposition 4.19 makes precise what the functional HSVD of a student measures when the blocks are dependent. It is a decomposition on recombined inputs, in which the student is evaluated on combinations $(x_{A_e}, x'_{A_e^c})$ of the two sides of different data points. If these combinations lie in the support of the data, as for jointly Gaussian blocks, they are merely reweighted; otherwise the student is evaluated where it has never been trained. In the extreme case where $X_{A_e^c}$ is a deterministic function of X_{A_e} , every function has rank one across the cut in $L^2(\nu_x)$, so no decomposition of $L^2(\nu_x)$ across the cut can separate the teacher's modes, while the functional HSVD still does, using values of the student that the training data do not constrain. Under dependence, the functional HSVD therefore asks whether a student has learned the teacher's *computation*, in which the two sides of each bond interact only through the bond, rather than only its values on the data. This is a meaningful question for the surrogate hypothesis, and the teacher, TTN students and DNN students can all be compared in this geometry, but the answer is not a decomposition of the training objective. It should be labelled as such, and complemented by the on-distribution output observables of Section 4.4; Proposition 4.19(iii) indicates when the two agree.

Comparison with the teacher. The strengths $s_{e,q}(f_t)$ are compared with the $\sigma_{e,q}^*$, and the directions through the alignment of the leading singular subspaces,

$$\text{Al}_{e,r}(t) := \frac{1}{r} \text{tr} (P_{e,r}^{f_t} P_{e,r}^{\star}) \in [0, 1],$$

where $P_{e,r}^{f_t}$ and $P_{e,r}^{\star}$ are the orthogonal projectors of $L^2(\nu_{A_e})$ onto the spans of the r leading left singular functions of f_t and of f^* , with r chosen at a gap of both spectra (Diagonal TTN note, 2026, Section 9.4); right subspaces are compared in the same way. Alignment measures the acquisition

of directions, and singular values measure amplification. Both are stable: by Proposition 4.19(i), $|s_{e,q}(f_t) - \sigma_{e,q}^*| \leq \|f_t - f^*\|_{L^2(\nu_e^\otimes)}$, and the deviation of the subspaces is controlled by the Davis–Kahan theorem relative to the gap (Local TN note, 2026, Section 8.3). When the two sides of the cut are independent, the coupling matrix sees the part of this decomposition that lies in the teacher’s factor spaces: the singular values of $C_e[f]$ are those of $\Pi_e f$, and $s_{e,q}(\Pi_e f) \leq s_{e,q}(f)$ for every q , because the cut operator of $\Pi_e f$ is that of f multiplied on both sides by orthogonal projectors. Under dependence, Π_e is a projection in $L^2(\nu_x)$ while the functional HSVD lives in $L^2(\nu_e^\otimes)$, and no such inequality holds in general.

Estimation. Draw $x_1^L, \dots, x_{n_L}^L \sim \nu_{A_e}$ and $x_1^R, \dots, x_{n_R}^R \sim \nu_{A_e^c}$ independently, for instance as the A_e -parts and the A_e^c -parts of two disjoint sets of data points, and form the crossed-grid matrix

$$\hat{K}_e[f] := \left[\frac{f(x_k^L, x_l^R)}{\sqrt{n_L n_R}} \right]_{k \leq n_L, l \leq n_R}. \quad (4.13)$$

Every entry is evaluated at a recombined input, which is exactly what the decoupled geometry requires, so no conditional sampler is needed, even under dependence. If $\hat{K}_e[f] = \hat{U} \hat{S} \hat{V}^\top$, the singular values estimate the $s_{e,q}(f)$, and $\sqrt{n_L} \hat{U}_{:,q}$ and $\sqrt{n_R} \hat{V}_{:,q}$ estimate the values of $\phi_{e,q}^f$ at the x_k^L and of $\psi_{e,q}^f$ at the x_l^R (Local TN note, 2026, Section 8.4). Indeed, substituting (4.12) gives $\hat{K}_e[f] = \sum_q s_{e,q}(f) p_q w_q^\top$ with $p_q = (\phi_{e,q}^f(x_k^L)/\sqrt{n_L})_k$ and $w_q = (\psi_{e,q}^f(x_l^R)/\sqrt{n_R})_l$, and these vectors are orthonormal up to sampling error by the law of large numbers. Letting first n_R and then n_L tend to infinity reduces the consistency of the leading singular values to the convergence of the spectra of kernel matrices to those of integral operators (Koltchinskii and Giné, 2000). The same grid should be used for the teacher and all students. Since the $n_L n_R$ entries share rows and columns, uncertainty should be estimated with independent grids or by resampling rows and columns, not by an i.i.d. bootstrap of the entries; a 64×64 grid with a stability check at 128×128 is a reasonable start (Surrogate note, 2026, Sections 9 and 10.2).

Remark 4.20 (Cost, and proxies for large inputs). The size of $\hat{K}_e$ is chosen by the analyst and does not grow with the input dimension; the matrix costs $n_L n_R$ evaluations of the student. The number of samples needed is set by the number of modes to resolve, by their gaps and by the tails of f , rather than by $\dim x$. When even this is too expensive, a Galerkin version needs fewer evaluations. Let Φ^D and Ψ^D be finite families of functions of x_{A_e} and of $x_{A_e^c}$, orthonormal in $L^2(\nu_{A_e})$ and $L^2(\nu_{A_e^c})$ and containing the teacher’s singular functions. The SVD of $C_e^D[f] := \mathbb{E}_{\nu_e^\otimes} [f(X) \Phi^D(X_{A_e}) \Psi^D(X_{A_e^c})^\top]$ is the functional SVD of the projection of f onto $\text{span } \Phi^D \otimes \text{span } \Psi^D$ in $L^2(\nu_e^\otimes)$, and the captured fraction $\|C_e^D[f]\|_F^2 / \|f\|_{L^2(\nu_e^\otimes)}^2$ certifies how much of f the dictionaries resolve. Natural dictionaries add products of whitened leaf embeddings, or orthonormal polynomials of the blocks, to the teacher’s singular functions (Local TN note, 2026, Section 8.2). As the dictionaries become complete, the projections converge to f , and their singular values and well-separated singular subspaces converge to those of (4.12). The expectation defining $C_e^D[f]$ is over recombined inputs, and its Monte Carlo estimate from M of them has error $O(M^{-1/2})$ when the integrands have finite variance; under the product law, recombined inputs are ordinary samples. Under dependence, the SVD of the matrix $\mathbb{E}_{\nu_x} [f \Phi^D \Psi^D^\top]$ computed on ordinary samples is *not* a decomposition of f : when ν_x has a density w with respect to $\nu_e^\otimes$, it is the matrix $C_e^D[f w]$, which mixes the student with the dependence of the data.

Remark 4.21 (Scope: real data). For real data, the blocks are dependent and their joint law is unknown, which is part of the reason for using DNNs in the first place. Recombined inputs

(x_k^L, x_l^R) can then be far from the data, and the functional HSVD probes how a student extrapolates rather than what it has learned on the data. Conditional samplers of $\nu(dx_{A_e^c} | x_{A_e})$, when they are trustworthy, give access to on-distribution operators such as $\psi \mapsto \mathbb{E}[f(X) \psi(X_{A_e^c}) | X_{A_e} = \cdot]$, but their SVDs describe f weighted by the dependence of the data, as in Remark 4.20, not an expansion of f . Comparisons in representation space (Section 4.7) need no recombination at all (Surrogate note, 2026, Section 6.5). In the teacher–student benchmark, the joint law is chosen, and the functional HSVD under the product law serves as a ground truth against which the more scalable observables can be validated.

4.6 Teachers with repeated inputs

Consider now a teacher whose legs all receive the same input, $f^*(x) = \langle T^*, \bar{x}^{\otimes N} \rangle$ with $\bar{x} = (1, x)$, for instance the tied TTN of a deep polynomial network (Theorem 2.30); for a deep monomial network, $\bar{x}$ is replaced by x (Proposition 2.24). This is a *computational* TTN. By Remark 2.25, its legs are formal copies of one variable $X \sim \mu$, with $\bar{X} := (1, X)$, and the joint law of the legs is the image of μ under $x \mapsto (x, \dots, x)$, which is supported on the diagonal. This is not a small perturbation of the physical setting:

- (i) *The geometry of the loss.* By (4.7), $\langle g_T, g_{T'} \rangle_{L^2(\mu)} = \text{vec}(T)^\top \Sigma_{[N]} \text{vec}(T')$, now with $\Sigma_{[N]} = \mathbb{E}[\bar{X}^{\otimes N} (\bar{X}^{\otimes N})^\top]$, which contains the moments of X up to order $2N$. For Gaussian X , its entries are computed exactly by Wick contractions, but they do not reduce to products of leaf covariances. Since $\bar{X}^{\otimes N}$ is a symmetric tensor, $\Sigma_{[N]}$ is singular, and the population loss depends on T only through $\text{Sym } T$ (Remark 2.31(v)).
- (ii) *No physical cut.* The two sides of a bond are functions of the same variable, so every function of x has rank one across any cut in $L^2(\mu)$. Condition (4.9) fails: if D is antisymmetric in two legs that lie on opposite sides of the bond, $\langle D, \bar{x}^{\otimes N} \rangle$ vanishes identically, but in general not on recombined inputs. Different tensors computing the same function therefore have different decompositions across the bonds, as in Example 2.40.

Two kinds of observables are then needed: function-level observables, which depend only on the predictor and on μ , and computation-level observables, which describe the tied TTN itself.

Function track. For a law μ with enough moments, let $\mathcal{P}_k \subseteq L^2(\mu)$ be the space of polynomials of degree at most k , and $\mathcal{H}_k := \mathcal{P}_k \ominus \mathcal{P}_{k-1}$ the orthogonal complement of $\mathcal{P}_{k-1}$ in $\mathcal{P}_k$. Every $f \in L^2(\mu)$ has the orthogonal *degree decomposition* $f = \sum_{k \leq n} P_{\mathcal{H}_k} f + (I - P_{\mathcal{P}_n})f$, for every n , and a polynomial teacher of degree at most N is the sum of its *degree components* $f_k^* := P_{\mathcal{H}_k} f^*$, $k \leq N$, with $\|f^*\|^2 = \sum_k \|f_k^*\|^2$. For Gaussian μ , $\mathcal{H}_k$ is the k -th Wiener chaos, spanned by the Hermite polynomials of total degree k (Janson, 1997); for a general law, an empirically orthonormalized polynomial dictionary is used. The degree components, their energies and finer decompositions inside each component are function-level objects: they depend only on f^* and μ , and they are defined in the same way for every student. For instance, for standard Gaussian μ , the quadratic component is $f_2^*(x) = x^\top Q x - \text{tr } Q$ for a symmetric matrix Q , and it decomposes into the orthonormal features $((w_i^\top x)^2 - 1)/\sqrt{2}$, along the eigenvectors w_i of Q , with coefficients $\sqrt{2} \lambda_i(Q)$. The output observables of Section 4.4 apply with these features in place of the atoms: acquisition coefficients $a_k(t) := \langle f_t, f_k^* \rangle / \|f_k^*\|^2$, and the split of $\|f_t - f^*\|^2$ into the errors $\|P_{\mathcal{H}_k}(f_t - f^*)\|^2$ of each degree and the residual $\|(I - P_{\mathcal{P}_N})f_t\|^2$, all estimated on ordinary samples. A crossed grid would instead evaluate the multilinear extension on pairs (x, x') of different inputs, which depends on the off-diagonal part of T and not on the function; it should not be used as a function-level observable.

Computation track. The decompositions across the bonds of the tied TTN describe how the teacher computes. The Euclidean HSVD of Definition 2.34 is one such description. Definition 4.1, applied to the law of the legs on the diagonal, gives a data-weighted one: by Proposition 4.2(iii), it is the Schmidt decomposition of the function $(x, x') \mapsto \langle T^*, \bar{x}^{\otimes K^\ell} \otimes \bar{x}'^{\otimes (N-K^\ell)} \rangle$ in $L^2(\mu \otimes \mu)$ at a bond of height ℓ , in which the subtree below the bond receives copies of x and its environment receives copies of an independent input x' . For a polynomial network, it has a simple expression in terms of the network.

Proposition 4.22 (Data-weighted computational HSVD of a polynomial network). In the setting of Theorem 2.30, assume that $\mathbb{E}\|X\|^{2N} < \infty$, let $1 \leq \ell \leq L-1$, let $f_{>\ell}$ be the map computed by the layers above ℓ , so that $f_\theta = f_{>\ell} \circ h_\ell$, and let $m_\ell := K^{L-\ell}$. Define

$$\bar{g}_\ell(x) := \left(m_\ell f_\theta(x) - (h_\ell(x) - \mathbb{E}h_\ell)^\top \nabla f_{>\ell}(h_\ell(x)), \text{Cov}(h_\ell)^{1/2} \nabla f_{>\ell}(h_\ell(x)) \right) \in \mathbb{R}^{1+d_\ell},$$

where $\mathbb{E}h_\ell$ and $\text{Cov}(h_\ell)$ are taken under $X \sim \mu$. Then, at every bond of height ℓ , the squared data-weighted singular values of $T(\theta)$ are the nonzero eigenvalues of $m_\ell^{-2} \mathbb{E}[\bar{g}_\ell(X) \bar{g}_\ell(X)^\top]$, and, if w_q are corresponding orthonormal eigenvectors, the left singular functions are

$$\phi_{\ell,q}(x) = w_q^\top \left(1, (\text{Cov}(h_\ell)^{1/2})^+ (h_\ell(x) - \mathbb{E}h_\ell) \right).$$

Proof. Let e be a bond of height ℓ . By the proof of Theorem 2.30, the message of the subtree below e is $h_e(x) = \Phi_\ell(\bar{x}, \dots, \bar{x}) = \bar{h}_\ell(x)$. Let $\mathcal{F}_\ell$ be the m_ℓ -linear form computed by the vertices above height ℓ , whose arguments are the bonds of height ℓ , and let $F_\ell(\bar{h}) := \mathcal{F}_\ell(\bar{h}, \dots, \bar{h})$; it is a homogeneous polynomial of degree m_ℓ on H_ℓ , and $F_\ell(1, h) = f_{>\ell}(h)$ by (2.10). The environment message of e at an input x' is the linear form obtained from $\mathcal{F}_\ell$ by leaving the argument of e open and setting all other arguments to $\bar{h}_\ell(x')$. As in the proof of Proposition 2.33, $\mathcal{F}_\ell$ is invariant under the automorphisms of the tree above height ℓ , which act transitively on its arguments, so the m_ℓ partial maps of this form coincide, and $n_e(x') = m_\ell^{-1} \nabla F_\ell(\bar{h}_\ell(x'))$. By Euler's identity for homogeneous polynomials, the derivative of F_ℓ along the constant coordinate is $m_\ell f_{>\ell}(h) - h^\top \nabla f_{>\ell}(h)$ at $(1, h)$. Now let E be the invertible matrix with $E(a, h) = (a, h - a \mathbb{E}h_\ell)$, and apply Proposition 4.6 to the factorization $T^{(Ae)} = (F_e E^\top)(H_e E^{-1})^\top$, whose messages are $h'_e = E h_e = (1, h_\ell - \mathbb{E}h_\ell)$ and $n'_e = E^{-\top} n_e$. The second moment of h'_e is $\Sigma_e^{h'} = \text{diag}(1, \text{Cov}(h_\ell))$, and a direct computation gives $(\Sigma_e^{h'})^{1/2} n'_e(x') = m_\ell^{-1} \bar{g}_\ell(x')$. Under the decoupled law, the environment receives an input $X' \sim \mu$, so $(\Sigma_e^{h'})^{1/2} \Sigma_e^{n'} (\Sigma_e^{h'})^{1/2} = m_\ell^{-2} \mathbb{E}[\bar{g}_\ell \bar{g}_\ell^\top]$. Its nonzero eigenvalues are the squared singular values of $(\Sigma_e^{h'})^{1/2} (\Sigma_e^{n'})^{1/2}$, whose left singular vectors are the w_q , and Proposition 4.6 gives the singular functions. $\square$

The data-weighted computational HSVD at height ℓ thus ranks the whitened directions of the representation of layer ℓ , augmented by the constant direction, by the mean squared sensitivity of the rest of the network to them. The lower-right block of $\mathbb{E}[\bar{g}_\ell \bar{g}_\ell^\top]$ is the average gradient outer product AGOP_ℓ of Definition 4.28 below. The decomposition is invariant under the hidden-unit symmetries of Remark 2.26, which act on the bond by gauge transformations, and it is estimated from samples of X , with one forward and one backward pass per sample. For a DNN with a standard activation, AGOP_ℓ is defined in the same way, although it is no longer the matrix of a cut SVD; for its polynomial proxy and its TTN surrogate (Section 2.7), Proposition 4.22 applies exactly. The spectra and leading eigenspaces of AGOP_ℓ at each layer are therefore computation-level observables shared by a DNN and its polynomial counterparts. Like the Euclidean HSVD, however, the data-weighted computational HSVD is an invariant of the computation, not of the function: in Example 2.40, the two networks compute the same function but have data-weighted ranks 1 and 2 at height one, for every law of x with a density that is positive on a nonempty open set.

Comparing students with a teacher with repeated inputs. Function-level observables are computed for every student from its predictor, in the same way. Computation-level observables are compared through representations: the teacher’s hidden states $\bar{h}_\ell^\star$ and computational singular functions $\phi_{\ell,q}^\star$, which are functions of x , are probed in the layers of each student (Section 4.7), and the AGOP spectra of the students are compared with each other. Agreement in computation-level observables is evidence about mechanisms, but it does not identify copy-cut singular values with canonical features of the input–output function.

4.7 Representation observables: where is an atom represented?

Output observables say what a student computes, not where it computes it (Local TN note, 2026, Section 8.5). Let $h_\ell(x) \in \mathbb{R}^{d_\ell}$ be the representation at layer ℓ of a DNN student, as in (2.6), with $h_0 = x$. For a physical teacher, we ask whether the left factor $\phi_{e,q}^\star$, the right factor $\psi_{e,q}^\star$ and the normalized atom $\chi_{e,q}^\star$ can be read off h_ℓ . These are functions of the whole input, by composition with $x \mapsto x_{A_e}$ or $x \mapsto x_{A_e^c}$, so decoding them from $h_\ell(x)$ is well defined. The first two targets test whether the factors are available, and the third whether their product has been formed. For a teacher with repeated inputs, the targets are the teacher’s computational singular functions $\phi_{\ell,q}^\star$, its augmented hidden states $\bar{h}_\ell^\star$, as a vector target, and its degree components $f_k^\star$ (Section 4.6). As decoders we use affine maps, because the next layer acts on h_ℓ through an affine map.

Definition 4.23 (Linear acquisition scores). Let $h(x) \in \mathbb{R}^d$ be a representation with $\mathbb{E}\|h(X)\|^2 < \infty$. For a target $\zeta \in L^2(\nu_x)$ with $\text{Var } \zeta > 0$, let

$$R_h^2[\zeta] := 1 - \frac{\min_{w \in \mathbb{R}^d, b \in \mathbb{R}} \mathbb{E}[(\zeta(X) - w^\top h(X) - b)^2]}{\text{Var } \zeta} \in [0, 1]$$

be the population coefficient of determination of the best affine decoder. For a vector target $Z(x) \in \mathbb{R}^p$, let

$$R_h^2[Z] := \frac{1}{p_Z} \text{tr} \left(\text{Cov}(Z)^\top \text{Cov}(Z, h) \text{Cov}(h)^\top \text{Cov}(h, Z) \right), \quad p_Z := \text{rank Cov}(Z) \geq 1.$$

For layer ℓ of a student at time t , we write $R_\ell^2[\zeta](t)$. The three scalar scores of the atom (e, q) are $R_\ell^2[\phi_{e,q}^\star]$, $R_\ell^2[\psi_{e,q}^\star]$ and $R_\ell^2[\chi_{e,q}^\star]$; for a block B , the vector targets are $(\phi_{e,q}^\star)_{q \in B}$, $(\psi_{e,q}^\star)_{q \in B}$ and the products $(\chi_{e,ij}^\star)_{i,j \in B}$.

Proposition 4.24 (Closed forms and invariances). Let $h^\text{w} := (\text{Cov}(h)^{1/2})^\top (h - \mathbb{E}h)$ be the whitened representation.

- (i) $R_h^2[\zeta] = \text{Cov}(h, \zeta)^\top \text{Cov}(h)^\top \text{Cov}(h, \zeta) / \text{Var } \zeta = \|\mathbb{E}[h^\text{w} \zeta]\|^2 / \text{Var } \zeta$, and $w = \text{Cov}(h)^\top \text{Cov}(h, \zeta)$ is an optimal decoder.
- (ii) $R_h^2[Z]$ is the sum of the squared canonical correlations between h and Z (Hotelling, 1936), divided by p_Z . Equivalently, let $Z^\text{w} := (\text{Cov}(Z)^{1/2})^\top (Z - \mathbb{E}Z)$ be the whitened target; then $p_Z R_h^2[Z]$ is the total variance of Z^w explained by the best affine function of h . For a scalar target, $R_h^2[Z] = R_h^2[\zeta]$ with $\zeta = Z$.
- (iii) $R_h^2[\zeta]$ and $R_h^2[Z]$ do not change when h is replaced by $Qh + c$ and Z by $RZ + c'$, for invertible matrices Q and R . In particular, they are invariant under permutations and rescalings of hidden units, and the block scores $R_h^2[Z]$ are invariant under the rotations within degenerate blocks of Remark 4.13; the scalar scores of individual atoms inside a degenerate block are not.

Proof. (i) The optimal b centres the decoder, so the minimum equals $\min_w \mathbb{E}[(\zeta - \mathbb{E}\zeta - w^\top(h - \mathbb{E}h))^2]$. If $\text{Cov}(h)w = 0$, then $w^\top(h - \mathbb{E}h) = 0$ almost surely, hence $w^\top \text{Cov}(h, \zeta) = 0$. Thus $\text{Cov}(h, \zeta)$ lies in the range of $\text{Cov}(h)$, the normal equations $\text{Cov}(h)w = \text{Cov}(h, \zeta)$ are solved by $w = \text{Cov}(h)^+ \text{Cov}(h, \zeta)$, and the minimum is $\text{Var} \zeta - \text{Cov}(h, \zeta)^\top \text{Cov}(h)^+ \text{Cov}(h, \zeta)$. The second expression follows from $\mathbb{E}[h^\top \zeta] = (\text{Cov}(h)^{1/2})^\top \text{Cov}(h, \zeta)$, which holds because $h^\top$ is centred. (ii) The canonical correlations are the singular values of $(\text{Cov}(h)^{1/2})^\top \text{Cov}(h, Z)(\text{Cov}(Z)^{1/2})^\top$, and by cyclicity of the trace the sum of their squares is the trace in the definition of $R_h^2[Z]$. By (i) applied to each coordinate, the same trace is the total variance of the whitened target explained by h . (iii) Canonical correlations and best affine predictions depend only on the spaces of affine functions of h and of Z , which invertible affine maps leave unchanged. $\square$

None of these definitions involves the law of the blocks: they apply verbatim under dependence and for repeated inputs.

Reading the three scores. High scores $R_\ell^2[\phi_{e,q}^*]$ and $R_\ell^2[\psi_{e,q}^*]$ with a low $R_\ell^2[\chi_{e,q}^*]$ mean that both factors are available at layer ℓ but that their product has not been formed as an explicit feature; a later rise of $R_\ell^2[\chi_{e,q}^*]$ means that the composition has become explicit. At the last hidden layer, $f_\theta = c^\top h_L + b_{\text{out}}$ is itself an affine readout, so a student with $f_\theta \approx f^*$ represents the sum of the atoms there, but not necessarily each atom separately.

Remark 4.25 (What linear decodability does and does not mean). Linear decodability is deliberately restrictive: it says that a feature has become explicitly and cheaply accessible to the next layer. A low score does not show that the information is absent, since it can be encoded nonlinearly. Conversely, unrestricted nonlinear decoders are too permissive: if h_ℓ is an injective function of x , every function of x can be decoded from it, whether or not the network has computed it. A controlled hierarchy of decoders interpolates between the two, for instance polynomial decoders of a low degree k , which are linear decoders on $(h, h^{\otimes 2}, \dots, h^{\otimes k})$ so that Proposition 4.24 applies, or small multilayer perceptrons of fixed width; the score is then reported as a function of the complexity of the decoder. In every case, the same decoder should be applied to controls (Alain and Bengio, 2016): the input $h_0 = x$, the network at initialization, and *control targets* of the same complexity that play no role in the task, for instance the singular functions of an independently drawn teacher with the same tree and ranks. A representation that decodes the control targets as well as the teacher’s factors reflects the capacity of the probe and the richness of the representation rather than the learning of the task; this is the selectivity criterion of Hewitt and Liang (2019). The input baseline $R_0^2[\zeta]$ is essential. With linear leaf embeddings, the left factors at leaf bonds are linear functions of x and are perfectly decodable from the input itself, so only targets with a small input baseline say something about what the network has computed. Finally, decodability shows that a feature is present, not that it is used (Surrogate note, 2026, Section 7.3).

Remark 4.26 (Dependent blocks create a predictability confound). When the blocks are dependent, a representation can decode a function of some blocks without receiving them, through their dependence on the blocks it does receive. If $h = h(x_A)$ depends only on x_A and $\zeta = \zeta(x_{A^c})$, then

$$R_h^2[\zeta] \leq \frac{\text{Var}(\mathbb{E}[\zeta \mid X_A])}{\text{Var} \zeta},$$

the fraction of the variance of ζ that is predictable from X_A , because no function of X_A predicts ζ better than $\mathbb{E}[\zeta \mid X_A]$; under the product law this bound is zero. For a TTN student, whose bond states only see the blocks below them (Section 4.9), this bound is the baseline against which

decoding a right factor $\psi_{e,q}^*$ from a bond state must be compared. A DNN layer sees the whole input, and the analogous control is the *residual target* $\zeta - \mathbb{E}[\zeta \mid X_{A_e}]$, which is uncorrelated with every function of x_{A_e} : a positive score for the residual of a right factor shows that the layer carries information about the other side of the cut beyond what x_{A_e} predicts. The conditional expectations are known analytically for Gaussian blocks, and are otherwise estimated by nonparametric regression on held-out data.

For a DNN, one may expect the order of the layers to reflect the order of the bonds in the tree. To state this precisely, let the *height* $\text{ht}(e)$ of a bond be the height of the subtree below it, with $\text{ht}(e) = 1$ for a leaf bond, and let

$$\ell_{e,q}(t; \delta) := \min \{ \ell \geq 1 : R_\ell^2[\phi_{e,q}^*](t) \geq \delta \} \quad (\min \emptyset = +\infty)$$

be the first layer in which the left factor of the atom (e, q) is explicitly represented at time t .

Hypothesis 4.27 (Layer–bond correspondence). In a DNN student trained on a TTN teacher, once the factors have been acquired, $\ell_{e,q}(t; \delta)$ is nondecreasing in $\text{ht}(e)$: the factors of bonds closer to the leaves are first represented in earlier layers, and the factors of bonds closer to the root in later layers.

The first layer above the threshold, rather than the layer with the highest score, is the right statistic, because a feature computed early can remain decodable in all later layers, for instance through residual connections. The hypothesis is natural in view of (4.8): once the factors at the two bonds below e are linearly available, the factors at e are bilinear functions of them, which one layer with quadratic activations makes exactly available, and one layer with a standard activation approximately. It can be tested with the rank correlation between $\text{ht}(e)$ and $\ell_{e,q}$ across atoms with a small input baseline. In a TTN student on the teacher’s tree, the correspondence is built into the architecture (Section 4.9); in a DNN it has to be discovered, and whether it is discovered is part of the comparison.

4.8 Use and causal reliance

A feature can be represented without being used: residual connections and wide layers can carry many decodable features that the downstream computation ignores. Conversely, a student can contain an atom in its output without any single layer encoding it linearly. We therefore add observables of *use*, defined by derivatives of the downstream computation, and of *causal reliance*, defined by interventions. Raw weights are not suitable for this purpose. They are not invariant under the symmetries of hidden units (Remark 2.26), and unsupervised spectra of weight or activation Gram matrices can change considerably without revealing which computation has been learned; they should be paired with task-conditioned statistics (Literature review, 2026). All the expectations below are taken over ordinary samples of the data, whatever their law.

Write $f_\theta = f_{>\ell} \circ h_\ell$, where $f_{>\ell} : \mathbb{R}^{d_\ell} \rightarrow \mathbb{R}$ is the map computed by the layers above ℓ , and use the whitened representation $h_\ell^w := (\text{Cov}(h_\ell)^{1/2})^+(h_\ell - \mathbb{E}h_\ell)$, with the moments of the current checkpoint. In whitened coordinates, every unit direction in the range of $\text{Cov}(h_\ell)$ has unit variance and orthogonal directions are uncorrelated, so that a perturbation of a given size means the same thing in every direction. For a target ζ with $R_\ell^2[\zeta] > 0$, the *code direction*

$$u_\ell[\zeta] := \frac{\mathbb{E}[h_\ell^w \zeta]}{\|\mathbb{E}[h_\ell^w \zeta]\|}$$

is the direction of the optimal linear decoder (Proposition 4.24(i)), and the code $u_\ell[\zeta]^\top h_\ell^w(x)$ has unit variance and squared correlation $R_\ell^2[\zeta]$ with ζ . For a vector target Z , the code directions form an orthonormal basis $U_\ell[Z]$ of the span of the columns of $\mathbb{E}[h_\ell^w Z^\top]$.

Definition 4.28 (Downstream sensitivity). Let $\partial_\ell f(x) := \text{Cov}(h_\ell)^{1/2} \nabla f_{>\ell}(h_\ell(x))$ be the gradient of the output with respect to the whitened representation, and let $\text{AGOP}_\ell := \mathbb{E}[\partial_\ell f(X) \partial_\ell f(X)^\top]$ be the corresponding average gradient outer product (cf. Radhakrishnan et al., 2024). The *sensitivity* of the output to the code of ζ , and its normalized version, are

$$\text{sens}_\ell[\zeta] := u_\ell[\zeta]^\top \text{AGOP}_\ell u_\ell[\zeta] = \mathbb{E}[(\partial_\ell f(X)^\top u_\ell[\zeta])^2], \quad \overline{\text{sens}}_\ell[\zeta] := \frac{\text{sens}_\ell[\zeta]}{\text{tr}(\text{AGOP}_\ell) / \text{rank Cov}(h_\ell)},$$

and, for a vector target, $\text{sens}_\ell[Z] := \text{tr}(U_\ell[Z]^\top \text{AGOP}_\ell U_\ell[Z])$.

The sensitivity is the mean squared first-order change of the output per standard deviation of the code. The denominator of $\overline{\text{sens}}_\ell$ is the average of $u^\top \text{AGOP}_\ell u$ over uniformly random unit directions u in the range of the whitened representation, so $\overline{\text{sens}}_\ell[\zeta] > 1$ means that the downstream computation is more sensitive to the code than to a typical direction. A high $R_\ell^2[\zeta]$ together with $\overline{\text{sens}}_\ell[\zeta] \lesssim 1$ indicates a feature that is present but not preferentially used. For orientation: if h_ℓ contained $\phi_{e,q}^*$ as a coordinate uncorrelated with the other coordinates, and $f_{>\ell}$ used it as the teacher does, through the term $\sigma_{e,q}^* \phi_{e,q}^* \psi_{e,q}^*$, then $\text{sens}_\ell[\phi_{e,q}^*] = \sigma_{e,q}^{*2} \text{Var } \phi_{e,q}^*$. For a polynomial network, the eigendecomposition of AGOP_ℓ , augmented by the constant direction, is the data-weighted computational HSVD at height ℓ (Proposition 4.22).

The sensitivity has no sign. To know whether strengthening the code would increase the atom in the output, inject the atom along its code direction, $h_\ell^w(x) \mapsto h_\ell^w(x) + \varepsilon \chi_{e,q}^*(x) u$ with $u = u_\ell[\chi_{e,q}^*]$, and let $f^{(\varepsilon)}$ be the resulting output. Since $\frac{d}{d\varepsilon} f^{(\varepsilon)}(x)|_{\varepsilon=0} = \chi_{e,q}^*(x) \partial_\ell f(x)^\top u$ by the chain rule, and $f \mapsto a_e[f]$ is linear, the *gain*

$$\text{gain}_\ell(e, q) := \left. \frac{d}{d\varepsilon} \right|_{\varepsilon=0} a_{e,q}[f^{(\varepsilon)}] = a_{e,q}[\chi_{e,q}^* \partial_\ell f^\top u]$$

is positive when a slightly stronger internal code would increase the output acquisition coefficient. When the two sides of the cut are independent, $\text{gain}_\ell(e, q) = \mathbb{E}[\chi_{e,q}^*(X)^2 \partial_\ell f(X)^\top u] / \sigma_{e,q}^*$. The gain is meaningful only once the atom is represented, i.e. once $R_\ell^2[\chi_{e,q}^*]$ is clearly above its controls (Remark 4.25), since before that the code direction is dominated by noise. For a block B , the same definition applies to the normalized block atom $m_{e,B}^* / \|m_{e,B}^*\|$ and to $a_{e,B}$; both are invariant under the rotations of Remark 4.13.

Remark 4.29 (Two other Jacobian observables). (a) *Transmission*. Replacing $f_{>\ell}$ by the map from h_ℓ to the whitened representation of the next layer measures whether that layer transmits the code: $\mathbb{E}[(\text{Cov}(h_{\ell+1})^{1/2})^+ J_{\ell \rightarrow \ell+1} \text{Cov}(h_\ell)^{1/2} u]^2$, with $J_{\ell \rightarrow \ell+1} = \partial h_{\ell+1} / \partial h_\ell$, compared with the same quantity for random directions u . Unlike $\|W_{\ell+1} u\|$, it includes the current derivatives of the activations. (b) *Input side*. If ζ is a differentiable function of h_ℓ , then by the chain rule $\nabla_x \zeta$ lies in the row space of $\partial h_\ell / \partial x$, so a fraction equal to one of $\|\nabla_x \zeta\|^2$ lying in that row space, at almost every x , is necessary for ζ to be an exact differentiable function of h_ℓ . It is uninformative whenever the layer is at least as wide as the input and its Jacobian has full column rank, which is the typical case, so we do not use it as a primary observable.

Interventions. Neither projections nor derivatives show that a student relies on a feature: a derivative is local, and a code to which the output is sensitive may be bypassed. Causal tests intervene on the code and measure the response.

Definition 4.30 (Erasure and injection). Fix a layer ℓ and a target ζ , which may be a factor, an atom or the vector target of a block, with code directions $U = U_\ell[\zeta]$. The *erasure* of ζ at layer ℓ replaces h_ℓ^w by $(I - UU^\top)h_\ell^w$, that is,

$$h_\ell \mapsto h_\ell - \text{Cov}(h_\ell)^{1/2} UU^\top h_\ell^w,$$

and passes the result to the layers above ℓ . The *injection* of a scalar target ζ along a unit direction u in the range of the whitened representation replaces $h_\ell^w(x)$ by $h_\ell^w(x) + \varepsilon \zeta(x) u$; a vector target is injected along an orthonormal family of directions. Once ζ is represented, u can be its code direction $u_\ell[\zeta]$; before that, the code direction is undefined or dominated by noise, and u is a fixed direction chosen independently of the data, for instance at random. Either intervention can be applied once, at a checkpoint, or kept in place during training from some time on.

Proposition 4.31 (Erasure removes all linear information). If h'_ℓ denotes the erased representation, then $\text{Cov}(h'_\ell, \zeta) = 0$ (and $\text{Cov}(h'_\ell, Z) = 0$ for a vector target). Hence $R_{h'_\ell}^2[\zeta] = 0$: no affine function of h'_ℓ predicts ζ better than a constant.

Proof. Since $\mathbb{E}[h_\ell^w \zeta]$ lies in the span of U , $\text{Cov}(h'_\ell, \zeta) = \text{Cov}(h_\ell, \zeta) - \text{Cov}(h_\ell)^{1/2} \mathbb{E}[h_\ell^w \zeta] = \text{Cov}(h_\ell, \zeta) - \text{Cov}(h_\ell)^{1/2} (\text{Cov}(h_\ell)^{1/2})^+ \text{Cov}(h_\ell, \zeta) = 0$, because $\text{Cov}(h_\ell, \zeta)$ lies in the range of $\text{Cov}(h_\ell)$ (proof of Proposition 4.24). The conclusion follows from Proposition 4.24(i). $\square$

This erasure is the closed-form linear concept erasure (LEACE) of Belrose et al. (2023), which removes all linear information about ζ with the smallest possible change of the representation, in the sense made precise there. This is why the projection is done in whitened coordinates; it also replaces the code by its mean rather than by zero. Information about ζ can survive nonlinearly, so a null effect of an erasure is weaker evidence than a positive one.

What to measure. After an erasure at a checkpoint, measure the immediate response: the change of the coupling matrices $C_e[f]$ and of the acquisition coefficients, in particular the drop of $a_{e,q}$, the change of the loss, and the collateral changes of all other atoms. When the erasure is kept during training, measure the subsequent response, chiefly the acquisition times of the atoms at the bonds above e . By (4.8), the factors of these atoms are built from the left factors at e , not from the atoms at e , which also contain the right factors $\psi_{e,q}^*$ of the environment; the target to erase for this purpose is therefore the left factor $\phi_{e,q}^*$, or the vector $(\phi_{e,q}^*)_{q \in B}$ for a block. The moments $\mathbb{E}h_\ell$ and $\text{Cov}(h_\ell)$ and the code directions can then be frozen at the checkpoint, which applies a fixed affine map to the network, or re-estimated at every checkpoint. Proposition 4.31 holds for the moments with which the erasure is computed, so only the second version keeps the feature linearly erased while the student changes and possibly re-encodes it in other directions, and only the second comes close to blocking the feature; the two versions answer different questions. An injection of $\phi_{e,q}^*$ made before it is represented naturally asks the converse questions: does $a_{e,q}$ rise, do the atoms above e appear earlier, does the loss decrease faster, or does the student ignore the injected code? Immediate and subsequent responses should be reported separately (Surrogate note, 2026, Section 7.6).

Relation with acceleration. Kept in place during training, the erasure and the injection of a left factor $\phi_{e,q}^*$ implement, for a DNN, the “blocked” and “learned” interventions of Definition 1.1 on the earlier feature, the later feature being an atom (e', q') at a bond e' above e , measured by $a_{e',q'}$ with target value 1. In a TTN student, the same questions are asked by clamping cores (Diagonal TTN note, 2026, Section 3), or by applying the same erasure to the representation at a bond (Section 4.9). This is what makes the comparison mechanistic: at this level, a TTN is a faithful surrogate of a

DNN only if interventions on the same teacher feature have comparable effects on both ([Surrogate note, 2026](#), Section 7.6). A complementary intervention acts on the teacher instead of the students and does not require locating a code: weaken a prerequisite atom, for instance by replacing $f^\star$ with $f^\star - \frac{1}{2}\sigma_{e,q}^\star\chi_{e,q}^\star$, train both students on the modified teacher, and compare the shifts of the acquisition times of the atoms above e . Since the modification also changes the data-weighted HSVD at the other bonds, the atoms of the modified teacher must be recomputed; what matters is that both students are compared on the same modified teacher.

Controls. Every intervention needs controls: the same intervention along random directions of the same dimension in whitened coordinates, perturbations of matched norm, and sham interventions that run the same machinery with $U = 0$ ([Diagonal TTN note, 2026](#), Section 3.3). The data used to estimate the whitening and the code directions should be disjoint from the data on which effects are measured. Temporal order alone is not evidence of dependence: two atoms with different strengths are learned at different times even when they are independent ([Diagonal TTN note, 2026](#), Section 3.2). Only interventions separate such staging from a genuine dependence of one acquisition on another.

4.9 Observables for TTN students

All the observables above are defined for every student. For a TTN student g_ϑ on the teacher's tree with the same leaf embeddings, many of them can be computed exactly, to an extent that depends on the law of the data.

Product law.

- (i) *Functional HSVD.* By Proposition 4.2(iv), the functional HSVD (4.12) of g_ϑ is its data-weighted HSVD, which is computed exactly by the moment recursion of Remark 4.7, without crossed grids.
- (ii) *Coupling matrices.* Let $\tilde{M}_e(\vartheta) := \tilde{M}_e(T(\vartheta))$ be the data-weighted matricization of the student. Then $C_e[g_\vartheta] = U_e^{\star\top} \tilde{M}_e(\vartheta) V_e^\star$, which is computed exactly by contracting the student with the two orthonormalized halves of the teacher, and by (4.7), $\|g_\vartheta - f^\star\|^2 = \text{vec}(T(\vartheta) - T^\star)^\top (\bigotimes_a \Sigma_a) \text{vec}(T(\vartheta) - T^\star)$. The decomposition (4.11) is therefore exact, with no sampling error.
- (iii) *Representations.* The role of a hidden layer is played by the message that the subtree below a bond e sends through it, $h_e(x_{A_e}) = F_e^\top \Xi_{A_e}(x_{A_e}) \in \mathbb{R}^{D_e}$, where F_e is the contraction of the student's cores below e (Lemma 2.8). Unlike a hidden layer of a DNN, h_e only sees the blocks below e , and by independence it is uncorrelated with every function of the other blocks. No bond sees all the blocks, and no bond sees exactly the blocks of A_e^c unless e is a child bond of the root. Hence the atoms $\chi_{e,q}^\star$ cannot be fully read off any bond, and neither can the right factors $\psi_{e,q}^\star$, except at a child bond of the root, whose right factors are functions of the blocks below the other child bond. Representation observables at bonds therefore concern left factors: the probes $R_{h_{e'}}^2[\phi_{e,q}^\star]$, for the bond $e' = e$ and for the bonds e' above e , and the corresponding code directions and erasures. Since $g_\vartheta(x) = h_e(x_{A_e})^\top n_e(x_{A_e^c})$, with $n_e = H_e^\top \Xi_{A_e^c}$ the message that the rest of the network sends through e , the gradient of the output with respect to the whitened h_e , the other blocks being fixed, is $\text{Cov}(h_e)^{1/2} n_e(x_{A_e^c})$; it replaces $\partial_\ell f$ in the sensitivities. All these moments are computable exactly from the first and second moments $\mathbb{E}\xi_a$ and Σ_a of the embeddings. Finally, by Proposition 4.6, the student's own data-weighted features at e are linear readouts of h_e , as in Proposition 2.35(ii).

For (ii), Proposition 4.2(ii) applied to the student gives $g_\vartheta = \tilde{\Xi}_{A_e}^\top \tilde{M}_e(\vartheta) \tilde{\Xi}_{A_e}$ almost surely. Multiply by $\phi_{e,i}^* \psi_{e,j}^* = (u_{e,i}^\top \tilde{\Xi}_{A_e})(v_{e,j}^\top \tilde{\Xi}_{A_e})$ and take expectations: by independence, and since $\mathbb{E}[\tilde{\Xi}_A \tilde{\Xi}_A^\top]$ is the orthogonal projector onto the range of $\Sigma_A^{1/2}$ (proof of Proposition 4.2), which contains the singular vectors of the teacher, $C_e[g_\vartheta]_{ij} = u_{e,i}^\top \tilde{M}_e(\vartheta) v_{e,j}^*$.

Dependent blocks. The algebraic structure is unchanged: $g_\vartheta(x) = h_e(x_{A_e})^\top n_e(x_{A_e})$ at every bond, and by Propositions 4.2(iii) and 4.6, the functional HSVD of the student is still its data-weighted HSVD, obtained from the $D_e \times D_e$ matrices $\mathbb{E}[h_e h_e^\top]$ and $\mathbb{E}[n_e n_e^\top]$. These contain mixed moments of the embeddings within each side, and are estimated by Monte Carlo. The same computation as above shows that $U_e^{\star\top} \tilde{M}_e(\vartheta) V_e^*$ is now the matrix of inner products $\mathbb{E}_{\nu_e^\otimes}[g_\vartheta \phi_{e,i}^* \psi_{e,j}^*]$ in the decoupled geometry, not the on-distribution coupling matrix $C_e[g_\vartheta]$, which requires moments across the cut, as do Γ_e and the loss. Exact computations survive when the law has a compatible structure, for instance for a finite mixture of product laws (Remark 4.7). A bond state h_e is no longer independent of the blocks outside A_e : it can predict functions of them, including right factors, through the dependence of the blocks, up to the bound of Remark 4.26.

Repeated inputs. For the tied TTN of a polynomial network, the subtree state at height ℓ is the augmented hidden state $\bar{h}_\ell(x)$, and the gradient with respect to a shared core is the sum of its gradients over all occurrences. The loss is governed by the moment matrix $\Sigma_{[N]}$ of Section 4.6. The Euclidean HSVD of Definition 2.34 is computed exactly from the cores, and the data-weighted computational HSVD from samples, by Proposition 4.22; neither is an invariant of the function.

The difference between the two kinds of students is now clear. In a TTN student on the teacher's tree, the correspondence between the teacher's bonds and the student's representations is fixed by the architecture, and the question is only when each left factor and each atom are acquired. In a DNN student, this correspondence has to be discovered (Hypothesis 4.27), and the atoms need not be localized in any layer. A TTN on a wrong tree is an intermediate control: its output observables are those of any student, but its bonds do not match those of the teacher.

4.10 Acquisition times, comparison of students, and estimation

Definition 4.32 (Acquisition times). Fix a threshold $\delta \in (0, 1)$ and a persistence window $\Delta \geq 0$. The *output acquisition time* of a block B at the bond e and the *representation acquisition times* of the atom (e, q) are

$$\begin{aligned} \tau_{e,B}^{\text{out}}(\delta) &:= \inf \{t \geq 0 : a_{e,B}(s) \geq \delta \text{ for all } s \in [t, t + \Delta]\}, \\ \tau_{e,q}^\zeta(\delta) &:= \inf \left\{ t \geq 0 : \max_{\ell \geq 1} R_\ell^2[\zeta](s) \geq \delta \text{ for all } s \in [t, t + \Delta] \right\}, \quad \zeta \in \{\phi_{e,q}^*, \psi_{e,q}^*, \chi_{e,q}^*\}, \end{aligned}$$

with $\inf \emptyset = +\infty$; we write $\tau_{e,q}^{\text{out}}$ when $B = \{q\}$, and use the vector targets $R_\ell^2[Z]$ for blocks. For a TTN student, $\tau_{e,q}^{\phi_{e,q}^*}$ is defined with a maximum over bonds instead of layers. Under the product law, $\tau_{e,q}^{\psi_{e,q}^*}$ and $\tau_{e,q}^{\chi_{e,q}^*}$ are then in general infinite, since a bond only sees the blocks below it; under dependence, they can be finite through the dependence of the blocks, and must be compared with the bound of Remark 4.26.

For $\Delta = 0$ and independent sides of the cut, $\tau_{e,q}^{\text{out}}$ is the acquisition time $\tau_m(\delta)$ of (1.1). The persistence window prevents noisy, transient crossings from being counted as acquisitions; with discrete checkpoints, it is a number of consecutive checkpoints. A time that is not observed before the end of training is reported as right-censored, not dropped (Surrogate note, 2026, Section 7.3).

When the acquisition coefficients are not identifiable (Proposition 4.16(ii)), acquisition is defined through a statistic of projections instead, such as $1 - \|P_{\mathcal{A}_e}(f_t - f^*)\|^2 / \|f^*\|^2$, and the conditioning of Λ_e is reported. For teachers with repeated inputs, the same definitions are used with the degree coefficients $a_k(t)$ and the representation targets of Section 4.6.

Choosing the thresholds. No threshold is canonical. One should report the curves $a_{e,q}(t)$ and $R_\ell^2[\zeta](t)$ themselves, together with acquisition times for several thresholds, for example $\delta \in \{0.1, 0.25, 0.5\}$ (Surrogate note, 2026, Section 7.3), and draw only conclusions that hold over this range. In saddle-to-saddle dynamics, transitions are typically sharp and well separated, so that orders of acquisition do not depend on δ ; when two curves cross, the order depends on δ and should be reported as unresolved. For representation scores, the threshold must in addition exceed the input and initialization baselines, the scores of the control targets of Remark 4.25 and, under dependence, the bounds of Remark 4.26.

Comparing a TTN and a DNN. All the observables are functions of the predictor, or of a representation and of fixed teacher targets, so they can be computed for both students and compared in the same time variable (Hypothesis 1.2). The comparison has several levels:

- (i) *trajectories*: $\sup_t |a_{e,q}^{\text{DNN}}(t) - a_{e,q}^{\text{TTN}}(t)|$, or a root mean square over t , for every atom, and the time course of the four terms of (4.11) at every bond;
- (ii) *orders*: the rank correlation, for instance Spearman’s, of the acquisition times $\tau_{e,q}^{\text{out}}(\delta)$ across atoms, and whether atoms at lower bonds are acquired before atoms at higher bonds in both students;
- (iii) *delays*: the delay $\tau_{e,q}^{\text{out}} - \tau_{e,q}^{\phi_{e,q}^*}$ between the representation of the left factor and the appearance of the atom in the output, which is defined for both students; for a DNN, also whether the factors are represented before their product;
- (iv) *interventions*: whether erasing or injecting the same left factor, or weakening the same teacher atom, shifts the same acquisition times by comparable amounts in both students.

Functional HSVDs, through singular values and alignments, complement (i) and (ii) with teacher-free decompositions; under dependence, they compare the students on recombined inputs (Section 4.5). For teachers with repeated inputs, the same comparisons are made with the degree components in place of the atoms, and with AGOP spectra as computation-level observables.

Estimation protocol.

1. *Teacher*. Under the product law, compute the Σ_a analytically, or estimate them on a large sample independent of all other data, and compute the data-weighted HSVD at every bond exactly (Remark 4.7). Under dependence, estimate the moments Σ_e^h and Σ_e^n of the teacher’s bond messages on such a sample (Proposition 4.6). Record the singular values, their gaps, and the blocks of singular values whose gaps are below the estimation error. Estimate the Gram matrices Γ_e and Λ_e once, on the same independent sample ($\Gamma_e = I$ exactly under the product law); report their condition numbers, regularize small eigenvalues, and merge blocks whose atoms are nearly collinear.
2. *Output observables*. Fix a held-out audit set $x_1, \dots, x_M$ of ordinary samples, independent of the training data, and use it at every checkpoint; reusing the same inputs reduces the noise of differences over time. Estimate the inner products $\mathbb{E}[f_t \chi_e^*]$ and $(\langle f_t, m_{e,B}^* \rangle)_B$, and $\|f_t\|^2$, on this set, with error bars from a bootstrap over audit examples or from independent audit sets. These estimates are unbiased, with error $O(M^{-1/2})$ when the integrands have finite variance (Diagonal TTN note, 2026, Section 9.1), and so are the estimates of $C_e[f_t]$ and $a_e[f_t]$, which are fixed linear

maps of them once Γ_e and Λ_e are known. The four terms of (4.11) are quadratic, and plugging in estimates biases them. Under the product law, for instance, $\mathbb{E}\|\hat{C} - S_e^*\|_F^2$ exceeds $\|C - S_e^*\|_F^2$ by the sum of the variances of the entries of $\hat{C}$, of order r_e^2/M , so the amplitude, within-block and across-block terms are overestimated, the outside term is underestimated, and the across-block term has a positive floor even when the student pairs the factors correctly. Sample splitting removes this bias. Each term is a difference of quantities of the form $\|Pg\|^2 = c[g]^\top G^+ c[g]$, where P is the orthogonal projector onto the span of a family of functions with Gram matrix G , and $c[g]$ is the vector of inner products of g with that family. If $\hat{c}^{(1)}$ and $\hat{c}^{(2)}$ are estimates of $c[g]$ computed on two disjoint halves of the audit set, then $\hat{c}^{(1)\top} G^+ \hat{c}^{(2)}$ is an unbiased estimate of $\|Pg\|^2$; it can be negative when the true value is small. Under dependence, raw inner products must not be inserted into the formulas of Proposition 4.16(iv), and samples should be recombined only to estimate functional HSVDs, which are then labelled as such. For TTN students, use the exact formulas of Section 4.9 where they apply.

3. *Representation observables.* At each checkpoint and layer, collect $h_\ell(x_n)$ on separate sets for fitting probes, for validation and for testing. Fit ridge-regularized probes, choose the regularization on the validation set, and report the held-out R^2 . Held-out estimation is essential: the in-sample R^2 of an affine probe tends to 1 as d_ℓ approaches the number of probe examples, whatever the target. For blocks, use multivariate ridge regression on whitened targets, or canonical correlation analysis after reducing h_ℓ to its leading principal components, as in SVCCA (Raghu et al., 2017). Include the controls of Remark 4.25 and, under dependence, the bounds and residual targets of Remark 4.26; for teachers with repeated inputs, use the targets of Section 4.6.
4. *Use and interventions.* Since the output is scalar, one backward pass per audit example gives the gradient $\partial_\ell f(x)$, hence sens_ℓ , gain_ℓ and $\text{tr}(\text{AGOP}_\ell) = \mathbb{E}\|\partial_\ell f(X)\|^2$; Jacobian–vector products give transmissions without forming Jacobians.
5. *Statistics.* Pair the students: same teacher, same data and same seeds wherever pairing is meaningful. Use about five seeds to screen and at least twenty for confirmatory claims about orders of acquisition or effects of interventions, declare the primary bonds and atoms in advance, and report unresolved gaps and censored times (Surrogate note, 2026, Sections 9 and 10).

4.11 Summary

Table 1 collects the observables by level of evidence. The levels should not be collapsed: an atom can be present in the output without being localized in any layer, represented in a layer without being used, and used locally without the student relying on it. Table 2 collects what each regime of input laws allows. The hierarchical features of a TTN are its data-weighted singular functions; the decomposition of any student, DNN or TTN, is its functional HSVD; the two coincide for TTNs on the teacher’s tree. Under the product law both are Schmidt decompositions in the geometry of the loss; under dependence they live on recombined inputs, and the on-distribution comparison uses Gram-corrected projections; for repeated inputs, function-level and computation-level observables must be kept apart.

For an atom that a DNN acquires hierarchically, one expects the following sequence: its factors become decodable in some layer; then their product becomes decodable; the downstream computation becomes sensitive to its code; its output coefficient $a_{e,q}(t)$ grows towards 1; and erasing the code of its left factor removes the output component and delays the acquisition of the atoms above it. A TTN student on the teacher’s tree has no layer in which the product of the two factors could appear, but it has the same left factors at its bonds and the same output coefficients. The surrogate hypothesis is at its strongest if the trajectories of a TTN student predict, for a DNN student, when the left factors become accessible, when the output coefficients grow, and how interventions on one

Level	Question	Observables
Output	Does the predictor contain the atom, with the right factors, pairing and strength?	$a_{e,B}(t)$, $C_e[f_t]$ and the decomposition (4.11); functional HSVD $s_{e,q}(f_t)$ and alignment $\text{Al}_{e,r}(t)$; for repeated inputs, degree components
Representation	Can the factors and the atom be read off a layer? For a TTN: can the left factor be read off a bond?	$R_\ell^2[\phi_{e,q}^*]$, $R_\ell^2[\psi_{e,q}^*]$, $R_\ell^2[\chi_{e,q}^*]$; $R_\ell^2[Z]$ for blocks; first layer $\ell_{e,q}$; for a TTN, $R_{h_{e'}}^2[\phi_{e,q}^*]$; residual targets under dependence
Use	Is the downstream computation sensitive to the code, with the right sign?	sens_ℓ , $\overline{\text{sens}}_\ell$, $\text{gain}_\ell(e, q)$, transmission; spectra of AGOP_ℓ
Causal reliance	Does erasing or injecting the code of the left factor change the output, the loss, or the acquisition of the atoms above it?	erasure and injection, teacher-strength interventions; random-direction, matched-norm and sham controls

Table 1: Levels of evidence that a student has acquired an atom (e, q) of the teacher, and the corresponding observables.

feature change the acquisition of the others.

The methodological conclusion is two-tiered. Experiments should start with the product law, where the measurement apparatus can be validated against exact and additive decompositions. The same observables, with the Gram corrections and the predictability baselines, then carry over to dependent blocks, and the function and computation tracks to repeated inputs. There, success means agreement in the degree components of the output, in the hidden and computational feature subspaces, and in the responses to interventions; it does not identify copy-cut singular values with canonical features of the input–output function.

5 Experiments

The first order of buisness should be to have a much better understanding of the training dynamics of TTNs empirically and be systematic about it. Furthermore to understand the gap between TTNs and DNNs we should also compare their training dynamics in a range of setups.

We are going to work in teacher-student setups with a known ground truth coming from the teacher. We want to investigate different architectures, initialisations, data structure, optimizers, and eventually loss functions.

What phenomena do we want to investigate: saddle to saddle regime, intermediary regime (not too small, not too large init), lazy (large init) regime. What features of the TTN does a student TTN learns, in what order, what are the acquisition time, when we change the data distribution how do acquisition times differ. We also want to investigate other interventions: comparing the trajectories on GD and SG in a given setup, changing batch sizes, studying other optimizers (AdamW, muon), adding weight decay. We want to vary the architectures: TTNs without copy, TTNs with copying, Deep quadratic neural nets, ReLU and other realistic MLPs, adding attention (will require some thinking about how to change the data structure).

An example of a setup to start with. Consider some input data sampled from a gaussian with some covariance matrix Σ_x . Generically Σ_x is not necessarily invertible (we can start with the identity and change this). For a start, we consider a teacher matrix TTN with bond dimension

Object	Product law	Dependent blocks	Repeated inputs
Teacher features	Data-weighted HSVD: the HSVD of one weighted TTN; Schmidt decompositions in $L^2(\nu_x)$; orthonormal composition	Data-weighted SVDs: Schmidt decompositions on recombined inputs; one whitening per bond; composition if the Gram matrices are nonsingular; properties of the data under (4.9)	Data-weighted computational HSVD = eigendecomposition of the augmented AGOP; a property of the computation, not of the function
Output observables	Coupling matrix of inner products; additive per-atom errors; optimal truncations	Gram-corrected coupling matrix and projection coefficients; nested decomposition (4.11); identifiable if Γ_e, Λ_e are nonsingular	Degree components (Wiener chaos for Gaussian inputs) and their coefficients
Student decomposition	Functional HSVD in the geometry of the loss; crossed grid	Functional HSVD on recombined inputs; close to the data if $w \geq c > 0$	No physical cut; degree decomposition; crossed grids not meaningful
Representation and use	Probes, sensitivity, erasure, injection; bond states independent of the environment	Same, on joint samples; predictability bounds and residual targets	Same; teacher hidden states and computational singular functions as targets; AGOP spectra
Exact TTN computations	Second-moment recursion on the tree	Monte Carlo for the bond-message moments; exact for mixtures of product laws	Moments up to order $2N$; Wick contractions for Gaussian inputs

Table 2: What each regime of input laws allows. Independence of the blocks is a source of orthogonality and of exact computability, not a prerequisite for comparing the training dynamics of TTNs and DNNs.

e.g. somewhere between $d_e = 10 - 1e4$ (we can play with this) and hierarchical singular values and vectors given by power law distributions (we can try different distributions but power law is a reasonable start because they mimic the structure of more realistic data). Chose a binary tree TTN teacher with depth between 3 and 10 (start with smaller depth). Then we want to train a student on that tree. Take a binary tree student TTN with bond dimension at least twice the bond dimension of the student but more can also be tried (10x for example). We want to be in the realizable setup in which the teacher can fully learn the student. Train with square-loss with SGD (or some variant, as long as optimizer is fixed). Try with and without weight decay.

During training track at least the following observables: loss, alignment between hierarchical singular vectors of the teacher and the student, hierarchical singular values of the student, acquisition times.

Vary the data structure via the structure of the teacher (its HSVDs) to see cleanly saddle to saddle and other regimes, maybe

Vary the student architecture: vary the depth and bond dimension, use a TTN without copying, use a TTN with copying, a deep quadratic networks (see the previous section for the observable to track to define the features and acquisition times), a ReLU network.

For a given architecture vary the optimizer: SGD with large batch size, small batch size, change the learning rate, use AdamW, use Muon. Compare their loss trajectories, the features that they learn (singular vectors and values).

6 Experiments (slop)

This section is to be completed. For reference, the experiments suggested in the previous sections are listed below. Detailed protocols, with teachers, observables, statistical analysis and decision criteria, are given in [Surrogate note \(2026, Sections 6–10\)](#) and [Local TN note \(2026, Sections 9–11\)](#).

- E0. *Algebraic check*. Verify Theorem 2.30 numerically on a tiny network in double precision: the recurrence, the tied-tree contraction and the explicit coefficient tensor must agree in value and in gradient with respect to θ . An exactly polynomial activation, such as $\sigma(z) = z + z^2$, must give the same polynomial for every centre, and untying the cores must break the agreement of gradients. This check must pass before any scientific comparison.
- E1. *Global comparison* (Hypothesis 1.2). On a teacher with known features, for example the staircase teacher $f^*(x) = \sum_m \sqrt{\lambda_m} \prod_{i \leq m} u_i^\top x^{(i)}$ on independent Gaussian blocks $x^{(i)}$ with unit vectors u_i , train a DNN and TTNs (on the physical tree, and the tied TTN of the polynomial proxy) on the same data with the same optimizer. Compare losses, function-space distances, acquisition times and orders (1.1), and tangent kernels (Remark 1.4).
- E2. *Local surrogates* (Hypothesis 1.3). At several checkpoints, build the order- k surrogates of Definition 2.43 for $k \in \{1, 2, 3, 4\}$; measure $\varepsilon_{0,j}$, $\varepsilon_{1,j}$ and β_j at the checkpoint; run blind rollouts from the same parameters and optimizer state. Report the largest horizon over which predictions, feature coefficients and gradients agree within tolerance, as a function of k , depth and activation spread.
- E3. *Polynomial proxies* (Definition 2.51). Train proxies with $K \in \{2, 3, 4\}$ alongside the original DNN, track $\sigma_{\ell,j}(t)$ and $r_\ell(t)$ to test for saddle-to-saddle (rank-incremental) dynamics, compare them with the same diagnostics for the local surrogates of the DNN, and compare feature acquisition with the DNN.
- E4. *Interpretability* (Question 2.41). Train an MLP on modular addition, with concatenated one-hot inputs and quadratic or smooth activations, for example the quadratic network of [Gromov \(2023\)](#). Compute the physical HSVD of its logit tensor (Example 2.42) and the HSVD of its computational TTN, or of its TTN surrogate for smooth activations, and compare both with the Fourier subspaces of the key frequencies.

References

- G. Alain and Y. Bengio. Understanding intermediate layers using linear classifier probes. arXiv:1610.01644, 2016.
- Y. Bai, B. Krause, H. Wang, C. Xiong, and R. Socher. Taylorized training: towards better approximation of neural network training at finite width. arXiv:2002.04010, 2020.
- N. Belrose, D. Schneider-Joseph, S. Ravfogel, R. Cotterell, E. Raff, and S. Biderman. LEACE: perfect linear concept erasure in closed form. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- N. Cohen, O. Sharir, and A. Shashua. On the expressive power of deep learning: a tensor analysis. In *Conference on Learning Theory (COLT)*, PMLR 49:698–728, 2016.
- C. Davis and W. M. Kahan. The rotation of eigenvectors by a perturbation. III. *SIAM Journal on Numerical Analysis*, 7(1):1–46, 1970.

- L. De Lathauwer, B. De Moor, and J. Vandewalle. A multilinear singular value decomposition. *SIAM Journal on Matrix Analysis and Applications*, 21(4):1253–1278, 2000.
- V. de Silva and L.-H. Lim. Tensor rank and the ill-posedness of the best low-rank approximation problem. *SIAM Journal on Matrix Analysis and Applications*, 30(3):1084–1127, 2008.
- Hierarchical Feature Learning Project. Diagonal tree tensor networks: hierarchical feature learning through shared factors and residual paths. Internal research note, 30 September 2026.
- T. Dooms, W. Gauderis, G. A. Wiggins, and J. Oramas. Compositionality unlocks deep interpretable models. arXiv:2504.02667, 2025.
- S. S. Du and J. D. Lee. On the power of over-parametrization in neural networks with quadratic activation. In *International Conference on Machine Learning (ICML)*, 2018.
- C. Eckart and G. Young. The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3):211–218, 1936.
- Z. Furman, S. Wäldchen, Y. Bei, and L. Hodgkinson. Benign loss landscapes can coexist with worst-case hardness. arXiv:2609.13057, 2026.
- L. Grasedyck. Hierarchical singular value decomposition of tensors. *SIAM Journal on Matrix Analysis and Applications*, 31(4):2029–2054, 2010.
- L. Grasedyck and W. Hackbusch. An introduction to hierarchical (H-) rank and TT-rank of tensors with examples. *Computational Methods in Applied Mathematics*, 11(3):291–304, 2011.
- A. Gromov. Grokking modular arithmetic. arXiv:2301.02679, 2023.
- W. Hackbusch. *Tensor Spaces and Numerical Tensor Calculus*. Springer, 2012.
- W. Hackbusch and S. Kühn. A new scheme for the tensor representation. *Journal of Fourier Analysis and Applications*, 15(5):706–722, 2009.
- J. Håstad. Tensor rank is NP-complete. *Journal of Algorithms*, 11(4):644–654, 1990.
- J. Hewitt and P. Liang. Designing and interpreting probes with control tasks. In *Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019.
- C. J. Hillar and L.-H. Lim. Most tensor problems are NP-hard. *Journal of the ACM*, 60(6):45, 2013.
- F. L. Hitchcock. The expression of a tensor or a polyadic as a sum of products. *Journal of Mathematics and Physics*, 6:164–189, 1927.
- S. Holtz, T. Rohwedder, and R. Schneider. On manifolds of tensors of fixed TT-rank. *Numerische Mathematik*, 120(4):701–731, 2012.
- H. Hotelling. Relations between two sets of variates. *Biometrika*, 28(3/4):321–377, 1936.
- J. Huang and H.-T. Yau. Dynamics of deep neural networks and neural tangent hierarchy. In *International Conference on Machine Learning (ICML)*, PMLR 119:4542–4551, 2020.
- A. Jacot, F. Gabriel, and C. Hongler. Neural tangent kernel: convergence and generalization in neural networks. In *Advances in Neural Information Processing Systems*, volume 31, 2018.
- A. Jacot, F. Ged, B. Şimşek, C. Hongler, and F. Gabriel. Saddle-to-saddle dynamics in deep linear networks: small initialization training, symmetry, and sparsity. arXiv:2106.15933, 2021.
- S. Janson. *Gaussian Hilbert Spaces*. Cambridge Tracts in Mathematics 129. Cambridge University Press, 1997.

- T. G. Kolda and B. W. Bader. Tensor decompositions and applications. *SIAM Review*, 51(3):455–500, 2009.
- V. Koltchinskii and E. Giné. Random matrix approximation of spectra of integral operators. *Bernoulli*, 6(1):113–167, 2000.
- J. M. Landsberg, Y. Qi, and K. Ye. On the geometry of tensor network states. *Quantum Information and Computation*, 12(3–4):346–354, 2012.
- L.-H. Lim. Tensors in computations. *Acta Numerica*, 30:555–764, 2021.
- Hierarchical Feature Learning Project. Deep hierarchical feature learning: literature review and research agenda. Internal project document, 2026.
- Hierarchical Feature Learning Project. Local polynomial tensor networks for deep neural network training: exact tree representations, controlled changes of local model, and a teacher–student experimental protocol. Internal research note, 1 October 2026.
- P. Morala, J. A. Cifuentes, R. E. Lillo, and I. Ucar. NN2Poly: a polynomial representation for deep feed-forward artificial neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 36(1):781–795, 2025.
- N. Nanda, L. Chan, T. Lieberum, J. Smith, and J. Steinhardt. Progress measures for grokking via mechanistic interpretability. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2301.05217.
- I. V. Oseledets. Tensor-train decomposition. *SIAM Journal on Scientific Computing*, 33(5):2295–2317, 2011.
- M. T. Pearce, T. Dooms, A. Rigg, J. M. Oramas, and L. Sharkey. Bilinear MLPs enable weight-based mechanistic interpretability. arXiv:2410.08417, 2024.
- S. Pesme and N. Flammarion. Saddle-to-saddle dynamics in diagonal linear networks. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- A. Power, Y. Burda, H. Edwards, I. Babuschkin, and V. Misra. Grokking: generalization beyond overfitting on small algorithmic datasets. arXiv:2201.02177, 2022.
- A. Radhakrishnan, D. Beaglehole, P. Pandit, and M. Belkin. Mechanism for feature learning in neural networks and backpropagation-free machine learning models. *Science*, 383(6690):1461–1467, 2024.
- M. Raghu, J. Gilmer, J. Yosinski, and J. Sohl-Dickstein. SVCCA: singular vector canonical correlation analysis for deep learning dynamics and interpretability. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- N. Razin, A. Maman, and N. Cohen. Implicit regularization in hierarchical tensor factorization and deep convolutional neural networks. In *International Conference on Machine Learning (ICML)*, PMLR 162:18422–18462, 2022.
- O. Roy and M. Vetterli. The effective rank: a measure of effective dimensionality. In *European Signal Processing Conference (EUSIPCO)*, pages 606–610, 2007.
- S. Sarao Mannelli, E. Vanden-Eijnden, and L. Zdeborová. Optimization and generalization of shallow neural networks with quadratic activation functions. In *Advances in Neural Information Processing Systems*, volume 33, 2020.
- A. M. Saxe, J. L. McClelland, and S. Ganguli. Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. In *International Conference on Learning Representations (ICLR)*, 2014. arXiv:1312.6120.
- A. M. Saxe, J. L. McClelland, and S. Ganguli. A mathematical theory of semantic development in deep neural networks. *Proceedings of the National Academy of Sciences*, 116(23):11537–11546, 2019.

- Y.-Y. Shi, L.-M. Duan, and G. Vidal. Classical simulation of quantum many-body systems with a tree tensor network. *Physical Review A*, 74:022320, 2006.
- E. M. Stoudenmire and D. J. Schwab. Supervised learning with tensor networks. In *Advances in Neural Information Processing Systems*, volume 29, 2016.
- Hierarchical Feature Learning Project. Tree tensor networks as global and local surrogates for deep neural network training dynamics: mathematical construction and a staged empirical program. Internal research note, 2 October 2026.
- L. R. Tucker. Some mathematical notes on three-mode factor analysis. *Psychometrika*, 31(3):279–311, 1966.
- A. Uschmajew and B. Vandereycken. The geometry of algorithms using hierarchical tensors. *Linear Algebra and its Applications*, 439(1):133–166, 2013.
- P.-Å. Wedin. Perturbation bounds in connection with singular value decomposition. *BIT Numerical Mathematics*, 12(1):99–111, 1972.
- Y. Zhang, A. M. Saxe, and P. E. Latham. Saddle-to-saddle dynamics explains a simplicity bias across neural network architectures. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2512.20607.